

# Lecture 8: Computational Cameras

## Signal Processing Introduction

Instructor: Katie Bouman

TAs: Brandon Zhao, Brady Bhalla, Zachary Huang

# Course announcements

- HW3 is due on Tuesday at 10 am.
- Start thinking about project ideas...project proposals with a literature review will be due with HW4

# Slide credits

Many of these slides/images were borrowed/adapted from:

- Ioannis (Yannis) Gkioulekas (CMU)
- Kris Kitani (15-463, Fall 2016).
- Fredo Durand (MIT).
- Bernd Girod (Stanford University).
- James Hays (Georgia Tech).
- Steve Marschner (Cornell University).
- Steve Seitz (University of Washington).

# Overview of today's lecture

- Convolution.
- Linear time-invariant filters
- Fourier series.
- Fourier transform.
- Frequency-domain filtering.
- If time: Sampling & aliasing.

Who has taken EE 111. Signal-Processing Systems  
and Transforms?

or an equivalent Signals & Systems course?



YES



No

Who has learned about Fourier Transforms  
in a class?



YES



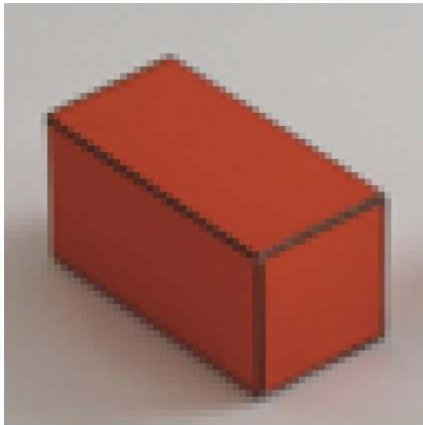
No

## 3+ Main Takeaways

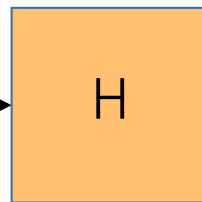
- A linear time-invariant system is fully characterized by its impulse response
- You can represent any signal as a sum of complex exponentials
- Convolution is multiplication in the frequency domain

Linear shift-invariant image filtering

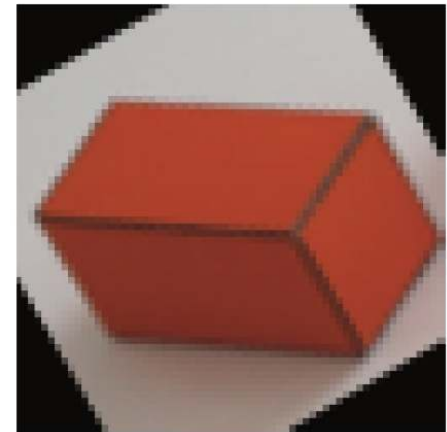
# System



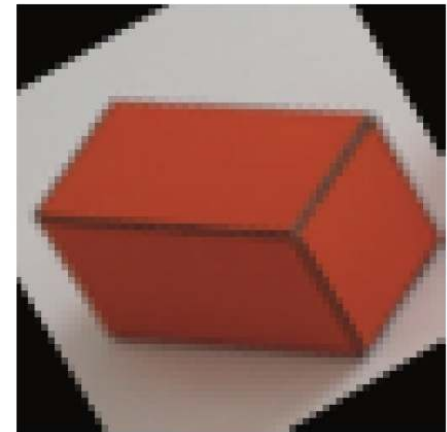
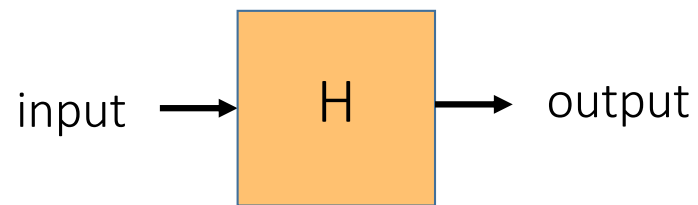
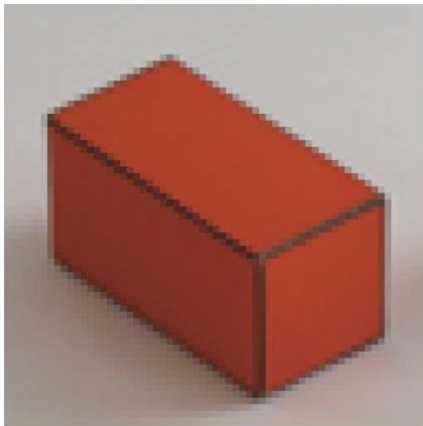
input



output



# Linear System

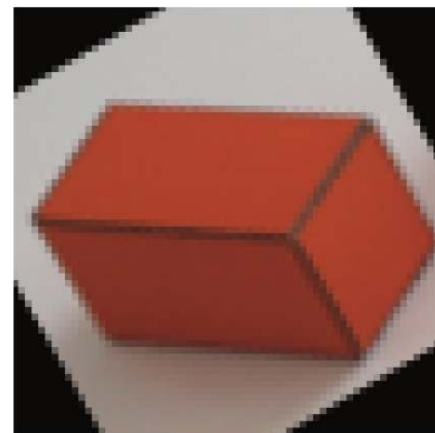
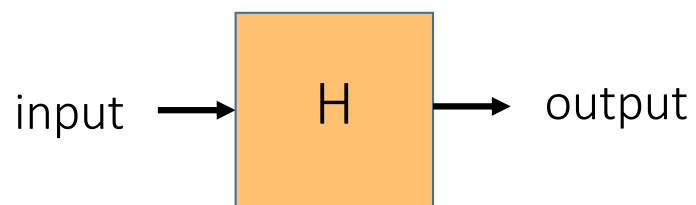
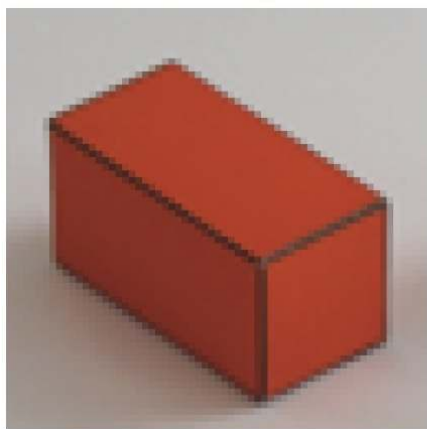


For a filter,  $H$ , to be linear, superposition must hold:

Additivity:  $H(a[m] + b[m]) = H(a[m]) + H(b[m])$

Homogeneity:  $H(c a[m]) = c H(a[m])$

# Linear System

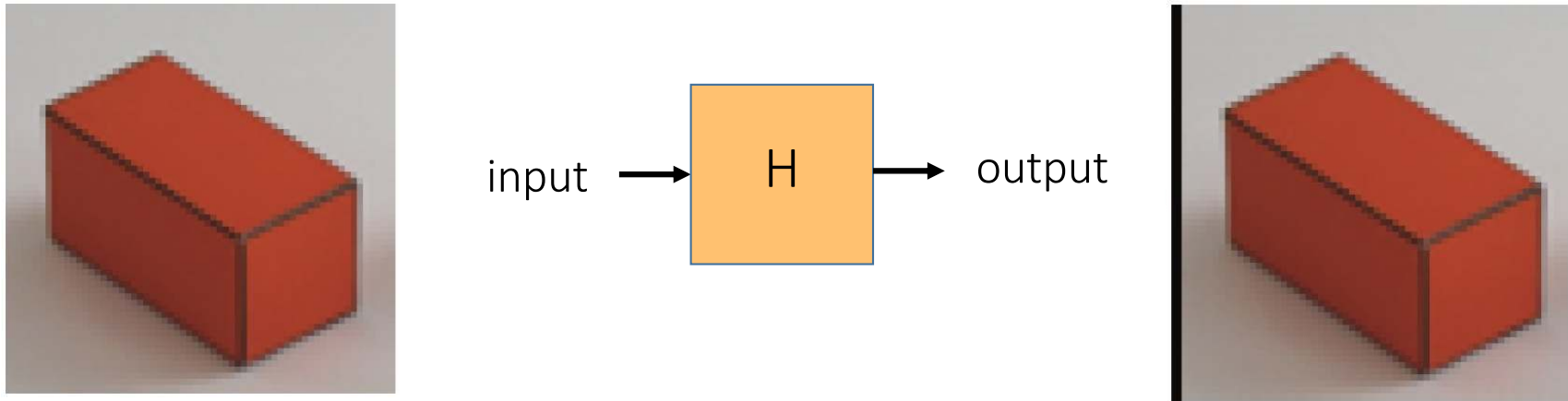


For a filter,  $H$ , to be linear, superposition must hold:

Additivity:  $H(a[m, n] + b[m, n]) = H(a[m, n]) + H(b[m, n])$

Homogeneity:  $H(c a[m, n]) = c H(a[m, n])$

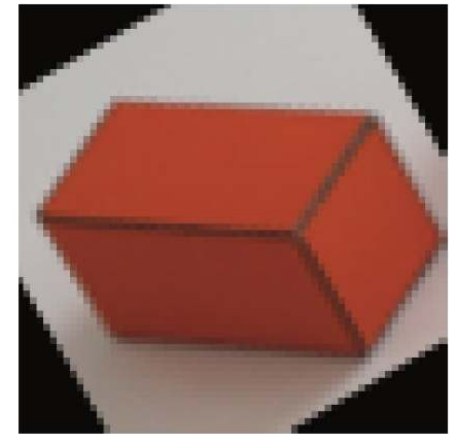
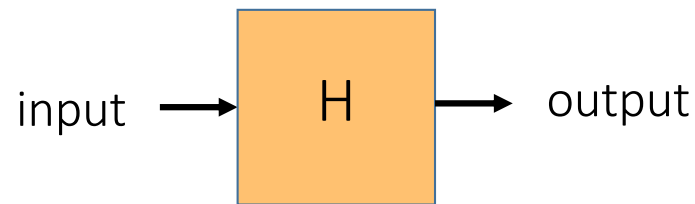
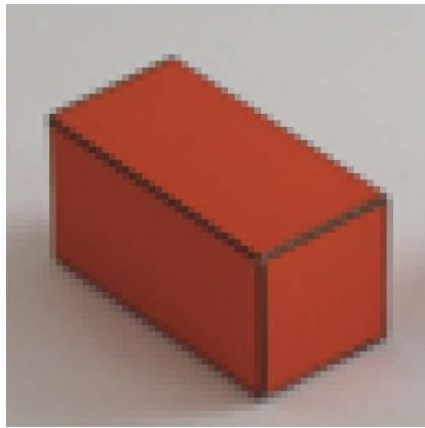
# Linear Time/Shift-Invariant System (LTI)



For a filter,  $H$ , to be time-invariant then:

$$\underbrace{H(a[m - t])[n]}_{\text{The output of a shifted input}} = \underbrace{H(a[m])[n - t]}_{\text{The shift of the output of the original input}}$$

# Linear Time/Shift-Invariant System (LTI)



For a filter,  $H$ , to be time-invariant then:

Is this time/shift invariant?



YES



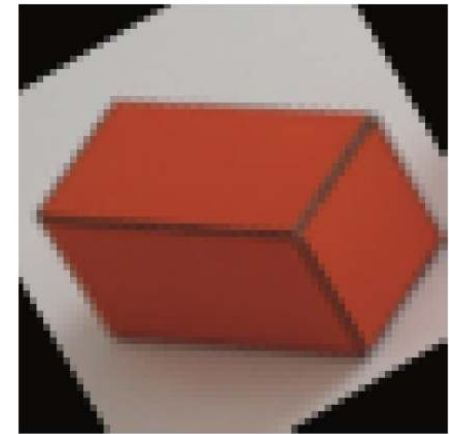
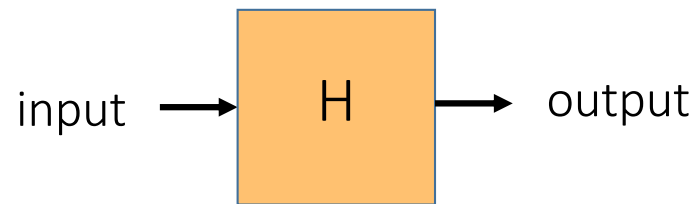
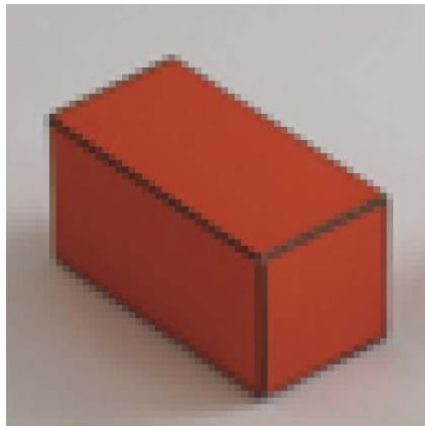
No

$$\underbrace{H(a[m - t])[n]}_{\text{The output of a shifted input}} = \underbrace{H(a[m])[n - t]}_{\text{The shift of the output of the original input}}$$

The output of a shifted input

The shift of the output of the original input

# Linear Time/Shift-Invariant System (LTI)



Is this time/shift invariant?

For a filter,  $H$ , to be time-invariant then:

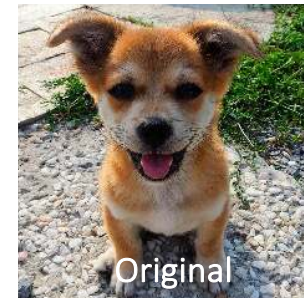
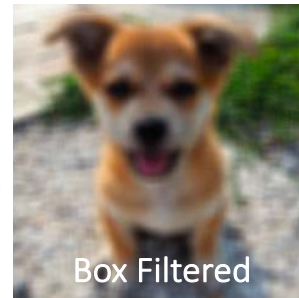
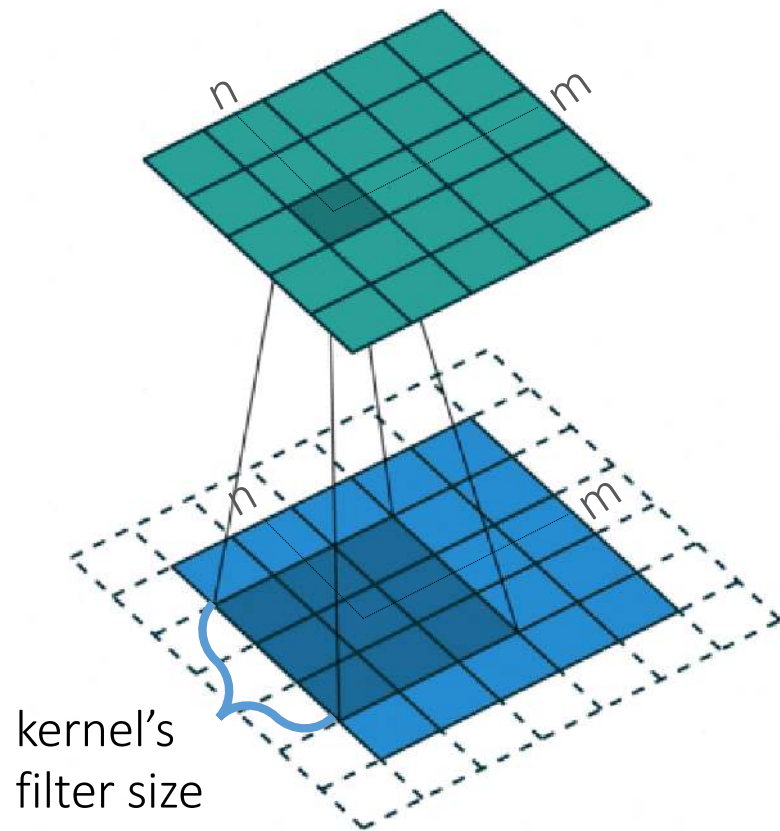
 No

$$\underbrace{H(a[m - t])[n]}_{\text{The output of a shifted input}} = \underbrace{H(a[m])[n - t]}_{\text{The shift of the output of the original input}}$$

The output of a shifted input

The shift of the output of the original input

# Box Filtering

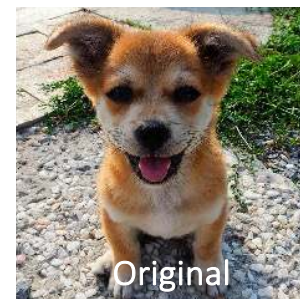
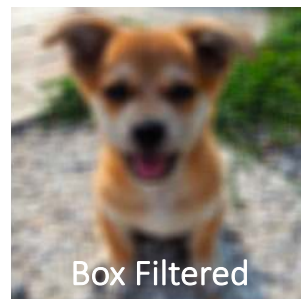
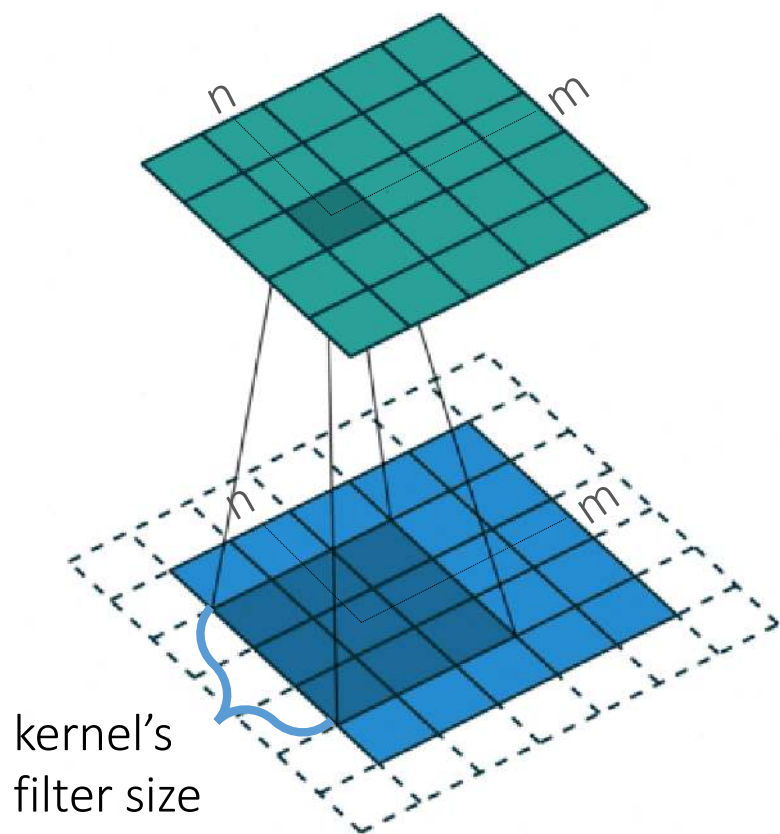


$$h[m, n] = \sum_{k, l} f[m + k, n + l]$$

$$k \in \left[ -\frac{\text{Filter Size}}{2}, \frac{\text{Filter Size}}{2} \right]$$

$$l \in \left[ -\frac{\text{Filter Size}}{2}, \frac{\text{Filter Size}}{2} \right]$$

# Box Filtering

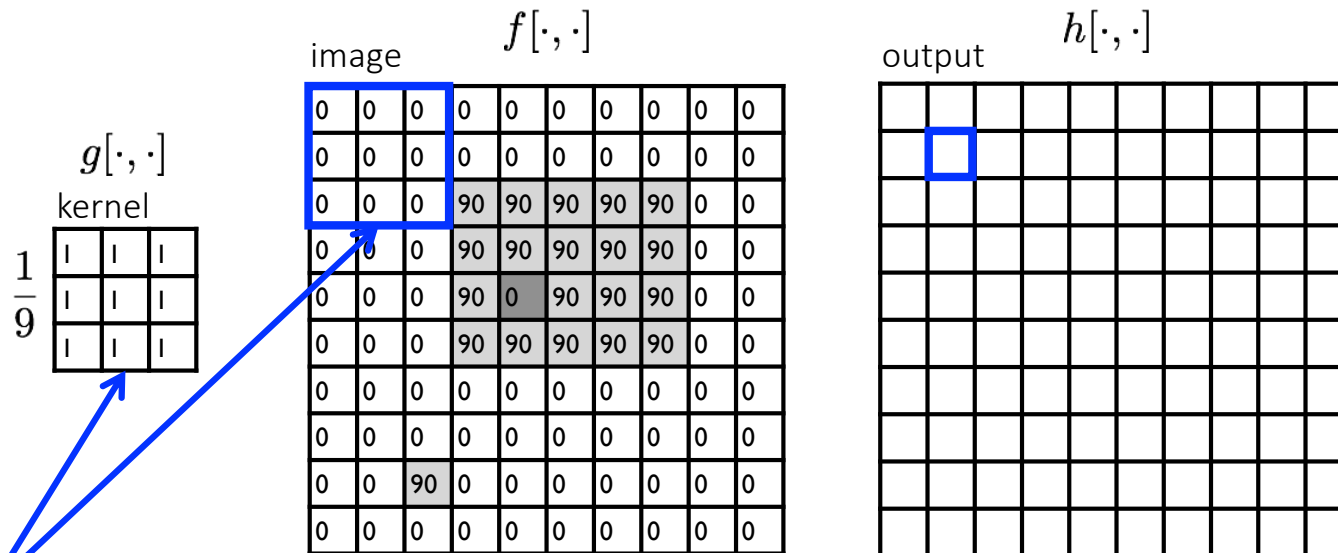


$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$



$$g[k, l] = \frac{1}{\text{Filter Size}^2}$$

# Let's run the box filter

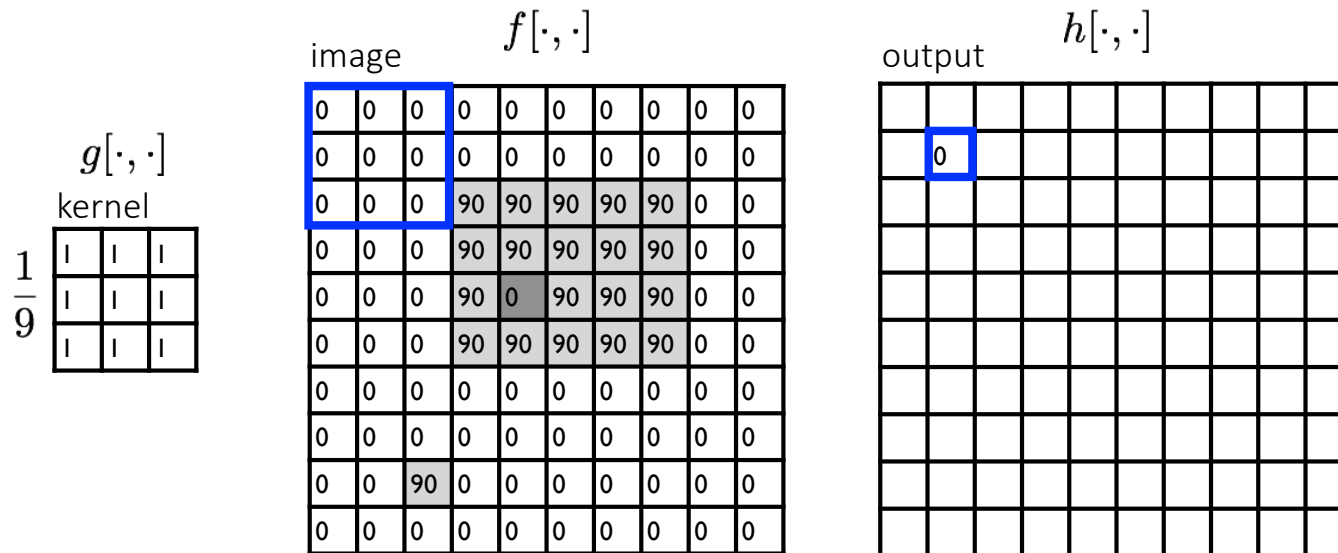


note that we assume that the kernel coordinates are centered

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
filter
image (signal)

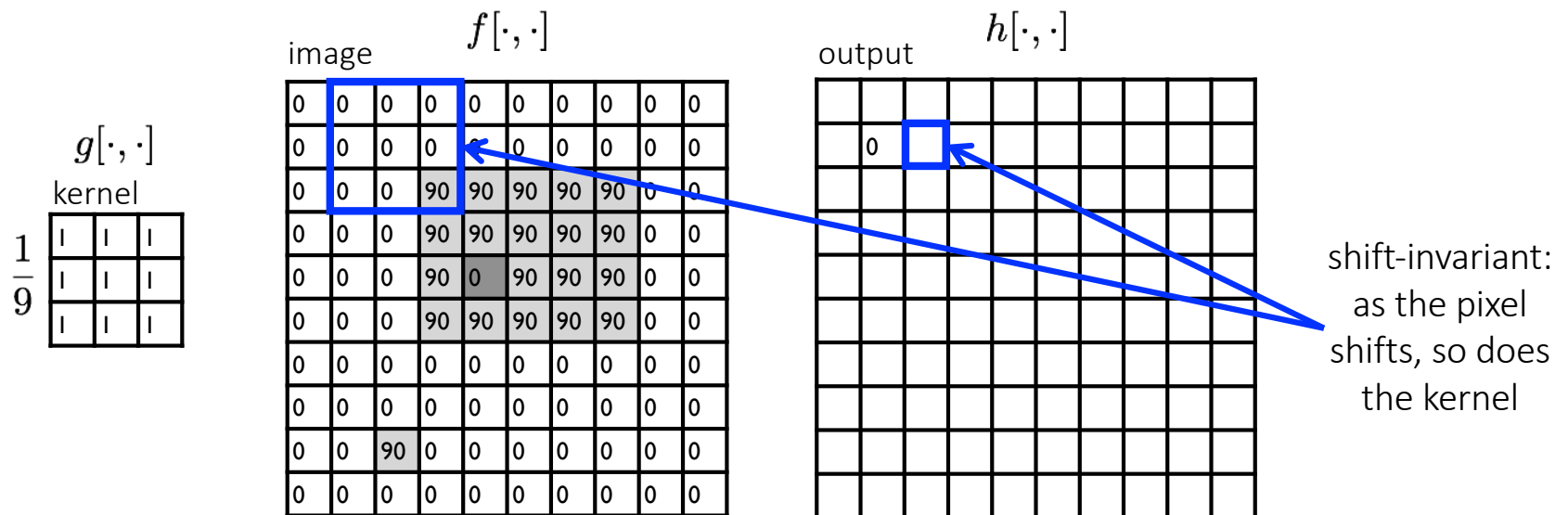
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 $k, l$ 
filter
image (signal)

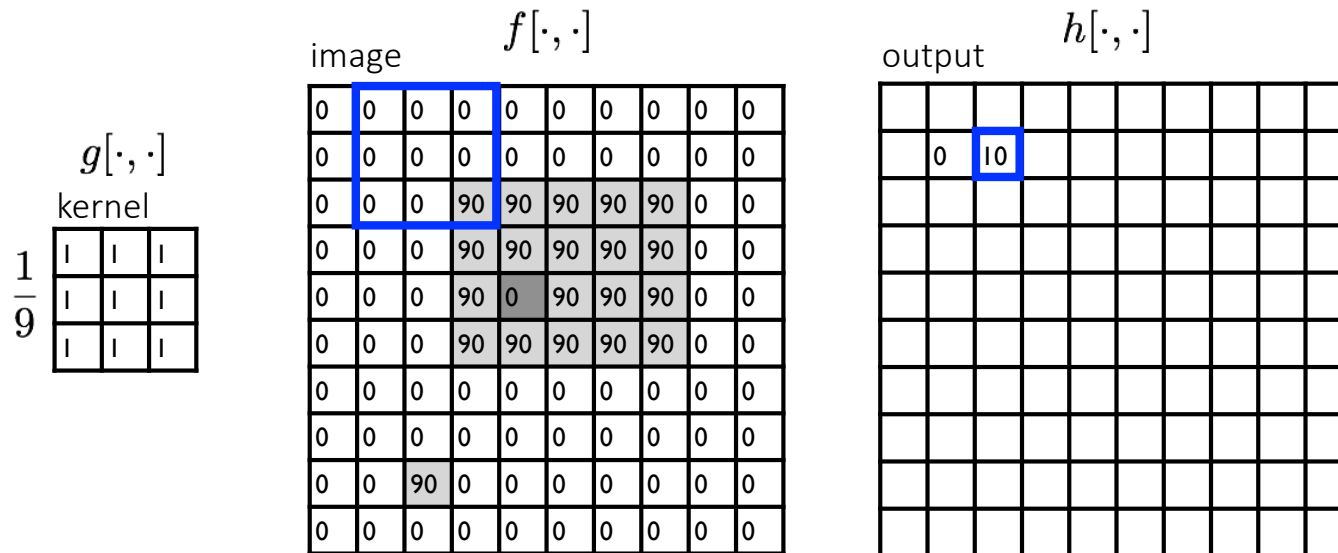
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output      filter      image (signal)

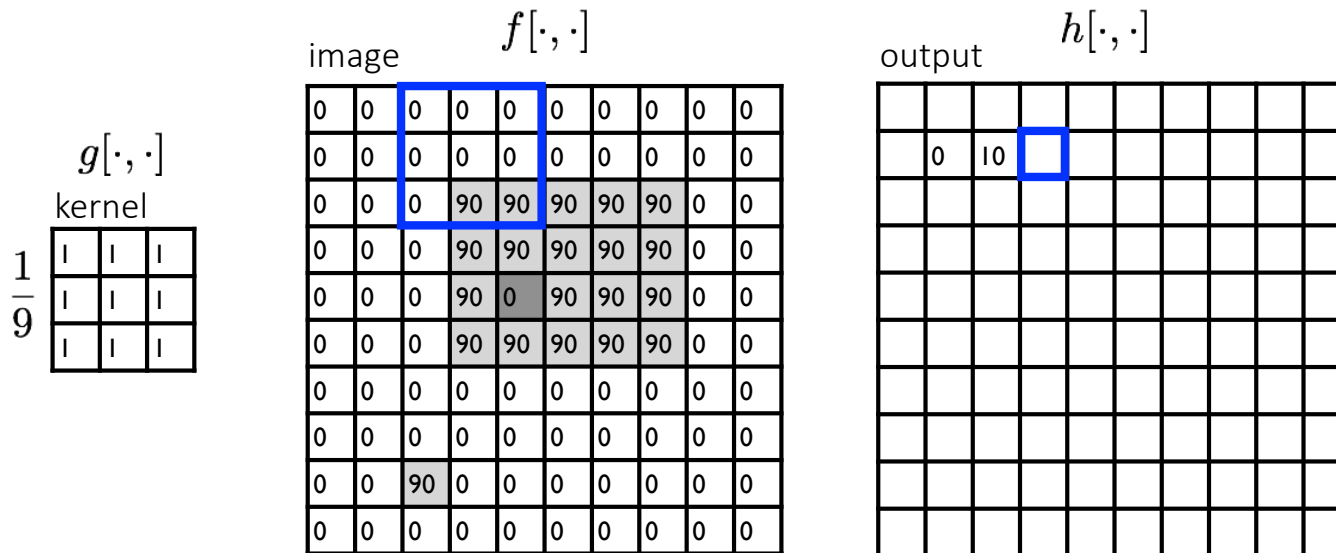
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 $k, l$ 
filter
image (signal)

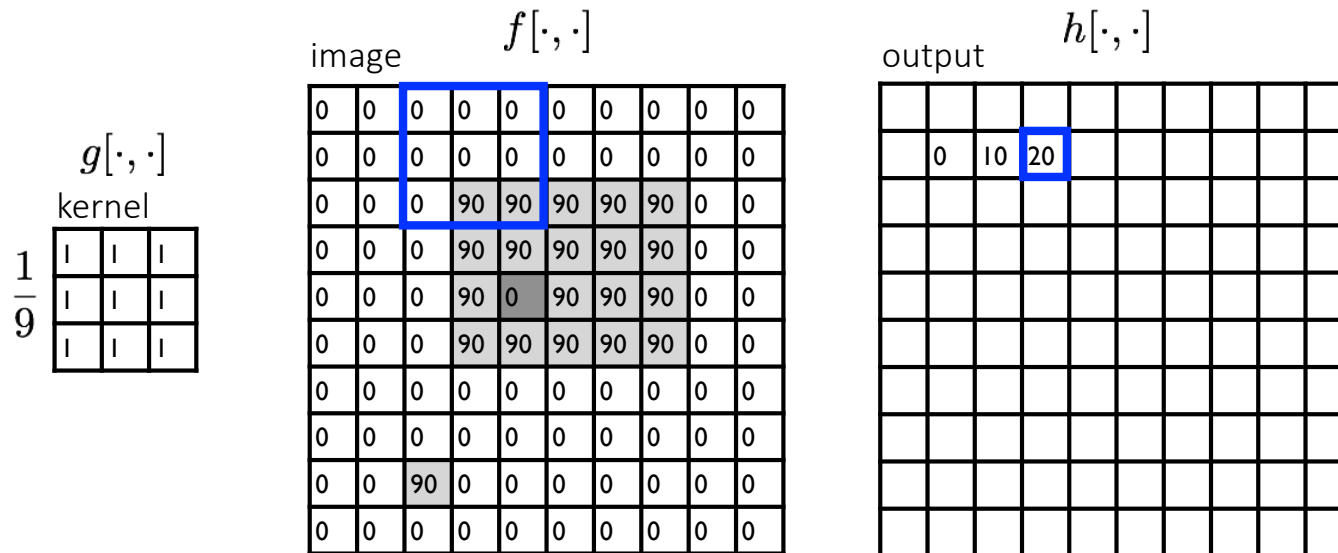
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output      filter      image (signal)

# Let's run the box filter

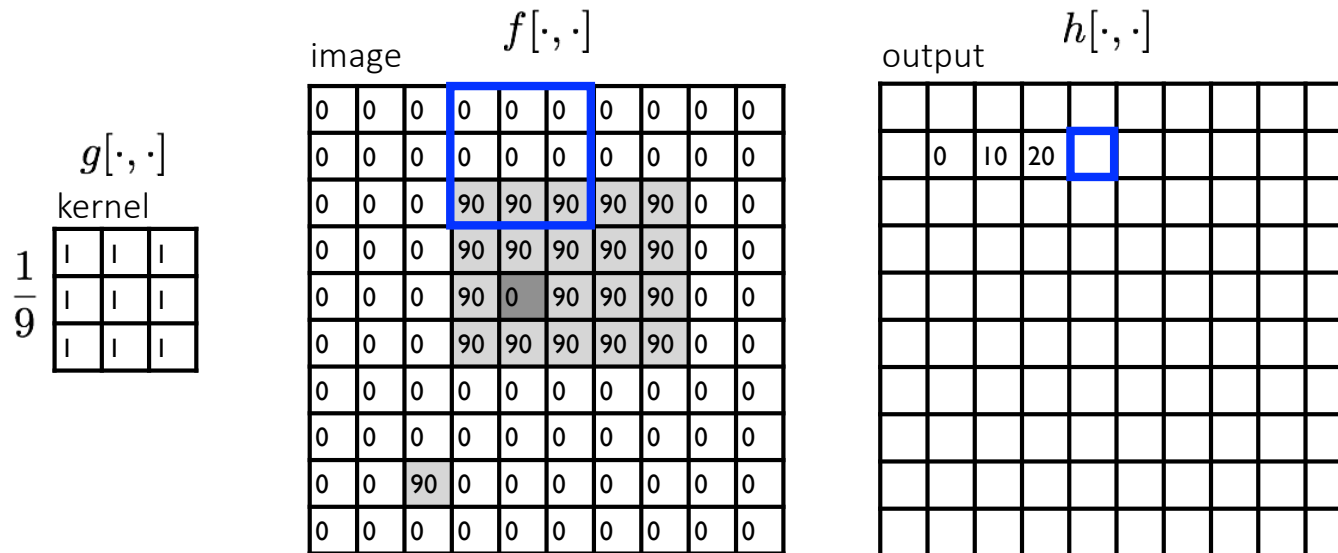


$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output

filter
image (signal)

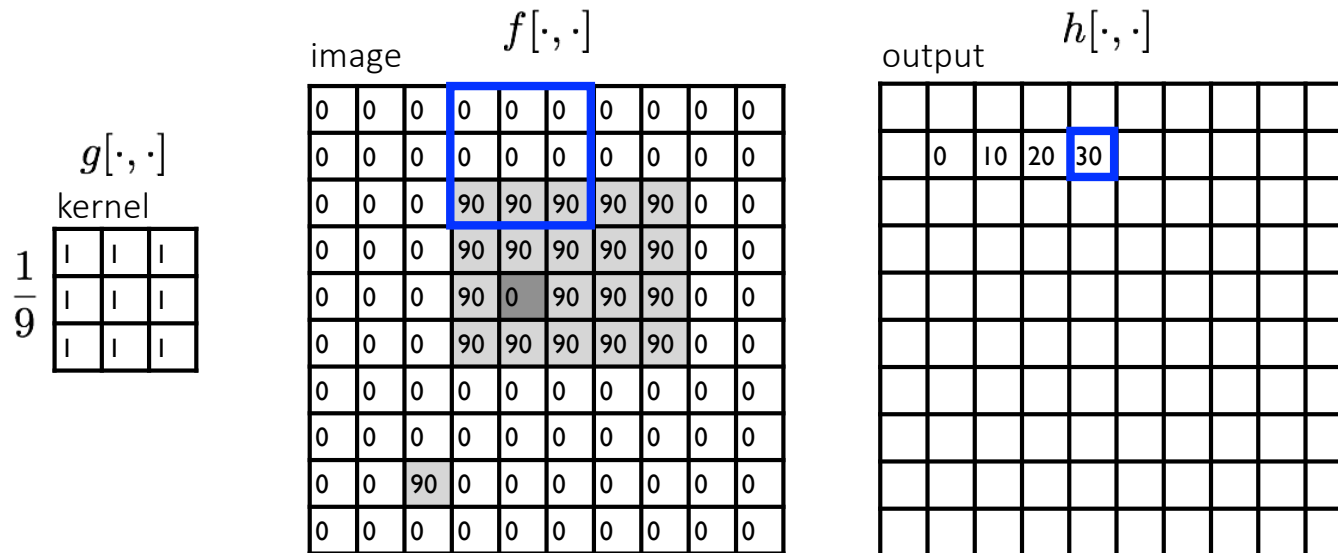
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output      filter      image (signal)

# Let's run the box filter

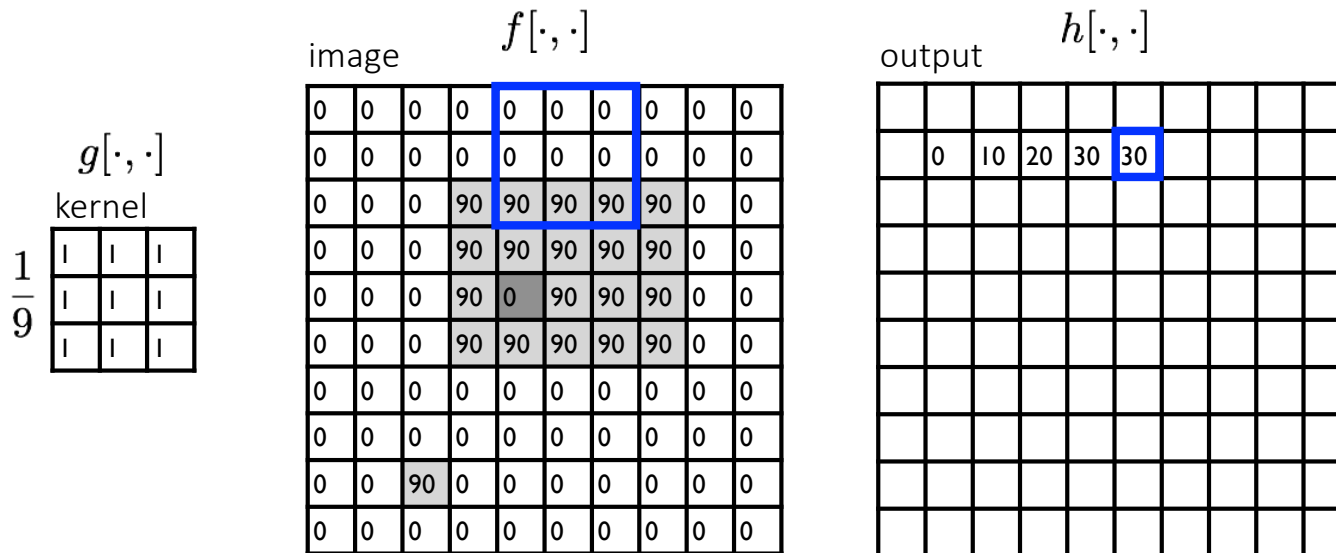


$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output

filter
image (signal)

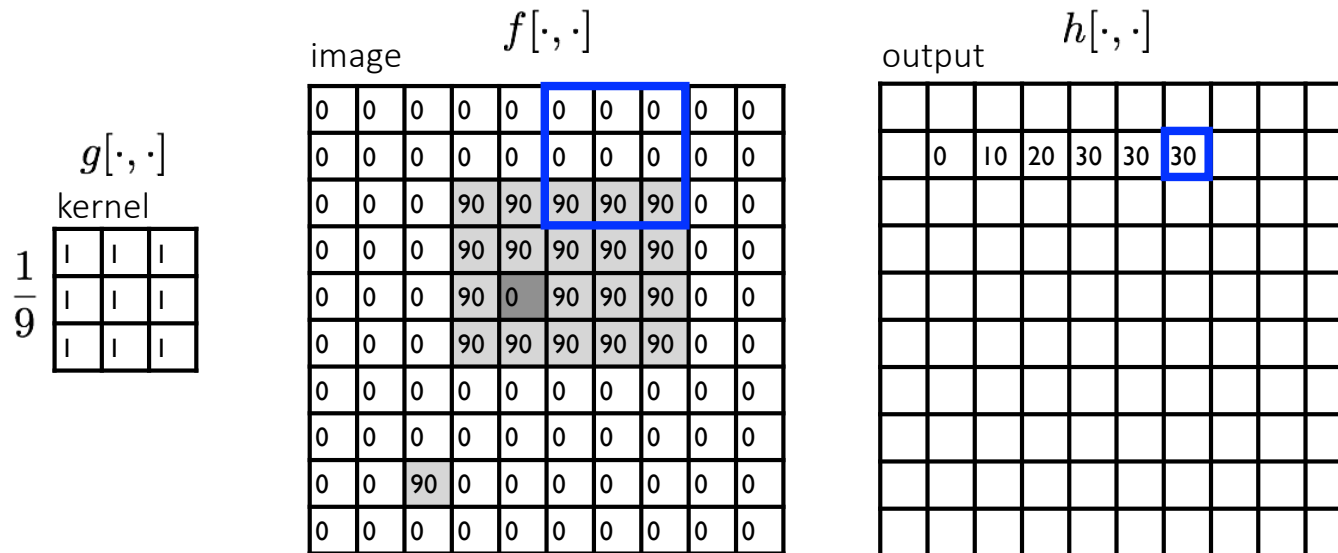
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 $k, l$ 
filter
image (signal)

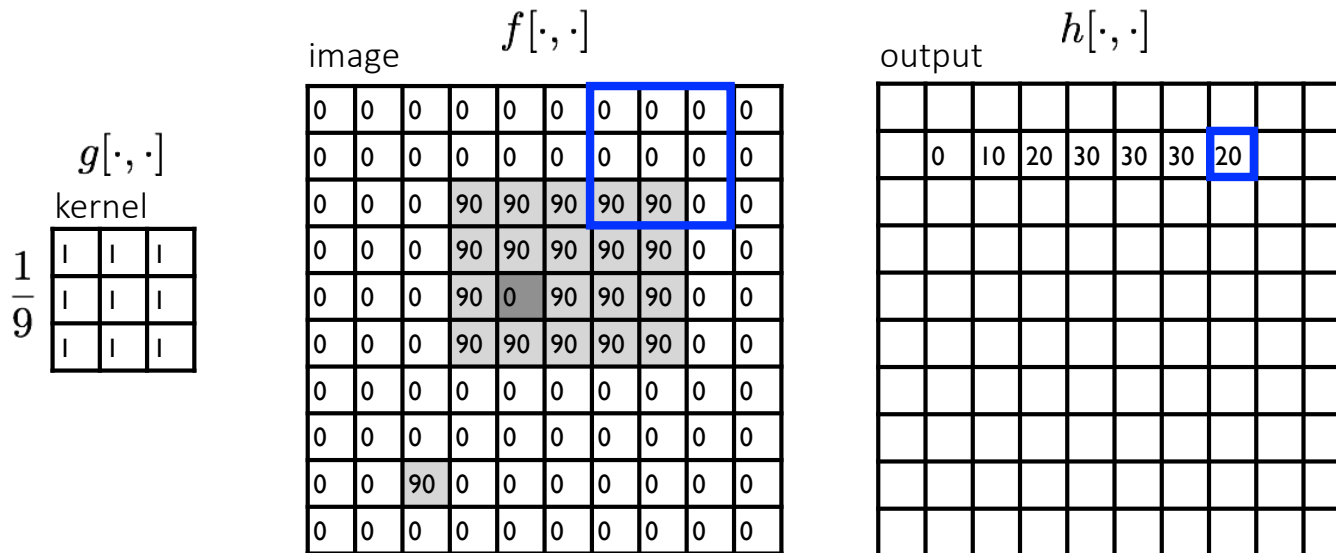
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output      filter      image (signal)

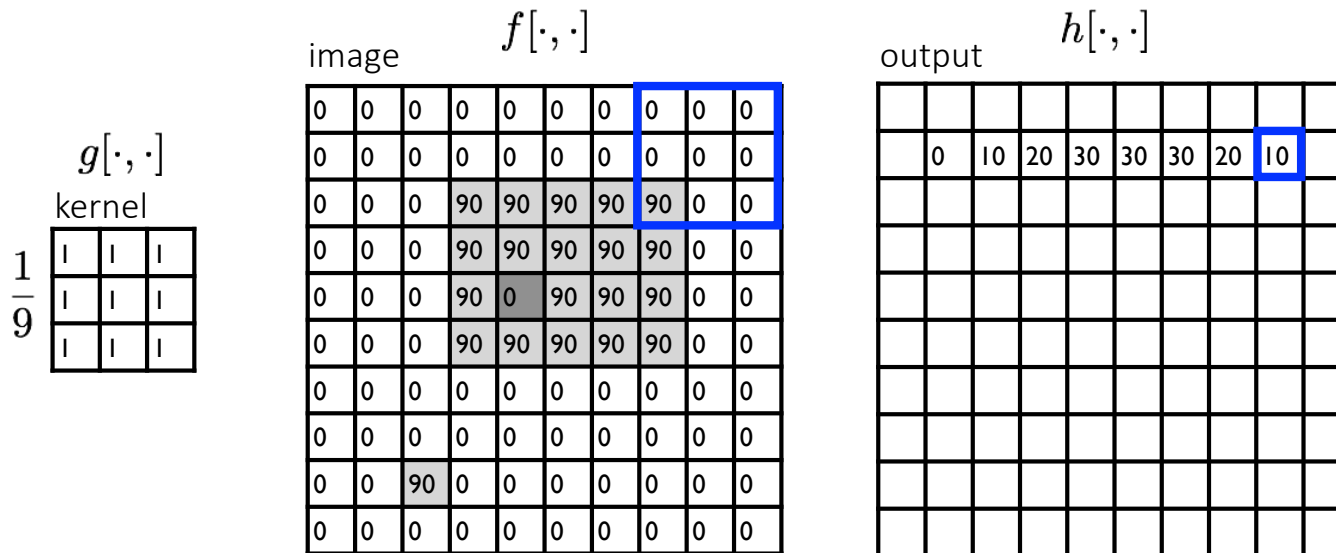
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 $k, l$ 
filter
image (signal)

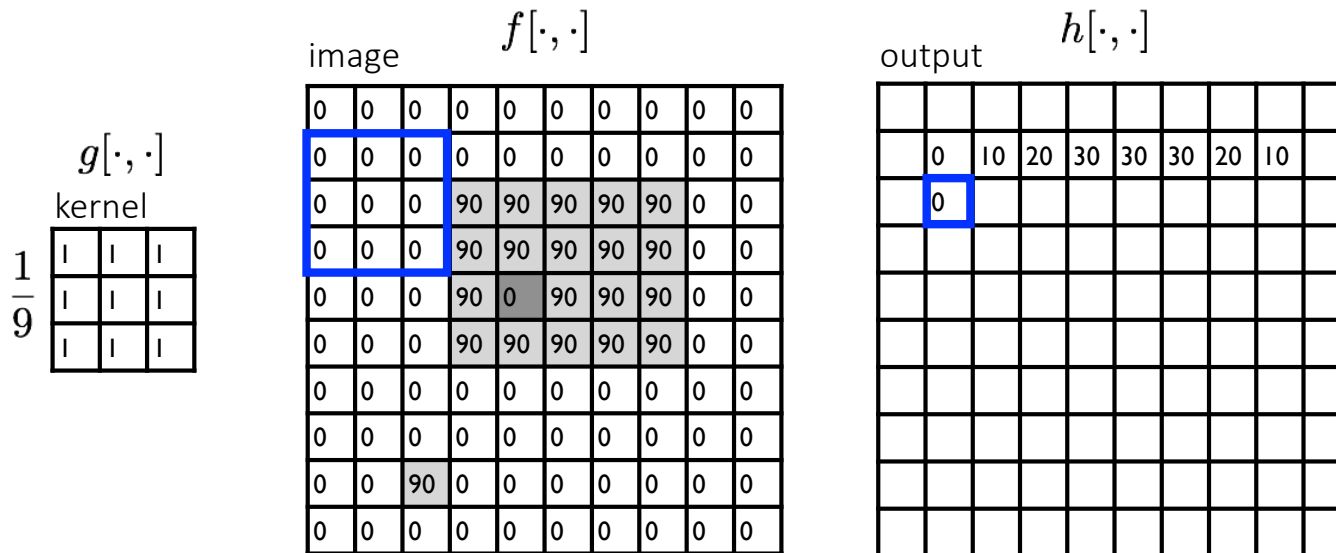
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 $k, l$ 
filter
image (signal)

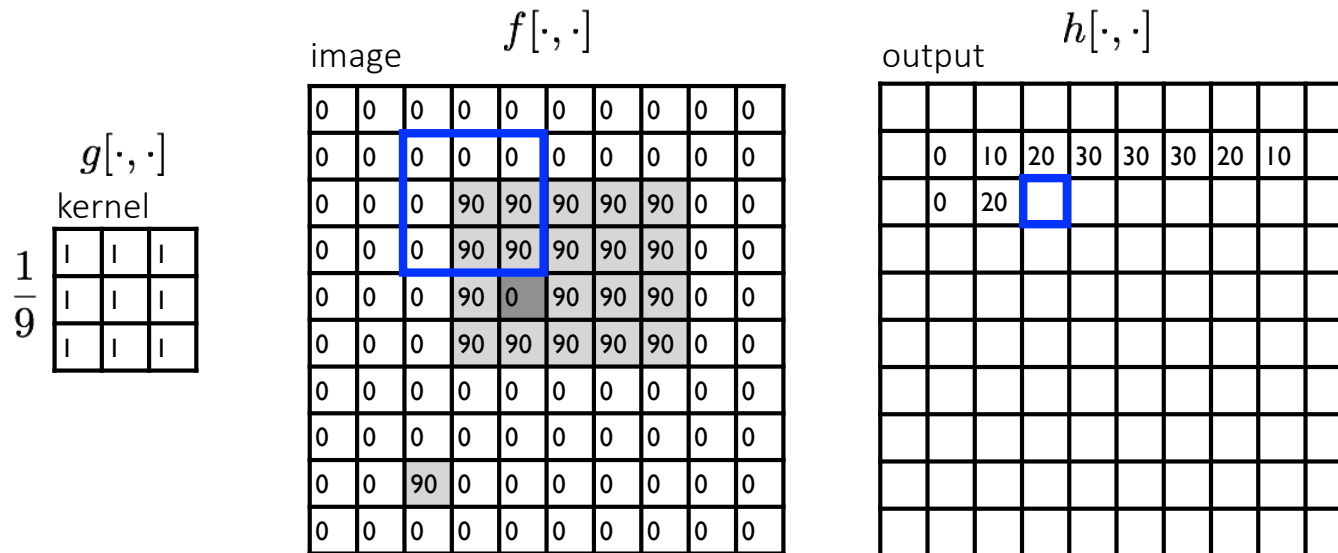
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output      filter      image (signal)

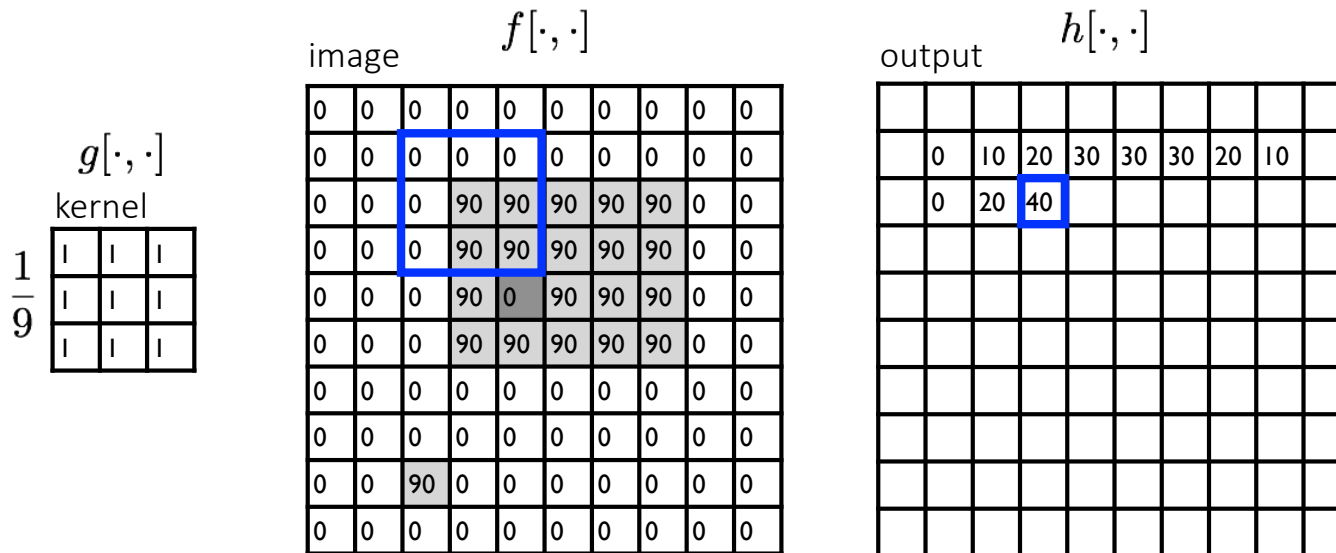
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output  $k, l$  filter image (signal)

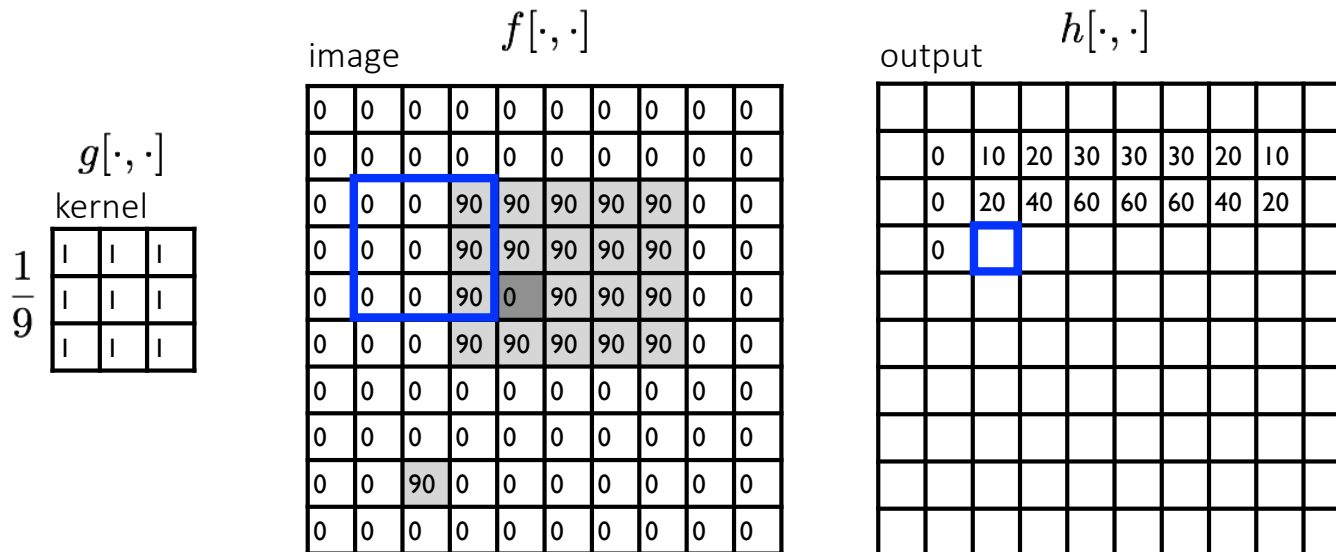
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output
 $k, l$ 
filter
image (signal)

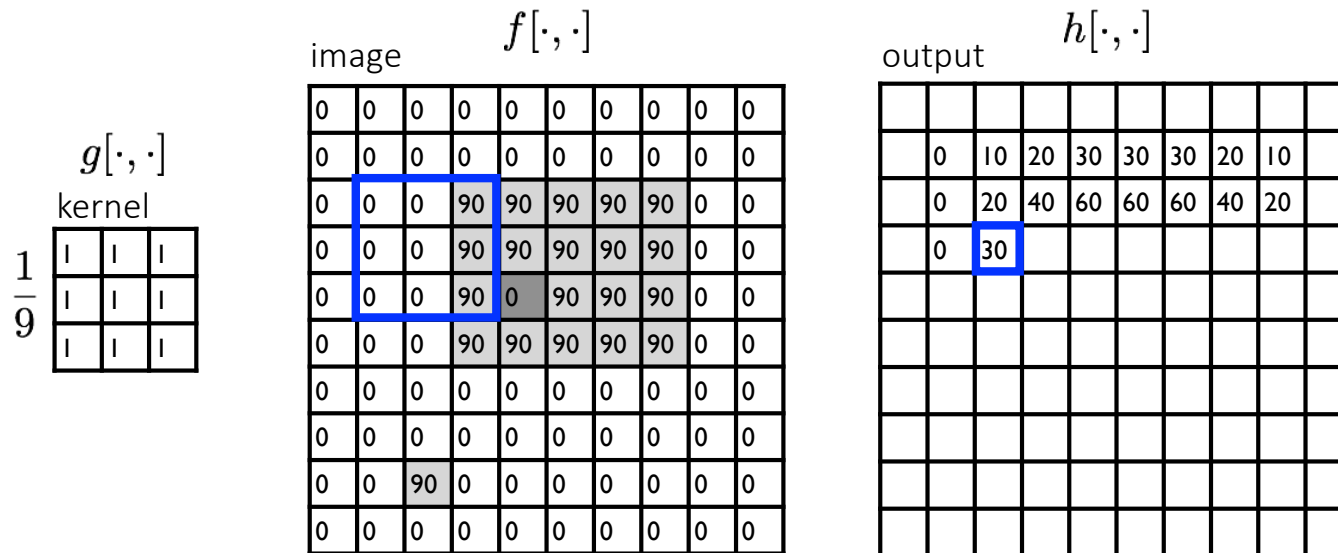
# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output      filter      image (signal)

# Let's run the box filter



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

output      filter      image (signal)

## Let's run the box filter

$$\frac{1}{9} \begin{matrix} & \text{kernel} \\ g[\cdot, \cdot] \\ \begin{bmatrix} | & | & | \\ | & | & | \\ | & | & | \end{bmatrix} \end{matrix}$$
[illegible][illegible]

$$\underset{\text{output}}{h[m,n]} = \sum_{k,l} \underset{\text{filter}}{g[k,l]} \underset{\text{image (signal)}}{f[m+k,n+l]}$$

## Let's run the box filter

$$\frac{1}{9} \begin{matrix} & g[\cdot, \cdot] \\ \text{kernel} & \begin{bmatrix} | & | & | \\ | & | & | \\ | & | & | \end{bmatrix} \end{matrix}$$
[illegible][illegible]

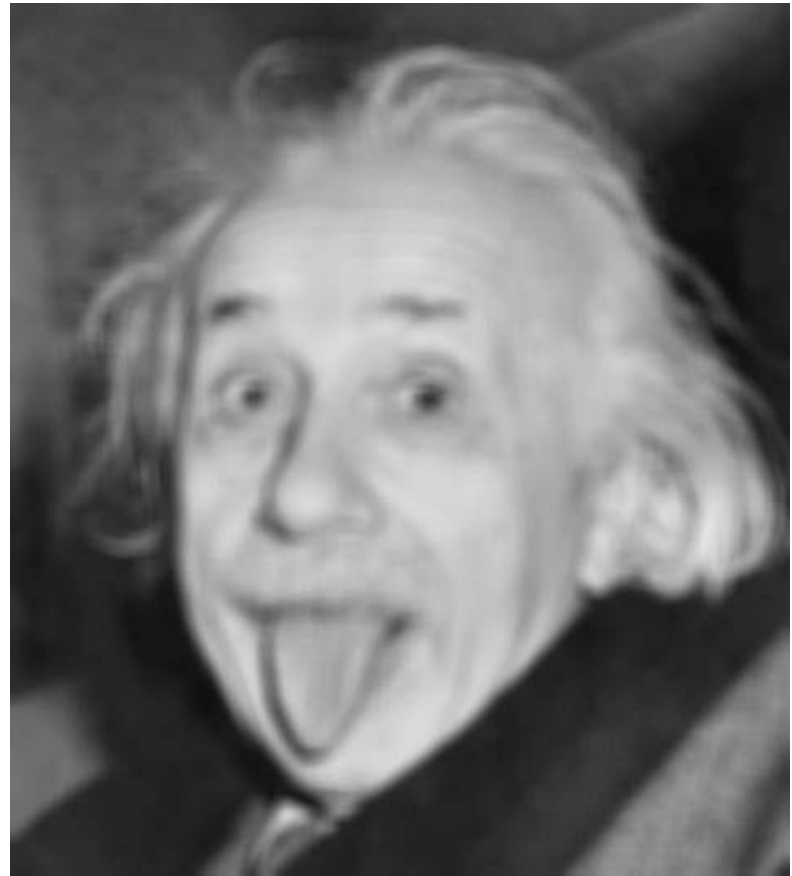
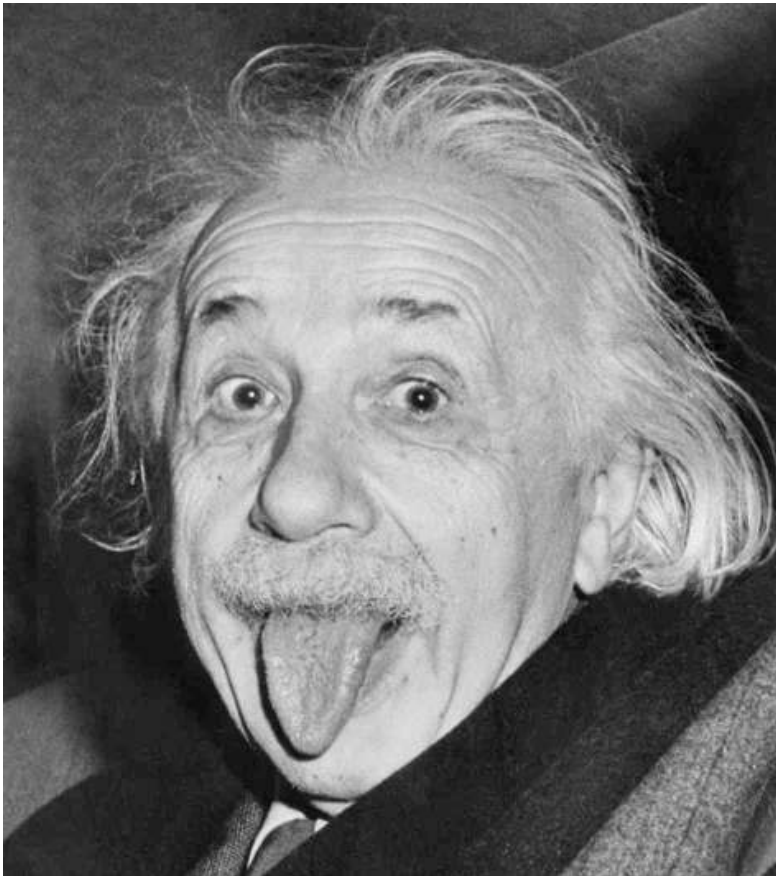
$$\underset{\text{output}}{h[m, n]} = \sum_{k, l} \underset{\text{filter}}{g[k, l]} \underset{\text{image (signal)}}{f[m + k, n + l]}$$

... and the result is

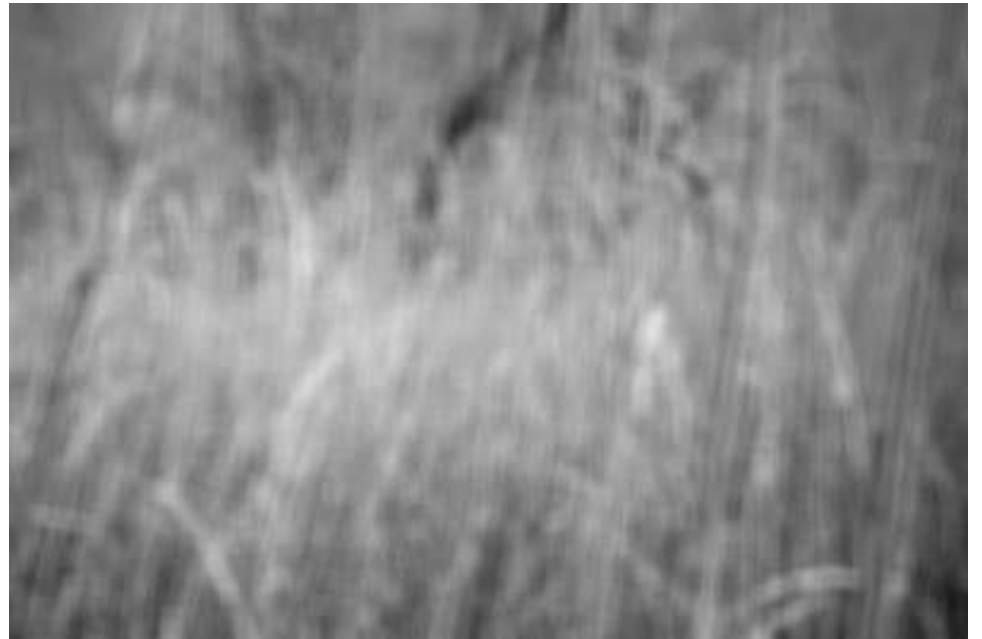
$$\frac{1}{9} \begin{matrix} & \text{kernel} \\ g[\cdot, \cdot] \\ \begin{matrix} | & | & | \\ | & | & | \\ | & | & | \end{matrix} \end{matrix}$$
[illegible][illegible]

$$\underset{\text{output}}{h[m,n]} = \sum_{k,l} \underset{\text{filter}}{g[k,l]} \underset{\text{image (signal)}}{f[m+k,n+l]}$$

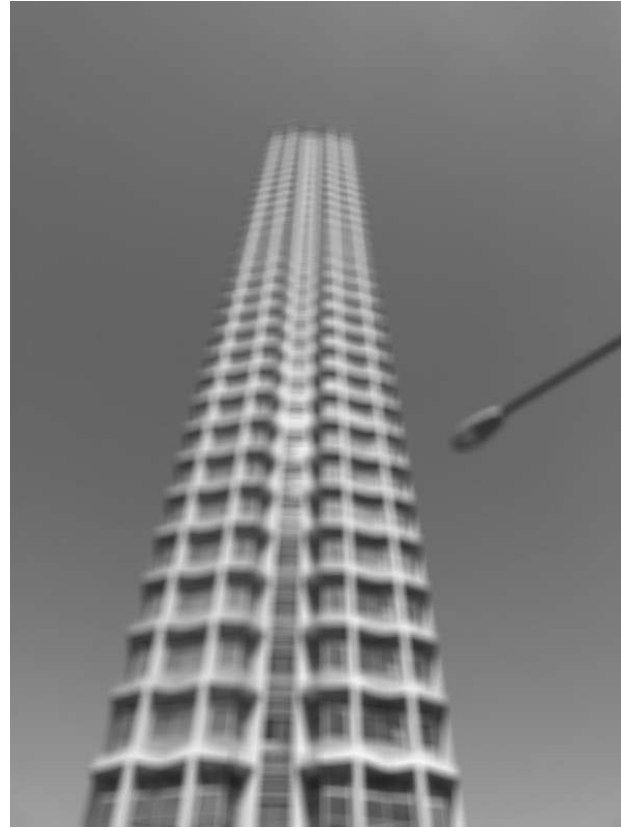
Some more realistic examples



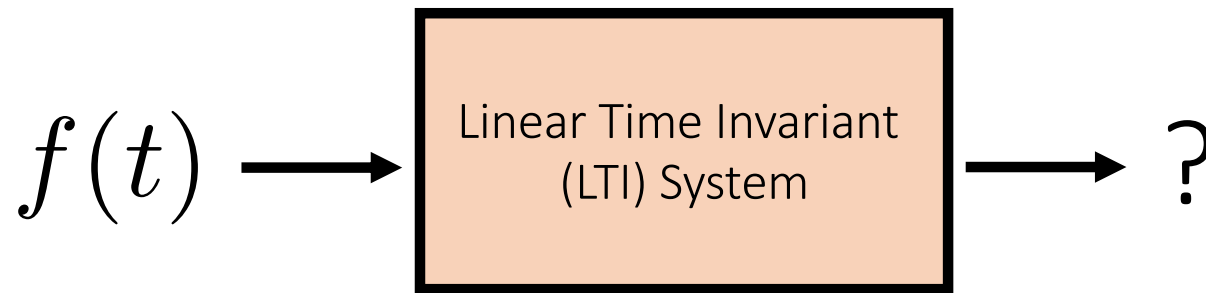
## Some more realistic examples



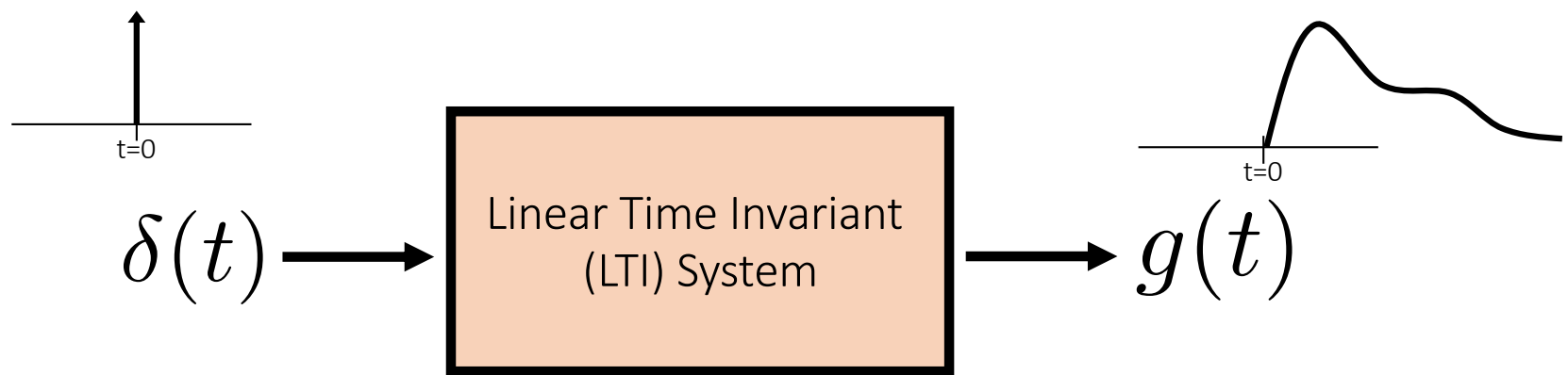
Some more realistic examples



What is the output of an LTI system for an input?

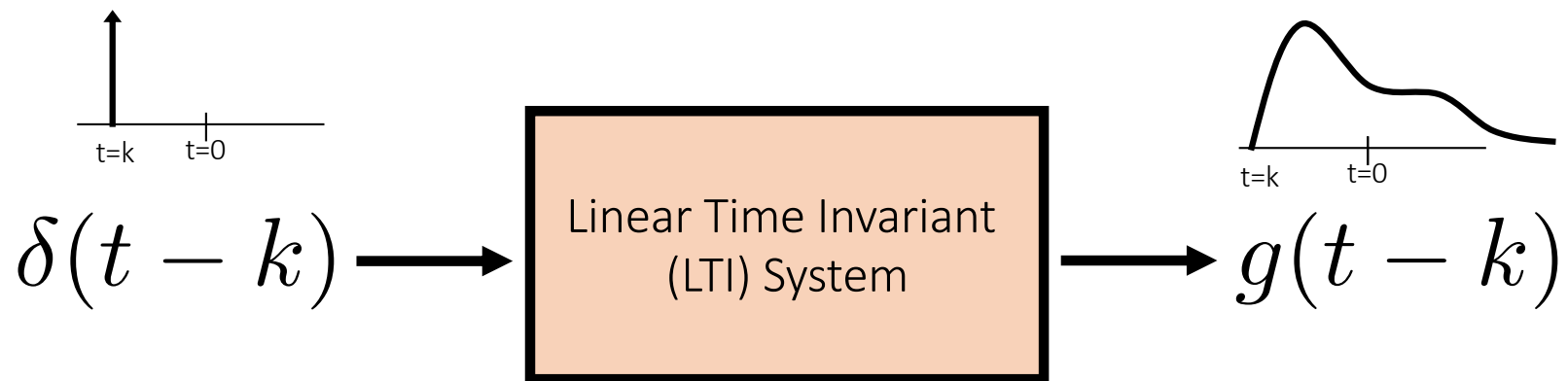


What is the output of an LTI system for an input?



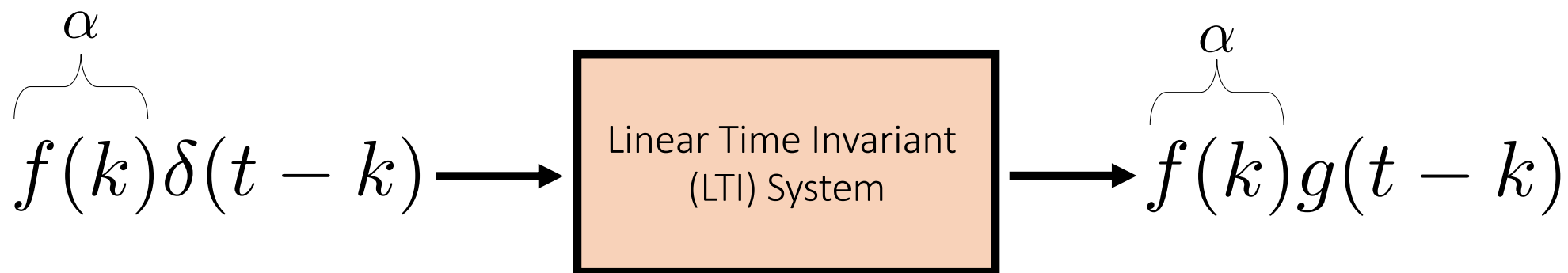
The response of an LTI system to a delta input is the impulse response

What is the output of an LTI system for an input?



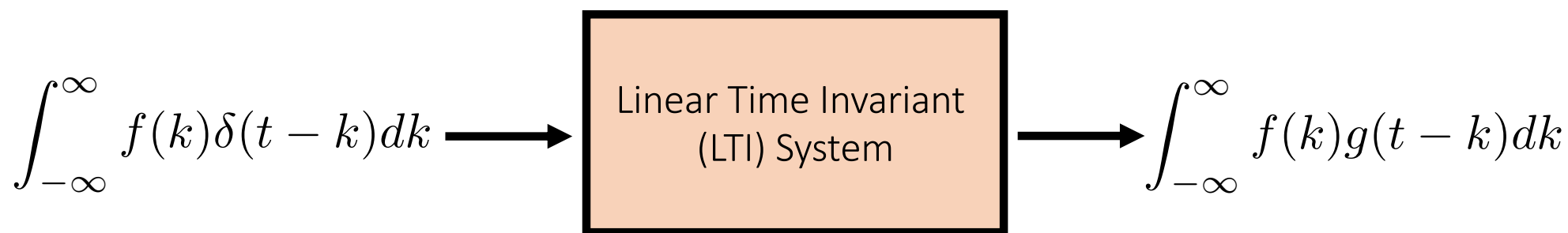
Invoking time/shift-invariance

What is the output of an LTI system for an input?



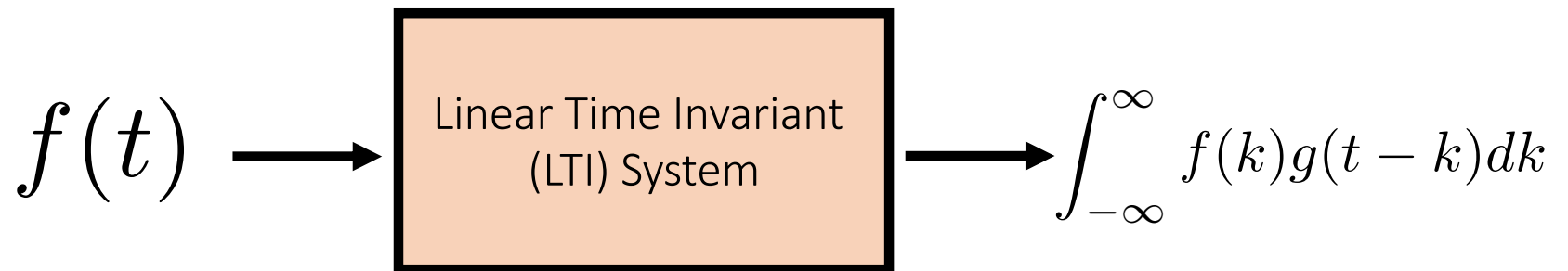
Invoking linearity

What is the output of an LTI system for an input?



Invoking linearity

What is the output of an LTI system for an input?



Any LTI system can be characterized entirely by the system's impulse response.

The output of an LTI system is simply the convolution of the input to the system with the system's impulse response.

## 3+ Main Takeaways

- A linear time-invariant system is fully characterized by its impulse response
- You can represent any signal as a sum of complex exponentials
- Convolution is multiplication in the frequency domain

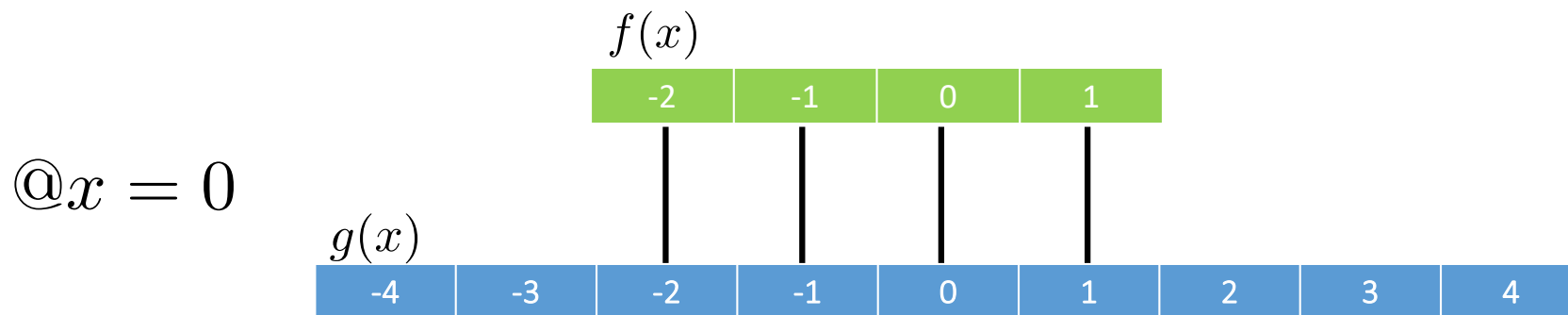
# Convolution

# Cross-correlation for 1D continuous signals

Definition of filtering:

$$\text{xcorr}(f, g)(x) = \sum_{y=-\infty}^{\infty} f(y)g(x+y),$$

filtered signal 



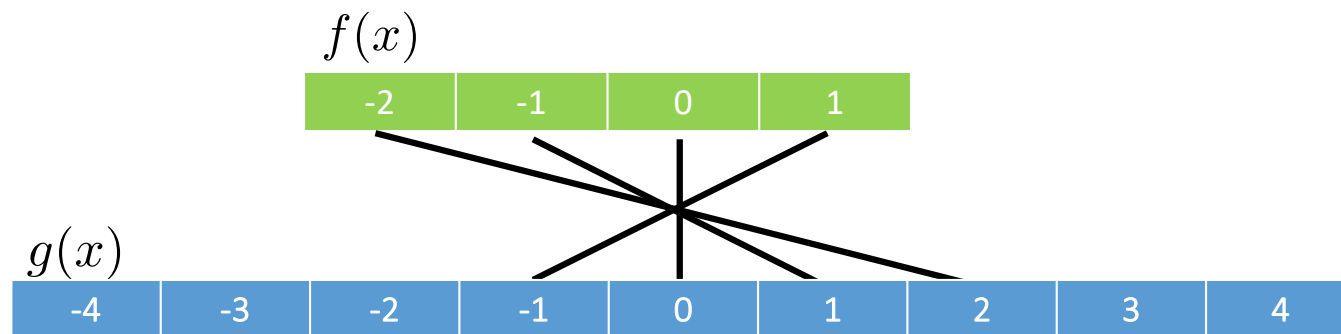
# Convolution for 1D continuous signals

Definition of filtering:

$$(f * g)(x) = \sum_{y=-\infty}^{\infty} f(y)g(x-y)$$

filtered signal  $\nearrow$  notice the flip  $\nwarrow$

@  $x = 0$

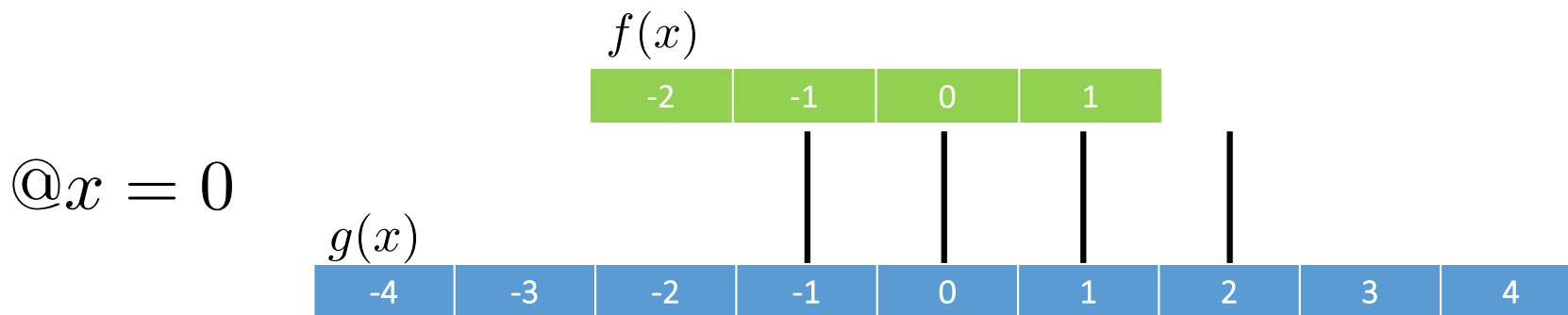


# Convolution for 1D continuous signals

Definition of filtering as convolution:

$$(f * g)(x) = \sum_{y=-\infty}^{\infty} f(y)g(x - y)$$

filtered signal  $\nearrow$  notice the flip  $\nwarrow$

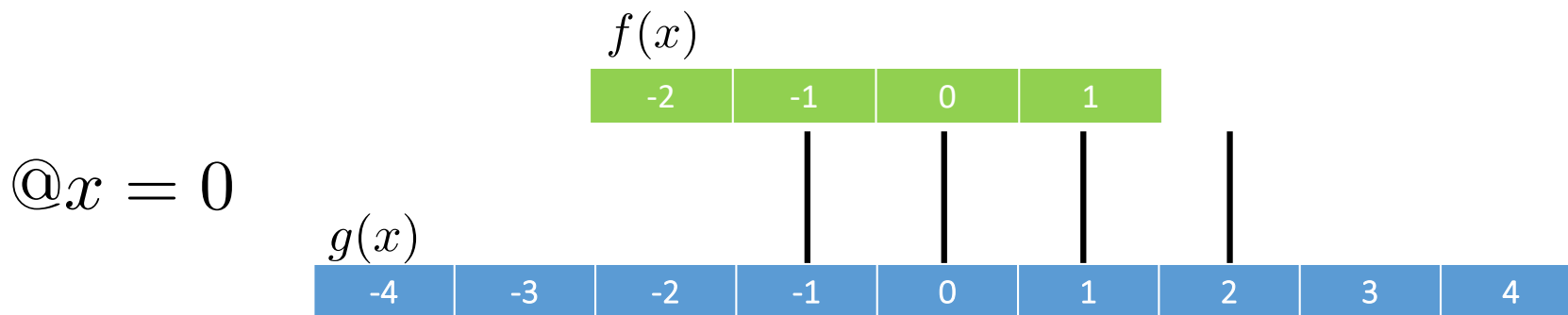


# Convolution for 1D continuous signals

Definition of filtering as convolution:

$$(f * g)(x) = \int_{-\infty}^{\infty} f(y)g(x - y)dy$$

filtered signal 

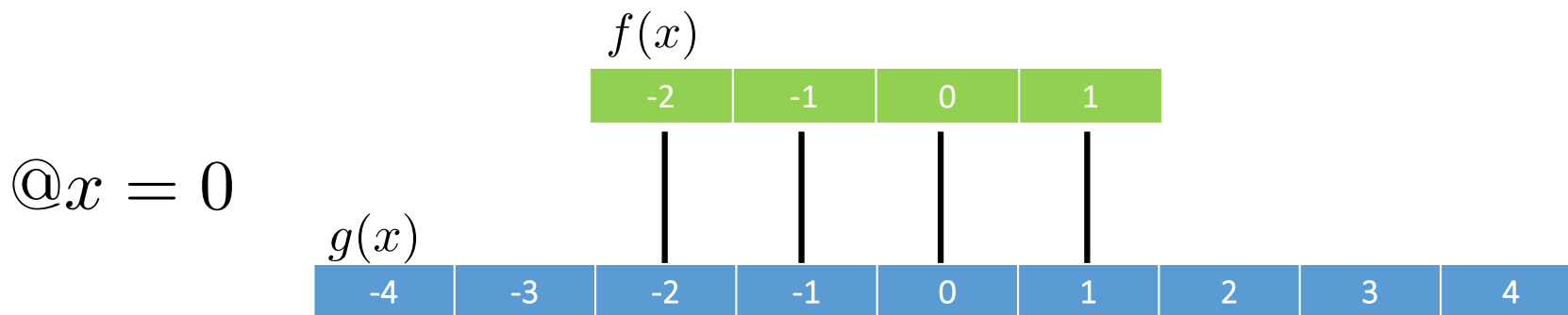


# Convolution for 1D continuous signals

Definition of filtering as convolution:

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x - y)g(y)dy$$

filtered signal 



# Convolution vs correlation

Definition of discrete 2D convolution:

$$(f * g)[x, y] = \sum_{-\infty}^{\infty} f(i, j)g(x - i, y - j)$$

notice the flip



Definition of discrete 2D correlation:

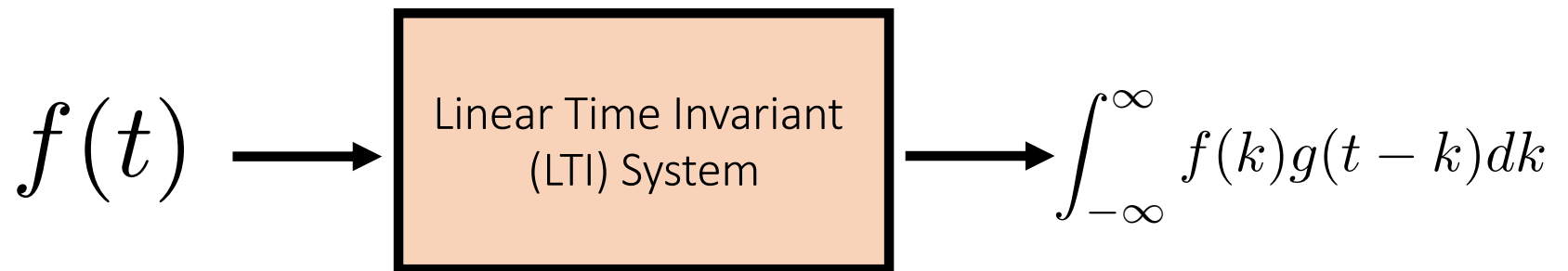
$$\text{xcorr}(f, g)[x, y] = \sum_{-\infty}^{\infty} f(i, j)g(x + i, y + j)$$

notice the lack of a flip



- Most of the time won't matter, because our kernels will be symmetric.

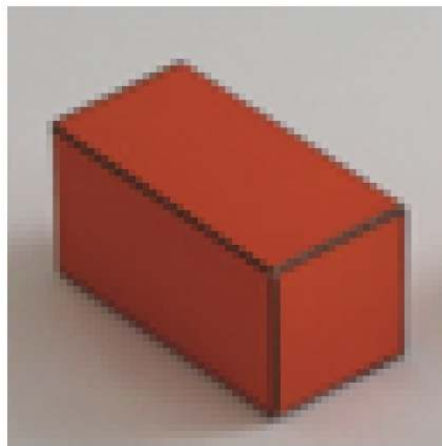
What is the output of an LTI system for an input?



Any LTI system can be characterized entirely by the system's impulse response.

The output of an LTI system is simply the convolution of the input to the system with the system's impulse response.

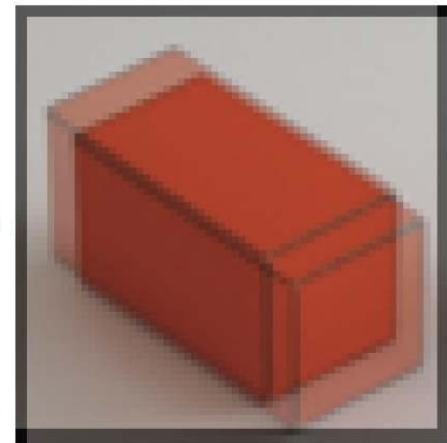
# Example



○

|    |   |   |   |    |
|----|---|---|---|----|
| .5 | 0 | 0 | 0 | 0  |
| 0  | 0 | 0 | 0 | 0  |
| 0  | 0 | 0 | 0 | 0  |
| 0  | 0 | 0 | 0 | 0  |
| 0  | 0 | 0 | 0 | .5 |

=

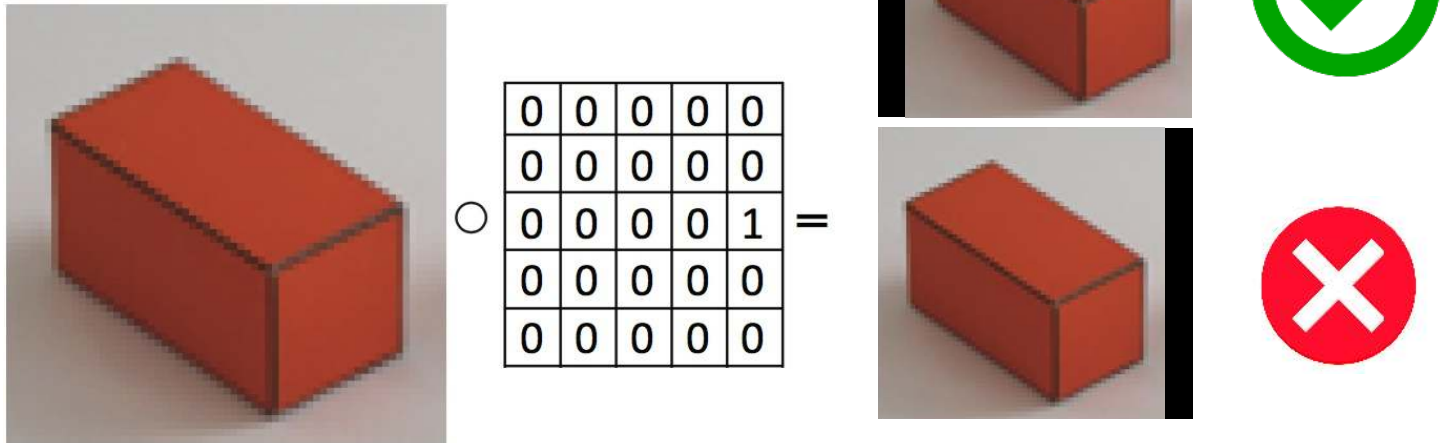


(using zero padding)

filtered signal  $\nearrow$

$$(f * g)[x, y] = \sum_{-\infty}^{\infty} f(i, j)g(x - i, y - j)$$

## Example

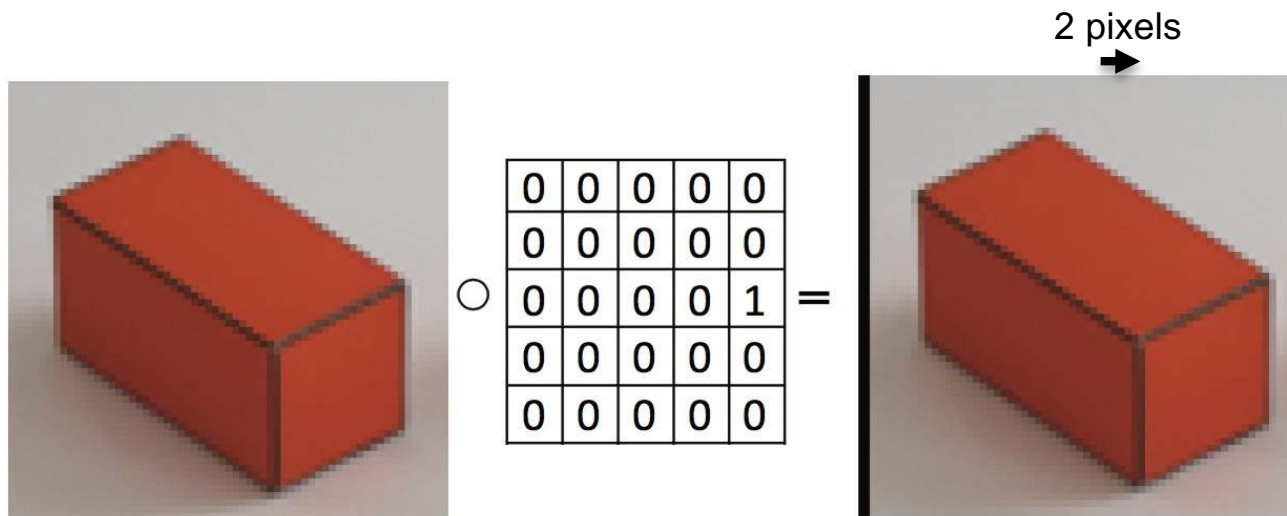


What does the resulting image look like?

filtered signal  $\nearrow$

$$(f * g)[x, y] = \sum_{-\infty}^{\infty} f(i, j)g(x - i, y - j)$$

# Example



What does the resulting image look like?

filtered signal  $\nearrow$ 

$$(f * g)[x, y] = \sum_{-\infty}^{\infty} f(i, j)g(x - i, y - j)$$

Fourier series

What is he famous for?



Jean Baptiste Joseph Fourier  
(1768-1830)

# What is he famous for?



Jean Baptiste Joseph Fourier  
(1768-1830)

The Fourier series claim (1807):

*'Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies.'*

# Is this claim true?



Jean Baptiste Joseph Fourier  
(1768-1830)

The Fourier series claim (1807):

*'Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies.'*



Malus



Lagrange



Legendre



Laplace

The committee examining his paper had expressed skepticism, in part due to not so rigorous proofs

# Is this claim true?



Jean Baptiste Joseph Fourier  
(1768-1830)

The Fourier series claim (1807):

*'Any univariate function can be rewritten as a weighted sum of sines and cosines of different frequencies.'*

Well, almost.

- The theorem requires additional conditions.
- Close enough to be named after him.

# Basic building block

$$A \sin(\omega x + \phi)$$

Fourier's claim: Add enough of these to get any *periodic* signal you want!

# Basic building block

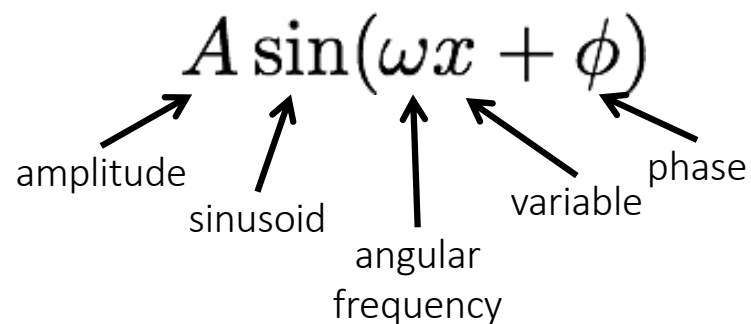
$$A \sin(\omega x + \phi)$$


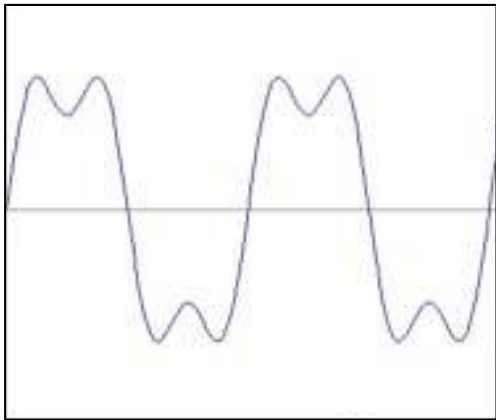
Diagram illustrating the components of the equation  $A \sin(\omega x + \phi)$ :

- $A$ : amplitude
- $\sin$ : sinusoid
- $\omega$ : angular frequency
- $x$ : variable
- $\phi$ : phase

Fourier's claim: Add enough of these to get any *periodic* signal you want!

# Examples

How would you generate this function?



=

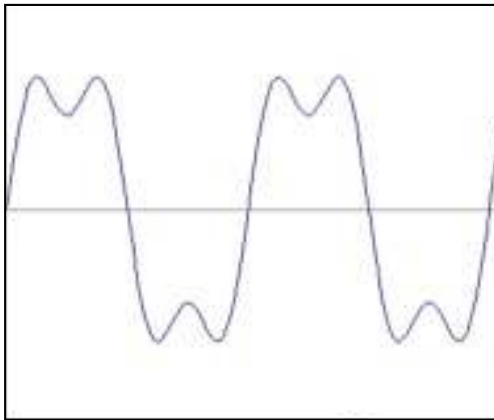
?

+

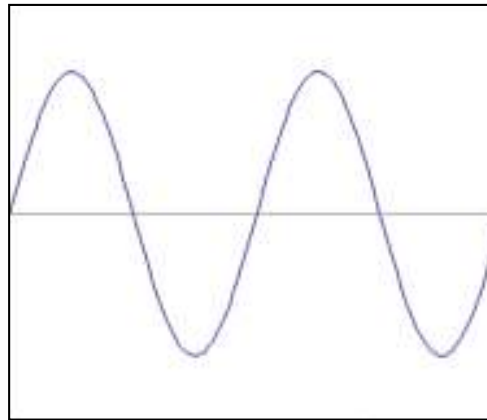
?

# Examples

How would you generate this function?



=



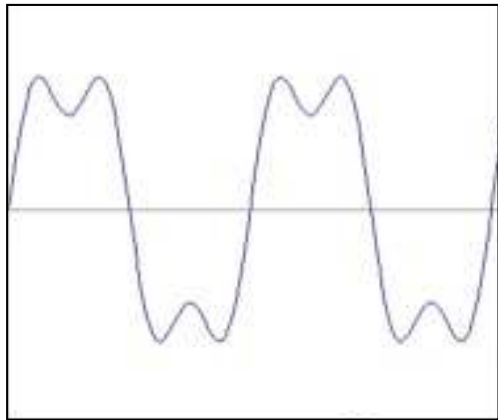
$\sin(2\pi x)$

+

?

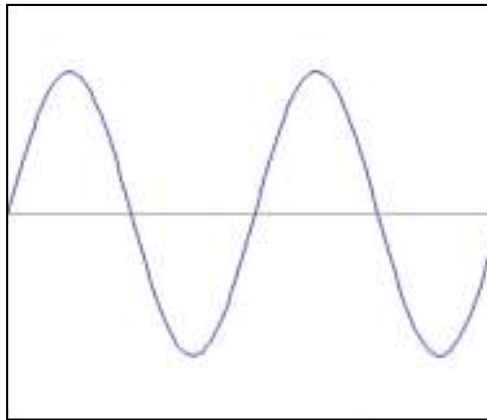
# Examples

How would you generate this function?



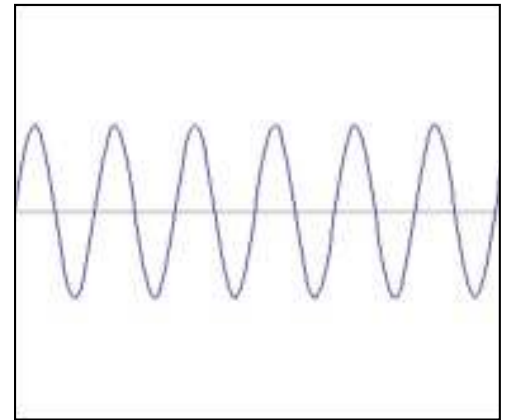
$$f(x) = \sin(2\pi x) + \frac{1}{3} \sin(2\pi 3x)$$

=



$$\sin(2\pi x)$$

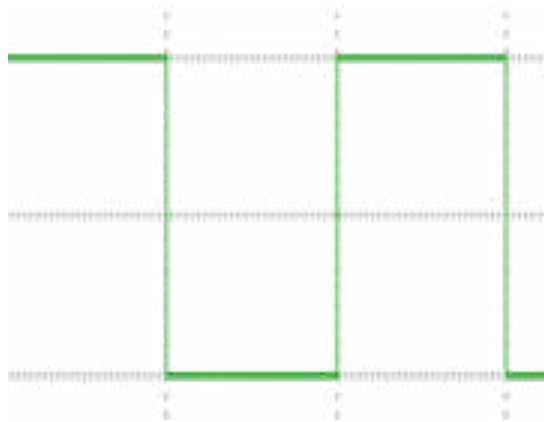
+



$$\frac{1}{3} \sin(2\pi 3x)$$

# Examples

How would you generate this function?



square wave

=

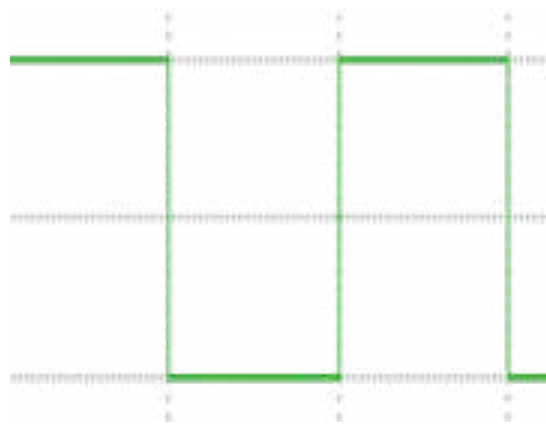
?

+

?

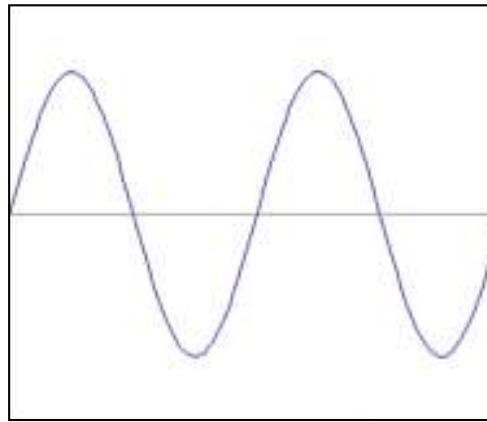
# Examples

How would you generate this function?

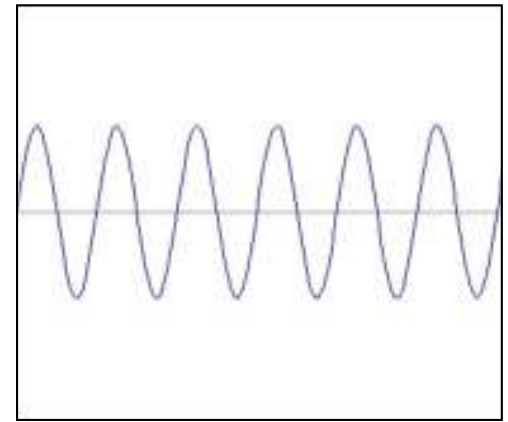


square wave

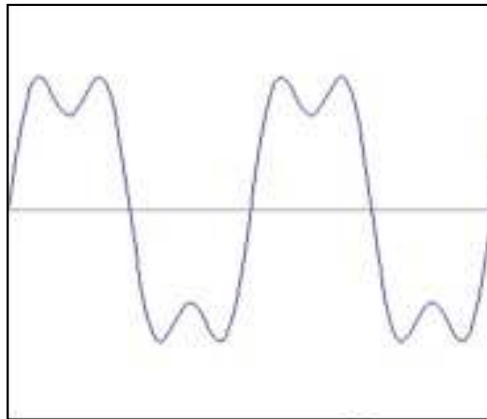
$\approx$



+

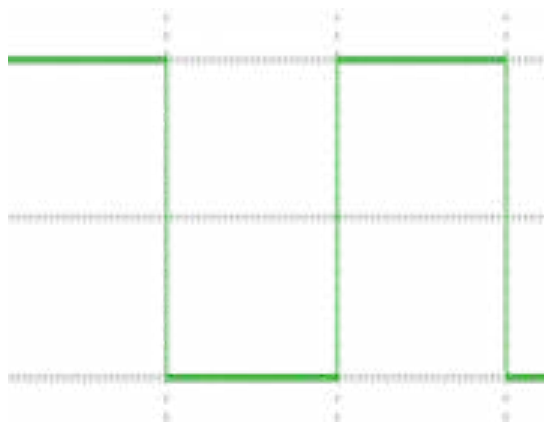


$=$



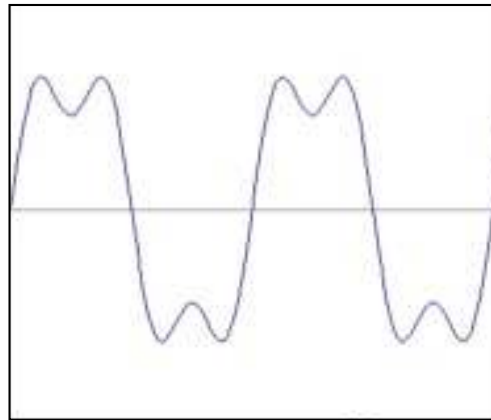
# Examples

How would you generate this function?

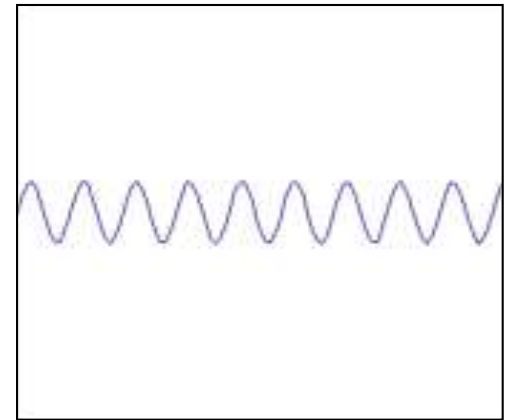


square wave

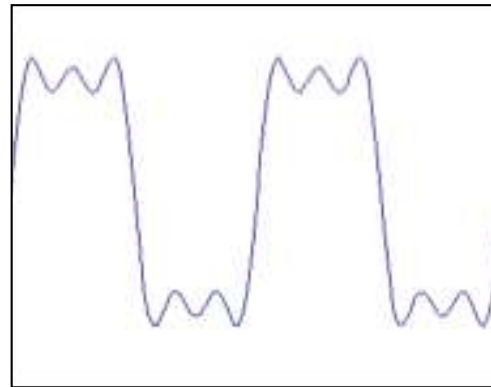
$\approx$



+

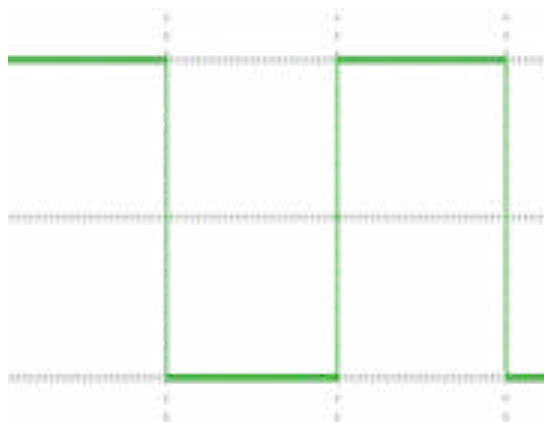


$=$



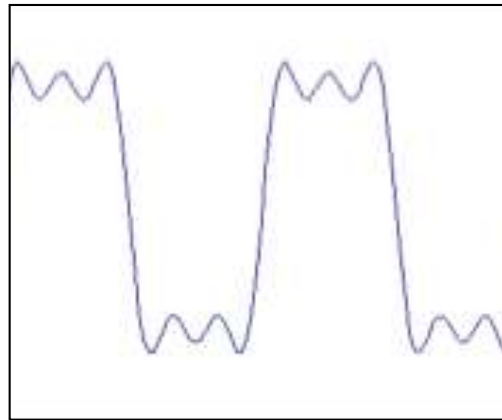
# Examples

How would you generate this function?

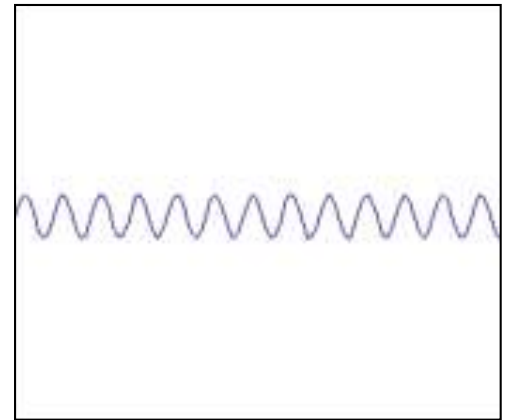


square wave

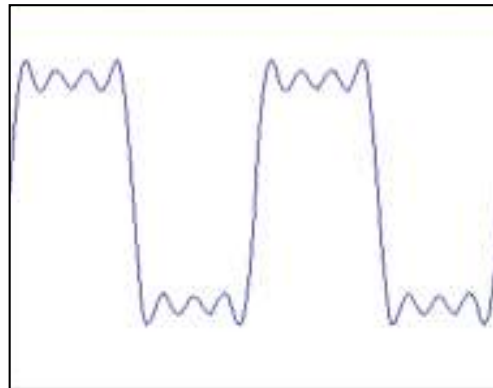
$\approx$



+

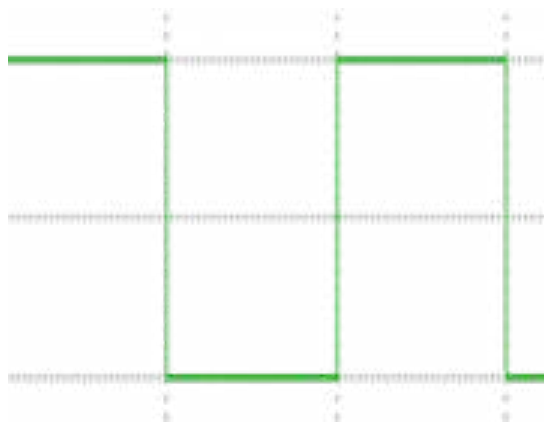


$=$



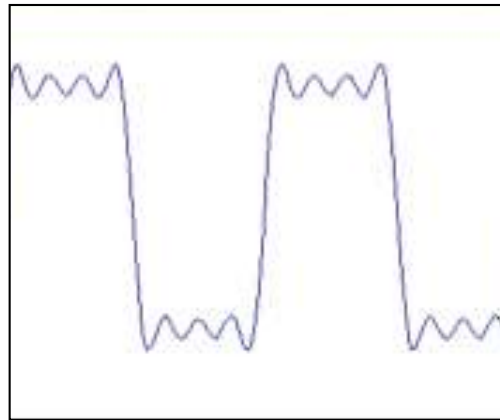
# Examples

How would you generate this function?

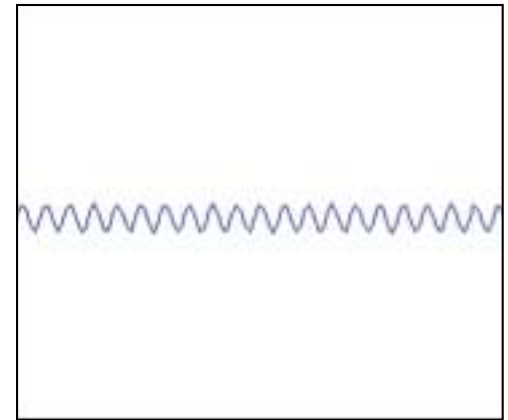


square wave

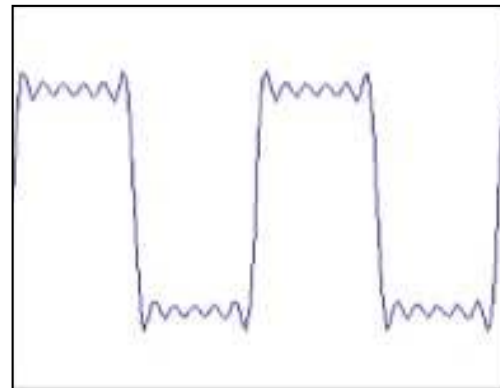
$\approx$



+

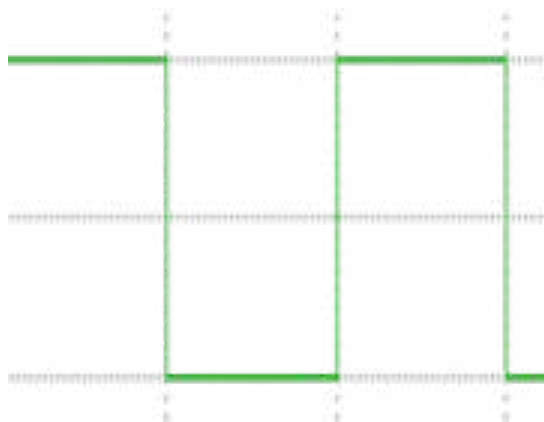


$=$



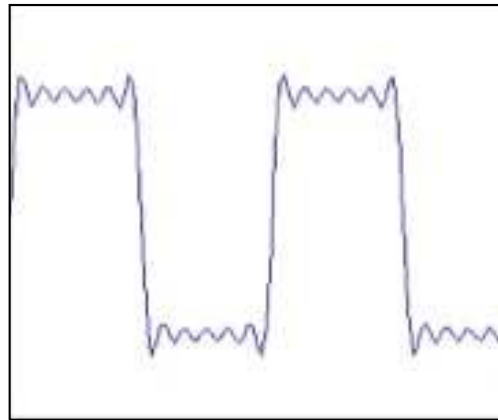
# Examples

How would you generate this function?

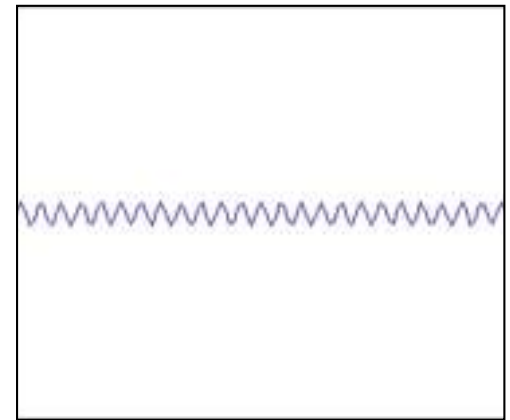


square wave

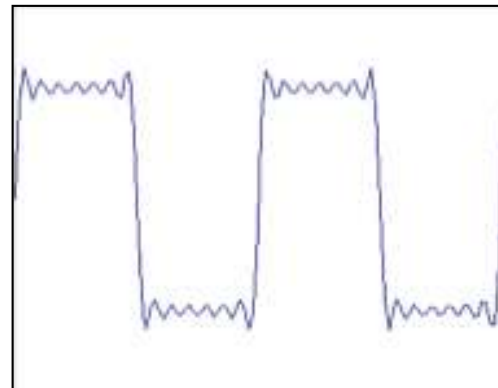
$\approx$



+

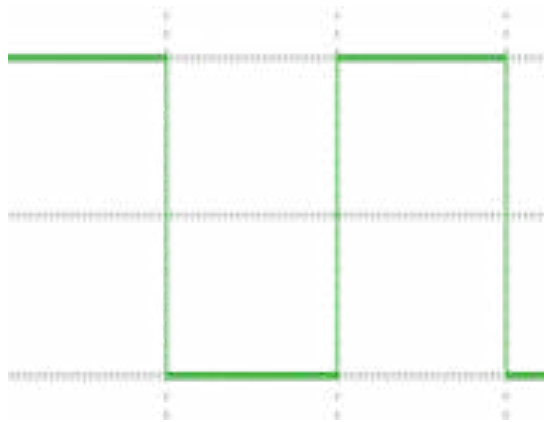


$=$



How would you express  
this mathematically?

# Examples



square wave

=

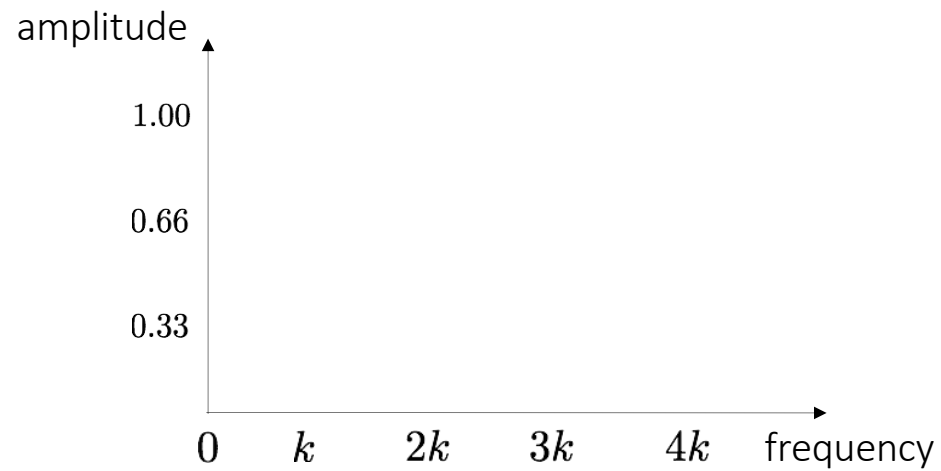
$$A \sum_{k=1,3,5,\dots}^{\infty} \frac{1}{k} \sin(2\pi kx)$$

infinite sum of sine waves

How would could you visualize this in the frequency domain?

Frequency domain

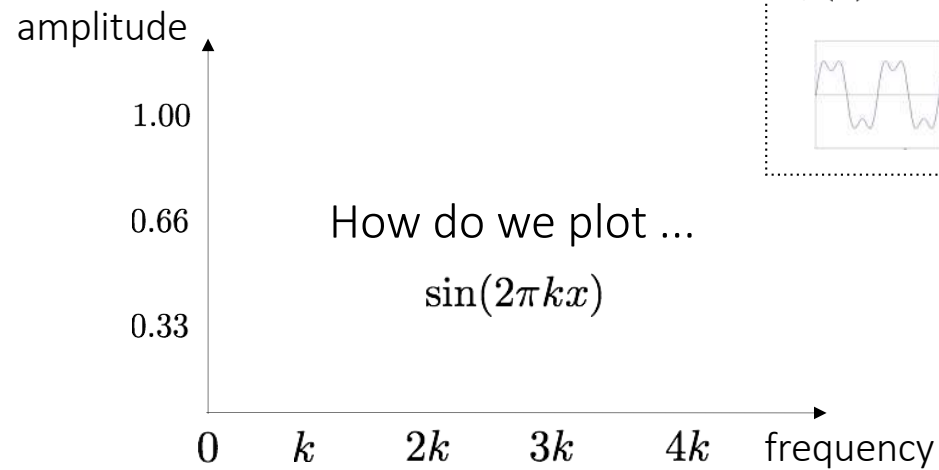
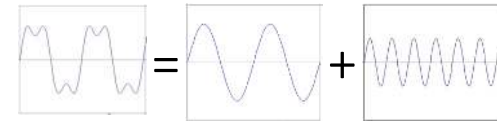
# Visualizing the frequency spectrum



# Visualizing the frequency spectrum

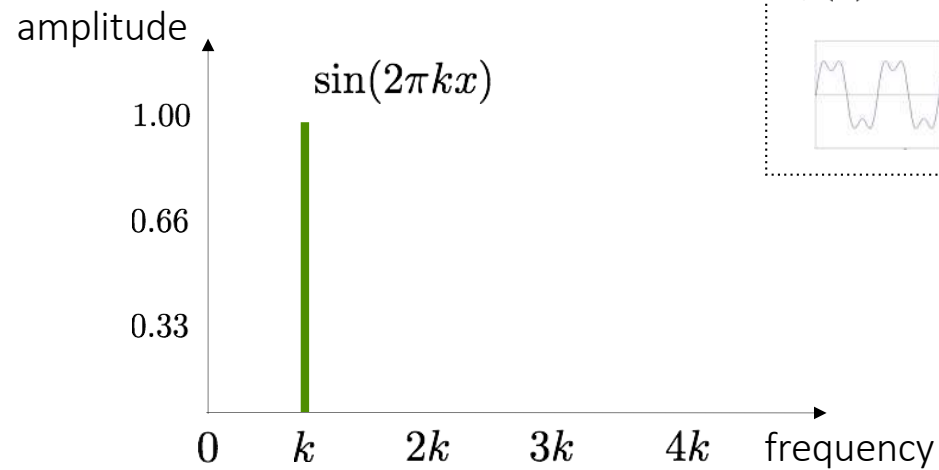
Recall the temporal domain visualization

$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$

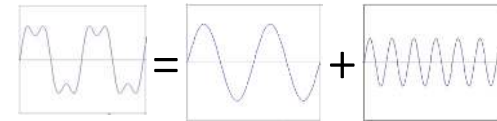


# Visualizing the frequency spectrum

Recall the temporal domain visualization

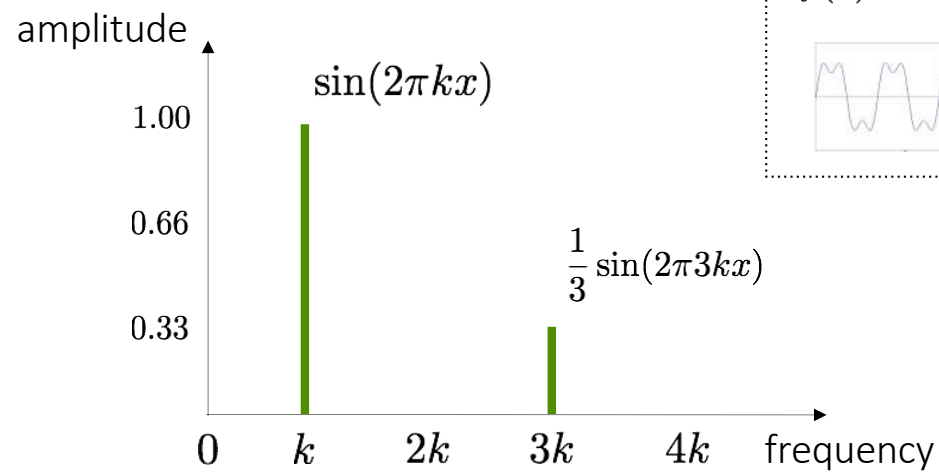


$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$

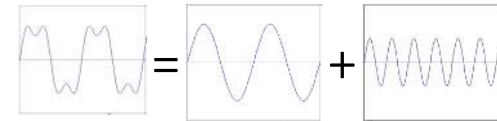


# Visualizing the frequency spectrum

Recall the temporal domain visualization



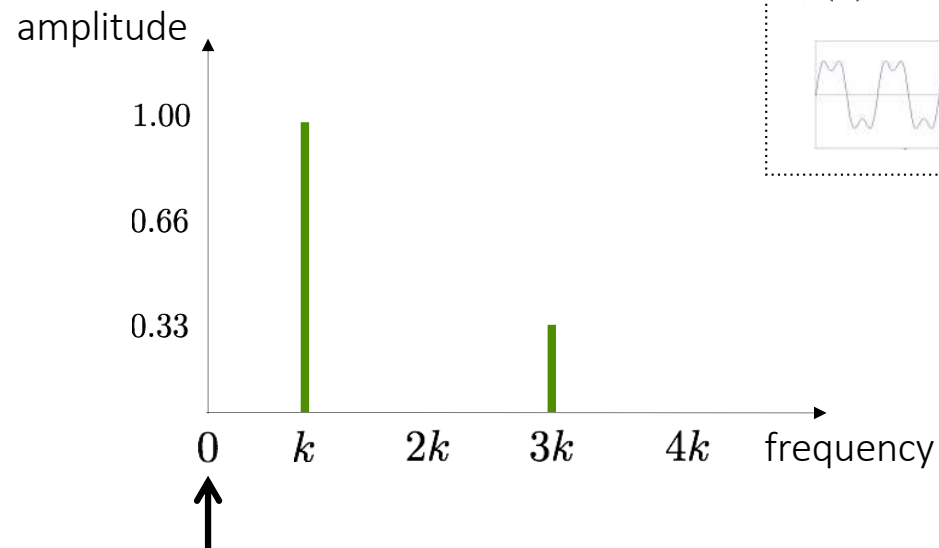
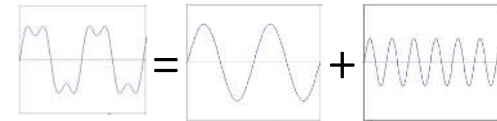
$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$



# Visualizing the frequency spectrum

Recall the temporal domain visualization

$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$

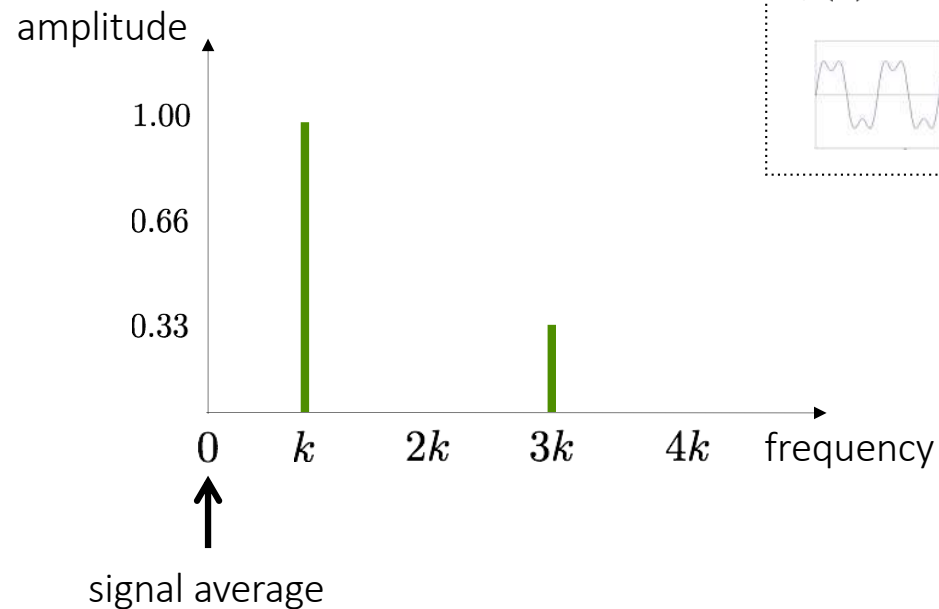
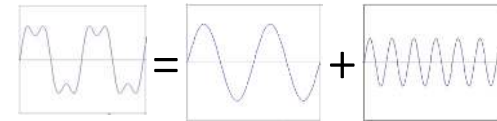


What is at zero frequency?

# Visualizing the frequency spectrum

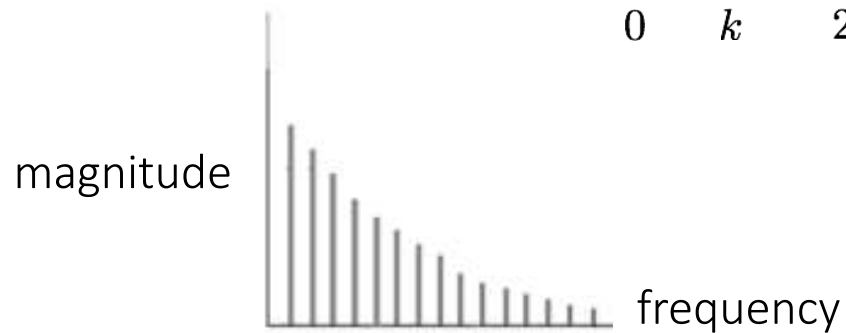
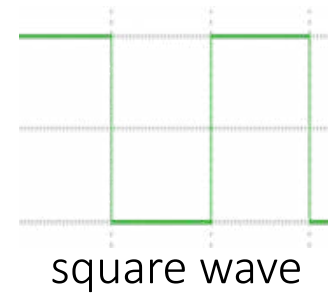
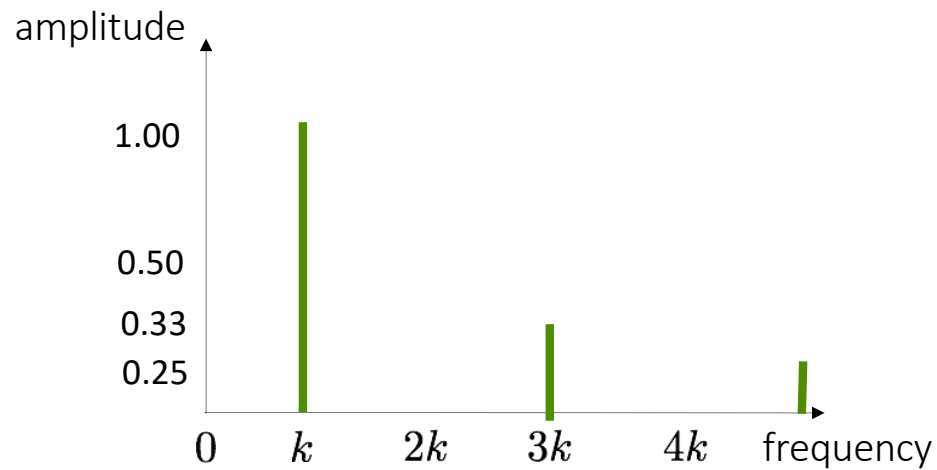
Recall the temporal domain visualization

$$f(x) = \sin(2\pi kx) + \frac{1}{3} \sin(2\pi 3kx)$$

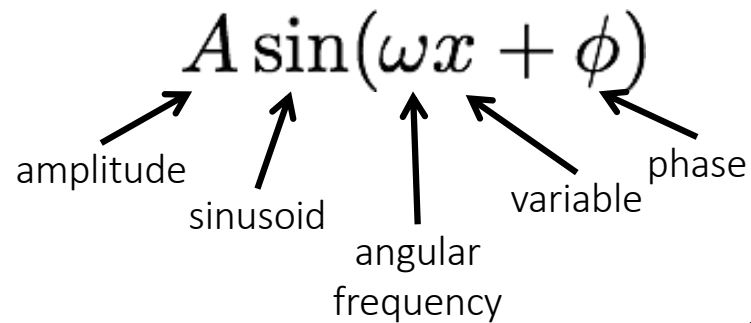


# Visualizing the frequency spectrum

$$A \sum_{k=1,3,5,\dots}^{\infty} \frac{1}{k} \sin(2\pi kx)$$



# Basic building block

$$A \sin(\omega x + \phi)$$


amplitude

sinusoid

angular frequency

variable

phase

What about non-periodic signals?

Fourier's claim: Add enough of these to get any periodic signal you want!

Fourier transform

# Fourier transform

Fourier transform

inverse Fourier transform

continuous

$$F(k) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi kx} dx$$

$$f(x) = \int_{-\infty}^{\infty} F(k) e^{j2\pi kx} dk$$

discrete

$$F(k) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) e^{-j2\pi kx/N}$$

$k = 0, 1, 2, \dots, N-1$

$$f(x) = \sum_{k=0}^{N-1} F(k) e^{j2\pi kx/N}$$

$x = 0, 1, 2, \dots, N-1$

Where is the connection to the 'summation of sine waves' idea?

# Fourier transform

Fourier transform

inverse Fourier transform

continuous

$$F(k) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi kx} dx$$

$$f(x) = \int_{-\infty}^{\infty} F(k) e^{j2\pi kx} dk$$

discrete

$$F(k) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) e^{-j2\pi kx/N}$$

$k = 0, 1, 2, \dots, N-1$

$$f(x) = \sum_{k=0}^{N-1} F(k) e^{j2\pi kx/N}$$

$x = 0, 1, 2, \dots, N-1$

Where is the connection to the 'summation of sine waves' idea?

# Fourier transform

Where is the connection to the ‘summation of sine waves’ idea?

$$f(x) = \sum_{k=0}^{N-1} F(k) e^{j2\pi kx/N}$$

Euler's formula  
 $e^{j\theta} = \cos \theta + j \sin \theta$

sum over frequencies

$$f(x) = \sum_{k=0}^{N-1} F(k) \left\{ \cos(2\pi kx) + j \sin(2\pi kx) \right\}$$

scaling parameter

wave components

## 3+ Main Takeaways

- A linear time-invariant system is fully characterized by its impulse response
- You can represent any signal as a sum of complex exponentials
- Convolution is multiplication in the frequency domain

# Fourier transform

Fourier transform

inverse Fourier transform

continuous

$$F(k) = \int_{-\infty}^{\infty} f(x) e^{-j2\pi kx} dx$$

$$f(x) = \int_{-\infty}^{\infty} F(k) e^{j2\pi kx} dk$$

discrete

$$F(k) = \frac{1}{N} \sum_{x=0}^{N-1} f(x) e^{-j2\pi kx/N}$$

$k = 0, 1, 2, \dots, N-1$

$$f(x) = \sum_{k=0}^{N-1} F(k) e^{j2\pi kx/N}$$

$x = 0, 1, 2, \dots, N-1$

# Discrete Fourier Transform

Discrete Fourier Transform (DFT) transforms a signal  $f[n]$  into  $F[u]$  as:

$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$

The inverse of the DFT is:

$$f[n] = \frac{1}{N} \sum_{u=0}^{N-1} F[u] \exp\left(2\pi j \frac{un}{N}\right)$$

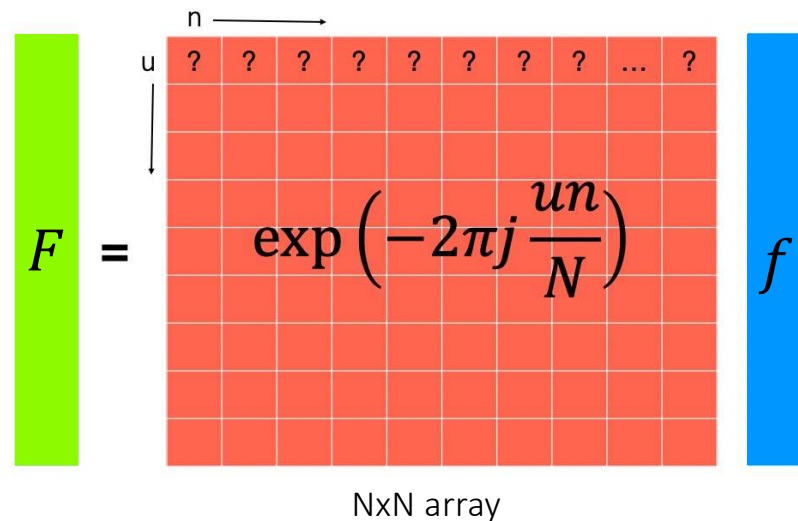
The signal  $f[n]$  is a weighted linear combination of complex exponentials with weights  $F[u]$

# Discrete Fourier Transform

Discrete Fourier Transform (DFT) transforms a signal  $f[n]$  into  $F[u]$  as:

$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$

Discrete Fourier Transform (DFT) is a linear operator. Therefore, we can write:



$$F = \begin{matrix} & \begin{matrix} n \longrightarrow \\ ? & ? & ? & ? & ? & ? & ? & ? & \dots & ? \end{matrix} \\ \begin{matrix} u \downarrow \\ \vdots \end{matrix} & \begin{matrix} \exp\left(-2\pi j \frac{un}{N}\right) \end{matrix} \end{matrix} f$$

NxN array

# Visualizing the Fourier transform

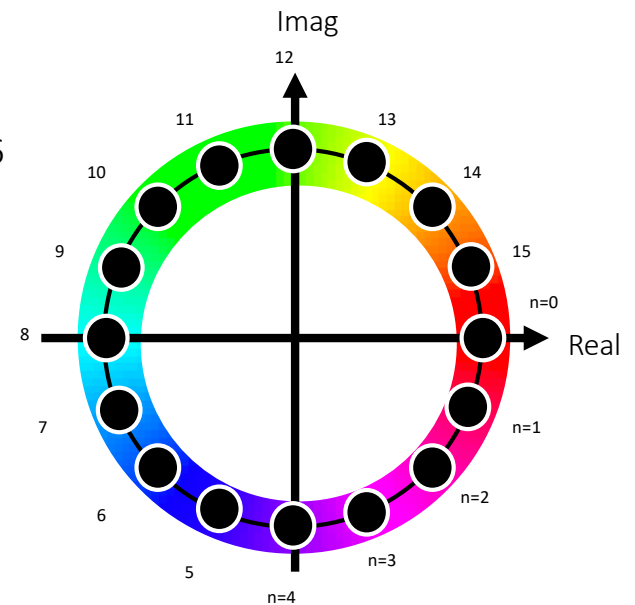
$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$



$$\exp(\alpha j) = \cos(\alpha) + j \sin(\alpha)$$

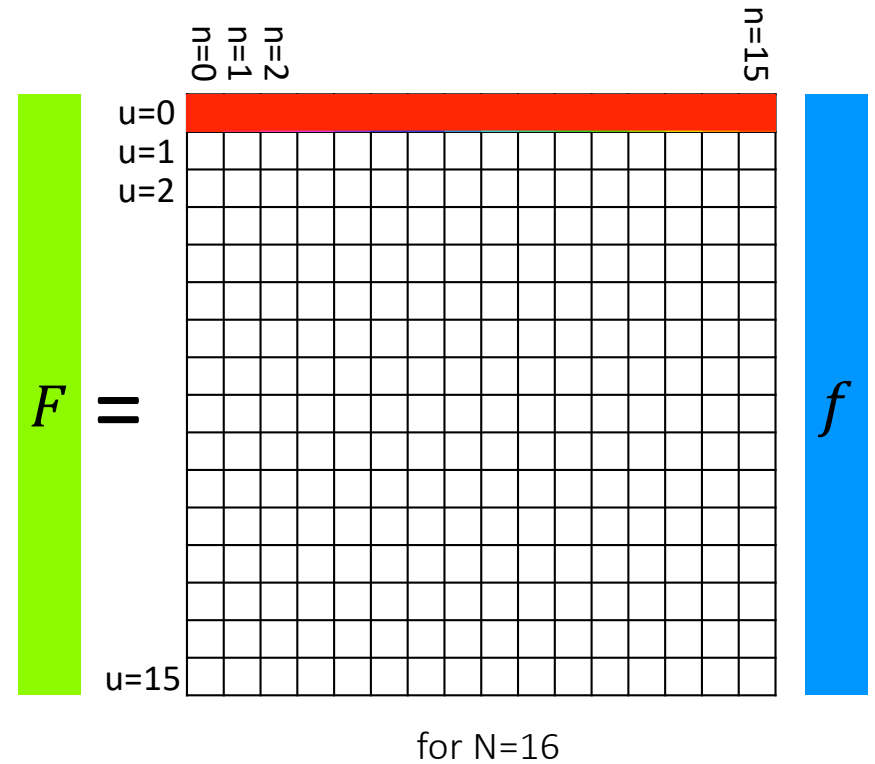
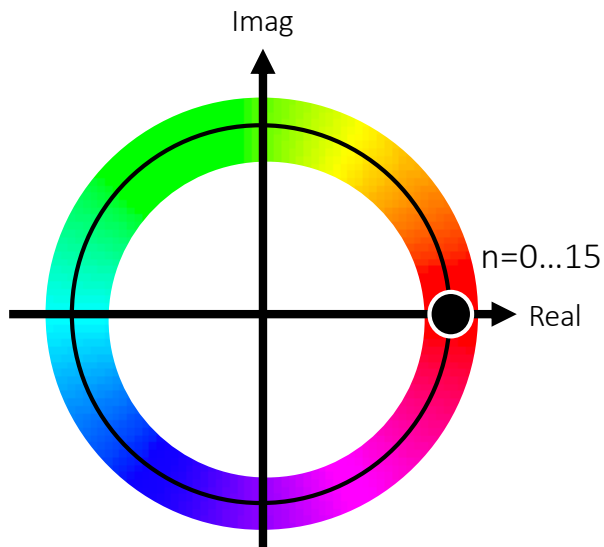
$$\cos\left(2\pi \frac{un}{N}\right) - j \sin\left(2\pi \frac{un}{N}\right)$$

For:  
u=1  
N=16



# Visualizing the Fourier transform

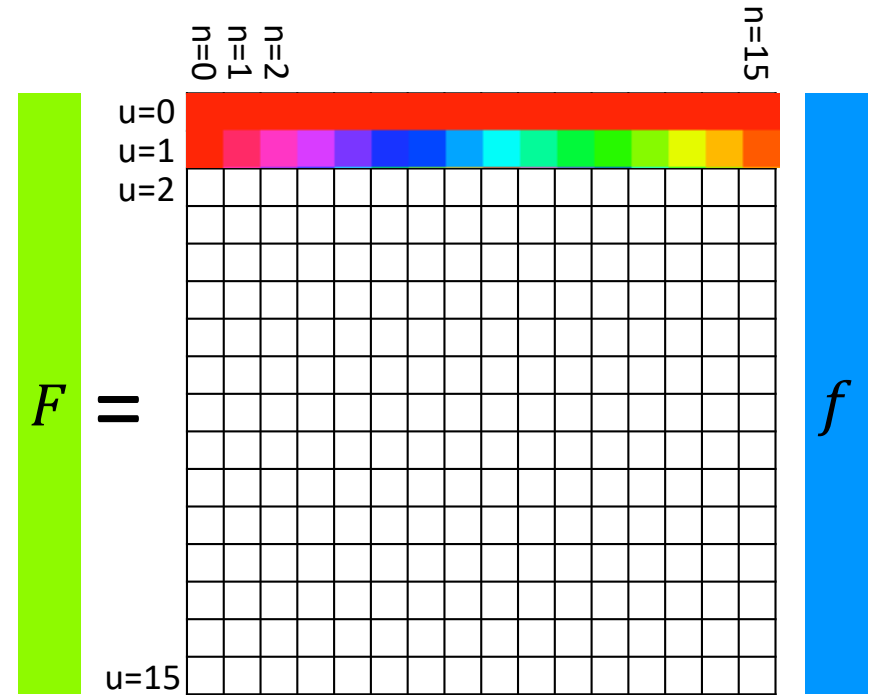
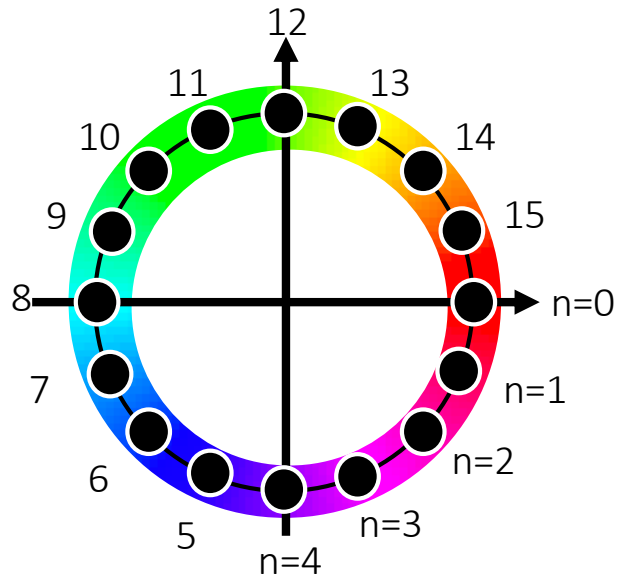
$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$



# Visualizing the Fourier transform

$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$

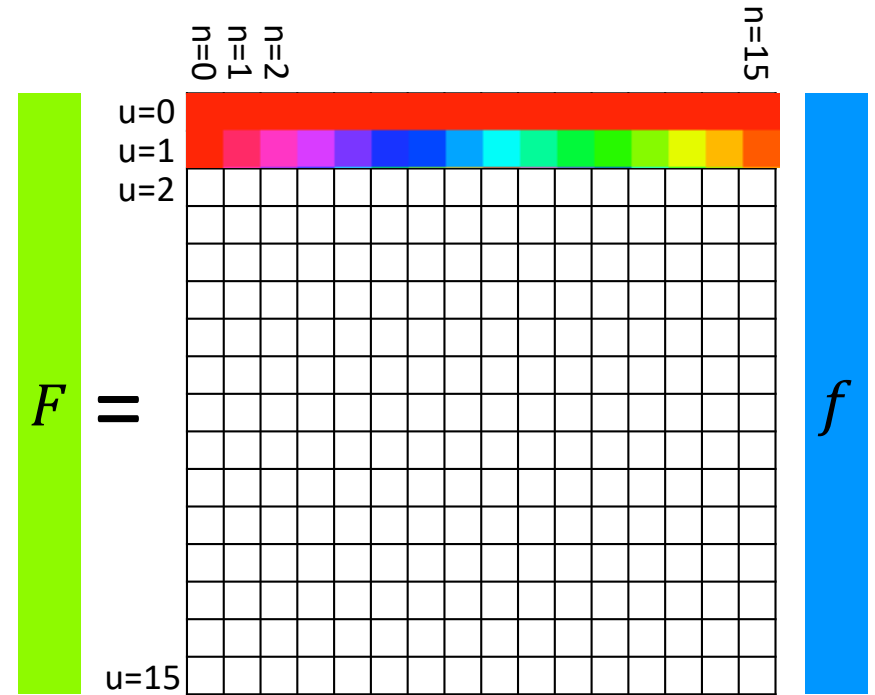
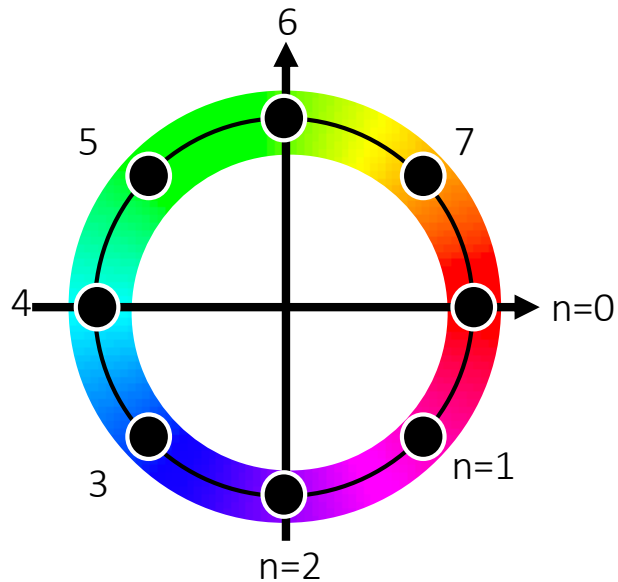
For N=16



# Visualizing the Fourier transform

$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$

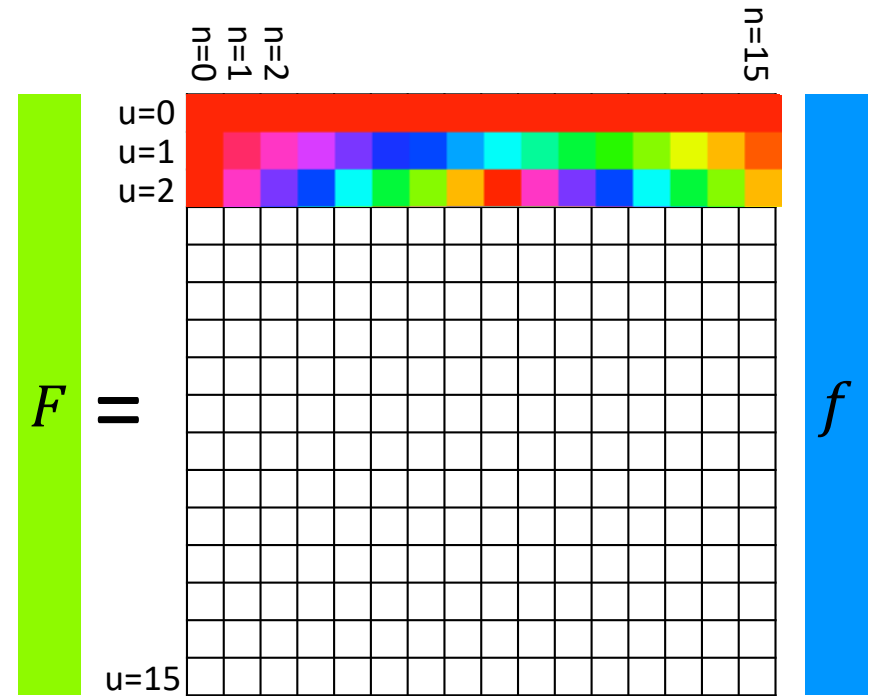
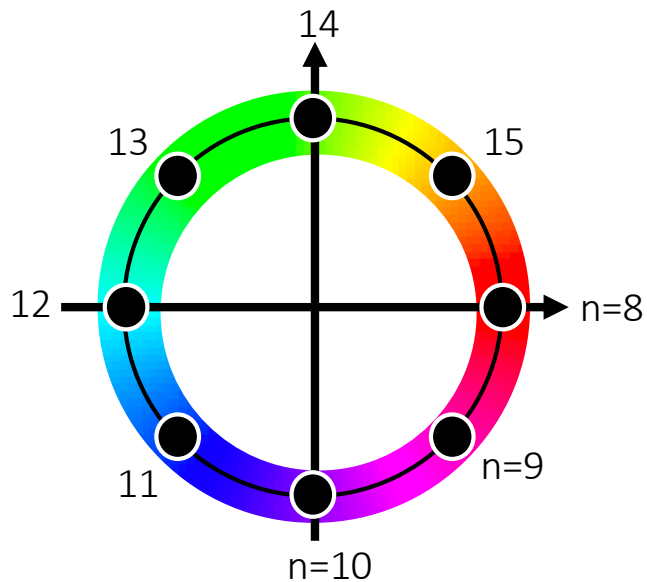
For N=16



# Visualizing the Fourier transform

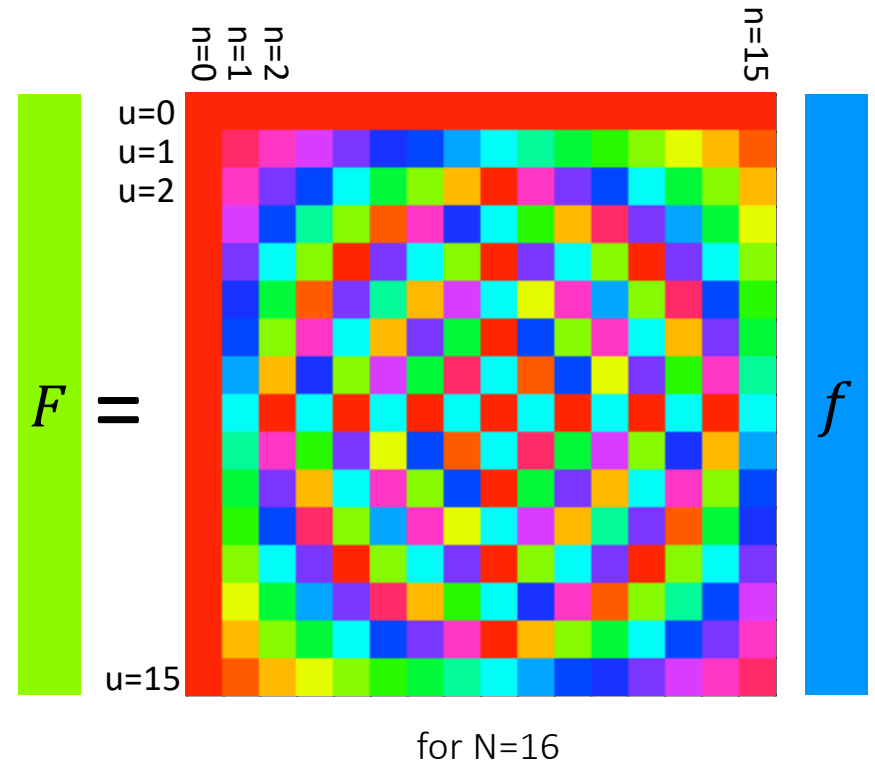
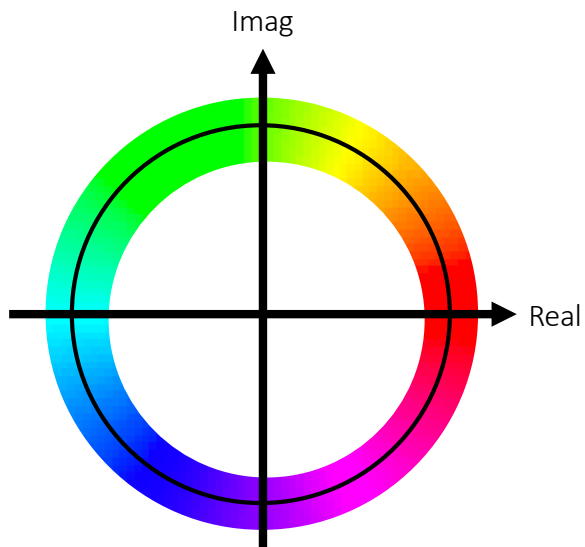
$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$

For N=16



# Visualizing the Fourier transform

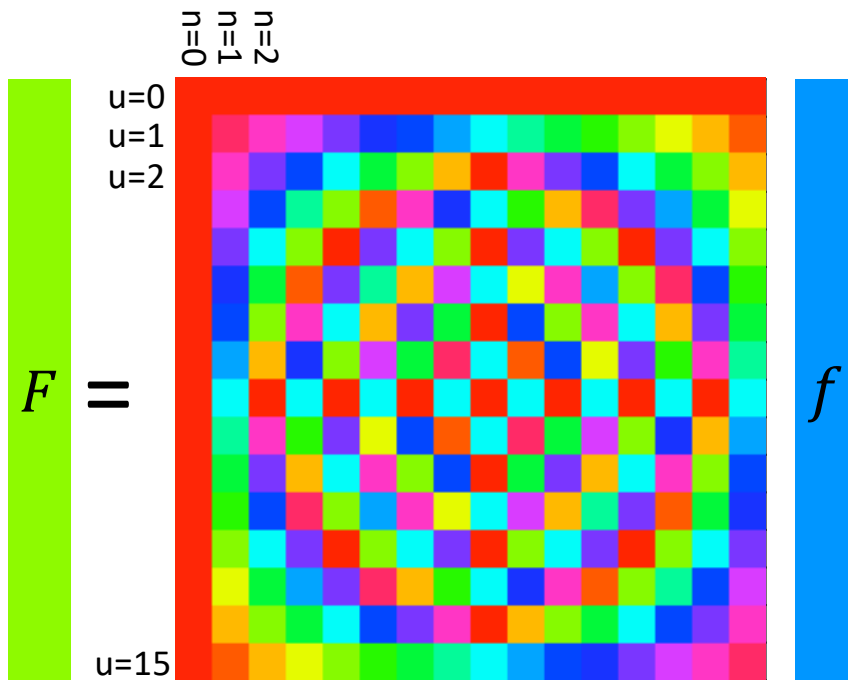
$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$



# Visualizing the Fourier transform

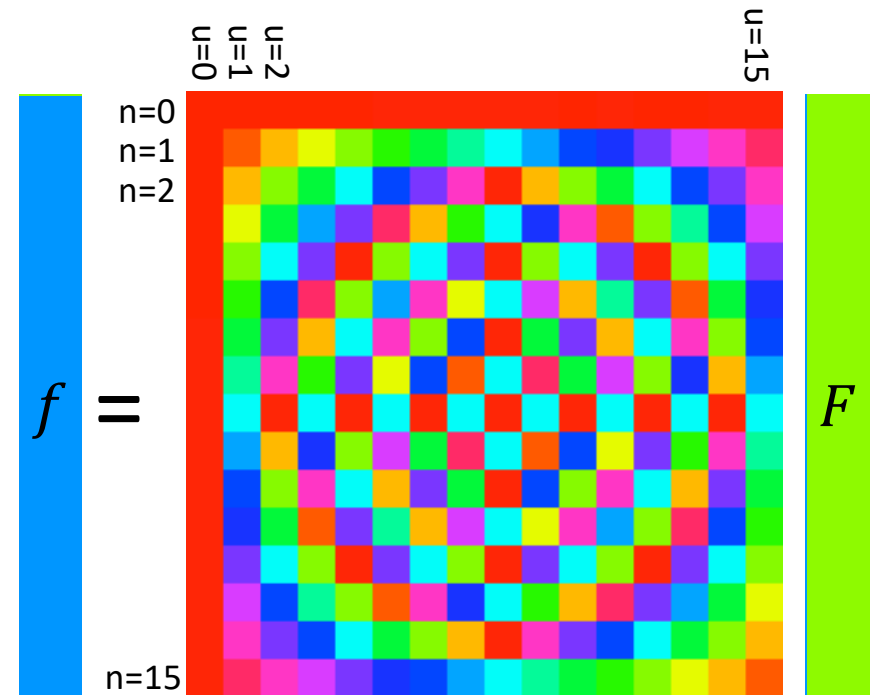
Discrete Fourier Transform (DFT):

$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$



Inverse Discrete Fourier Transform:

$$f[n] = \frac{1}{N} \sum_{u=0}^{N-1} F[u] \exp\left(2\pi j \frac{un}{N}\right)$$



# 2D Discrete Fourier Transform (for images)

Discrete Fourier Transform (DFT) transforms a signal  $f[n]$  into  $F[u]$  as:

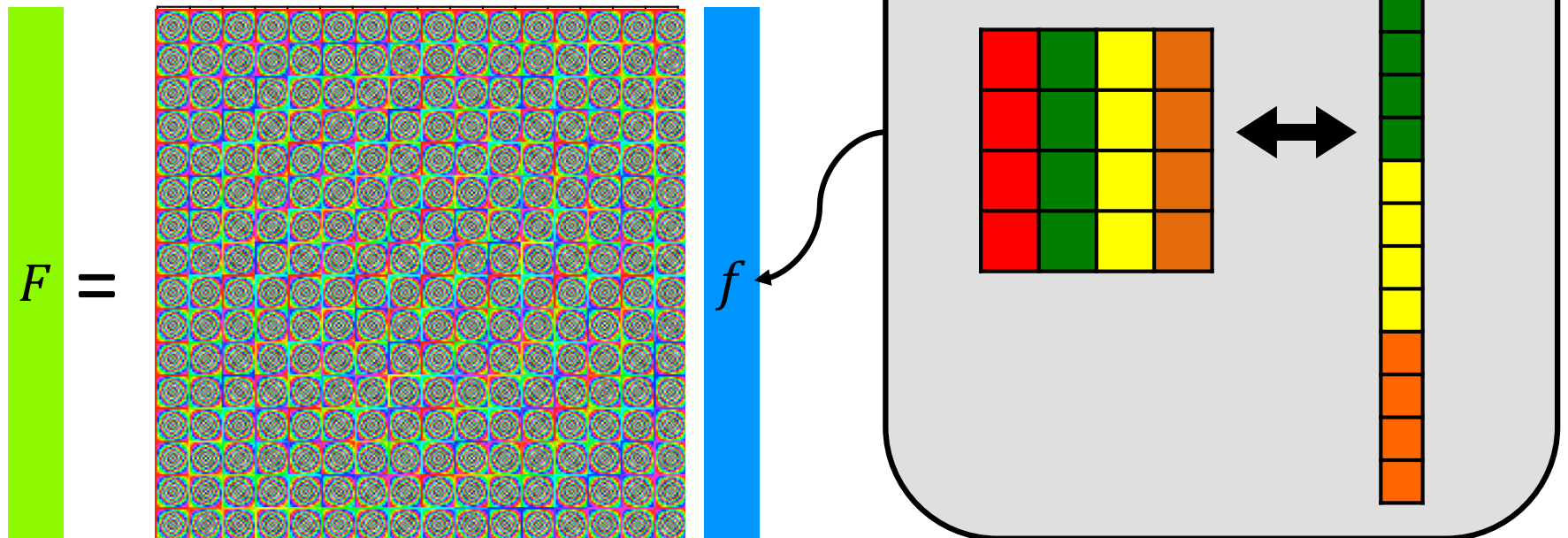
$$F[u] = \sum_{n=0}^{N-1} f[n] \exp\left(-2\pi j \frac{un}{N}\right)$$

2D Discrete Fourier Transform (DFT) transforms an image  $f[n,m]$  into  $F[u,v]$  as:

$$F[u, v] = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} f[n, m] \exp\left(-2\pi j \left(\frac{un}{N} + \frac{vm}{M}\right)\right)$$

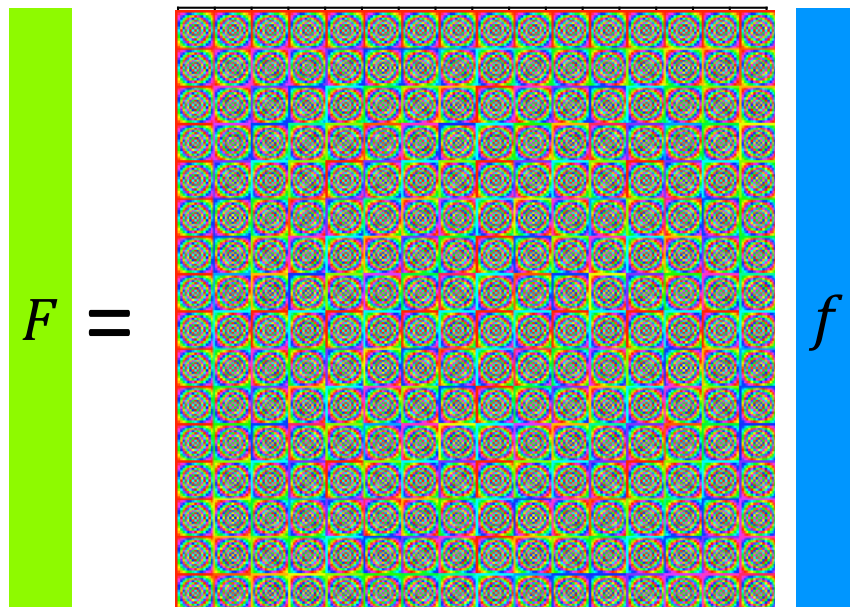
# Visualizing the 2D Fourier transform

$$F[u, v] = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} f[n, m] \exp \left( -2\pi j \left( \frac{un}{N} + \frac{vm}{M} \right) \right)$$



# Visualizing the 2D Fourier transform

$$F[u, v] = \sum_{n=0}^{N-1} \sum_{m=0}^{M-1} f[n, m] \exp \left( -2\pi j \left( \frac{un}{N} + \frac{vm}{M} \right) \right)$$



In practice this is implemented using the *fast Fourier transform* (FFT) algorithm.

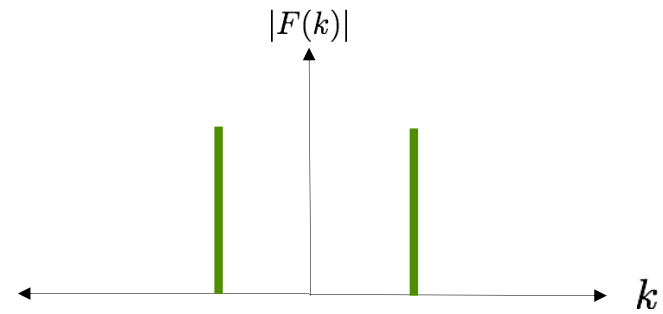
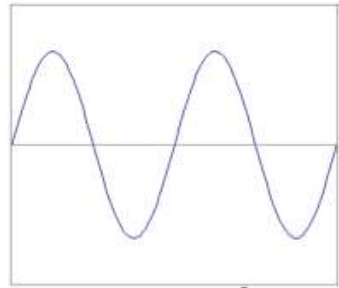
# Visualizing the Fourier transform

# Examples

Spatial domain visualization

Frequency domain visualization

1D



2D

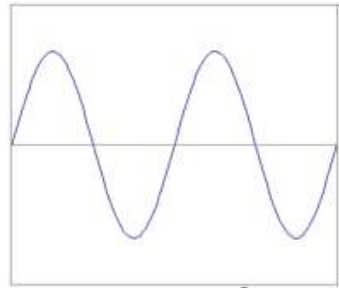


?

# Examples

Spatial domain visualization

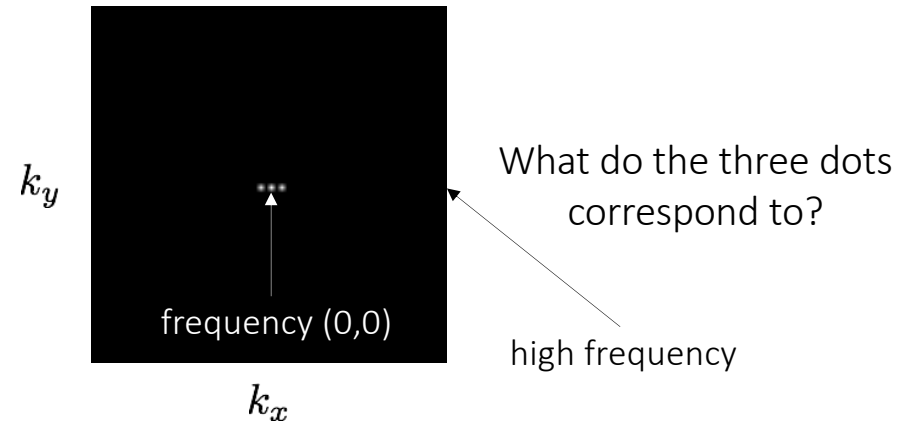
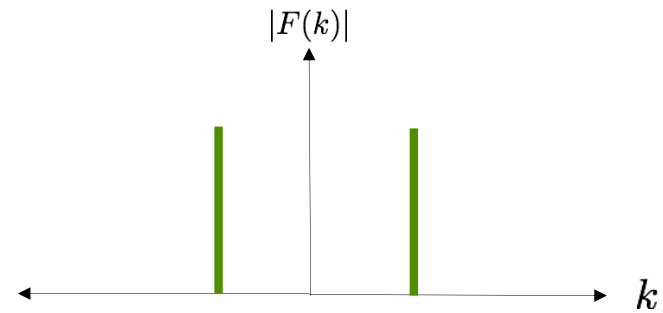
1D



2D

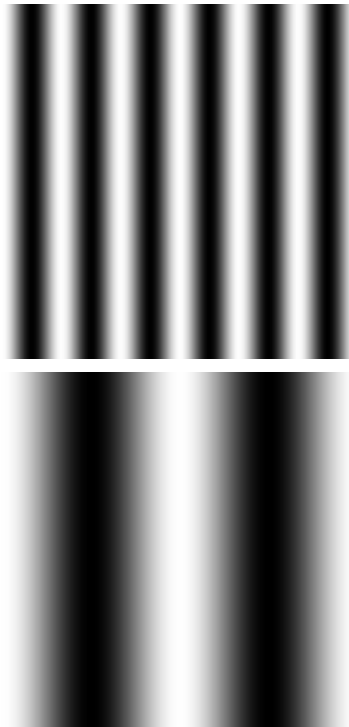


Frequency domain visualization

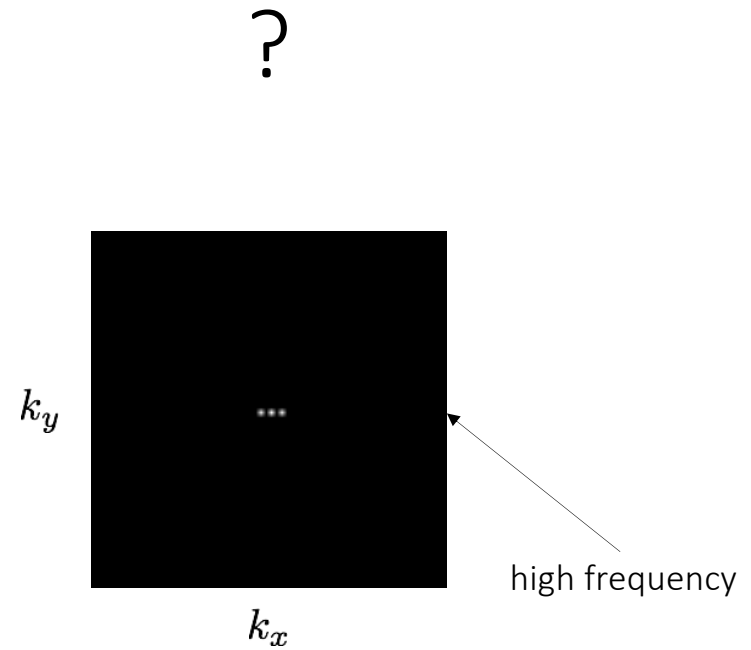


# Examples

Spatial domain visualization

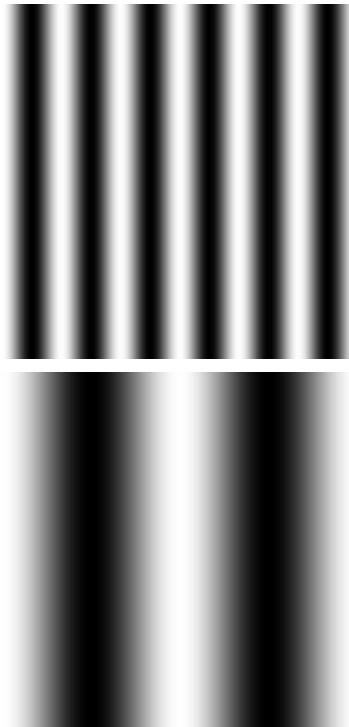


Frequency domain visualization

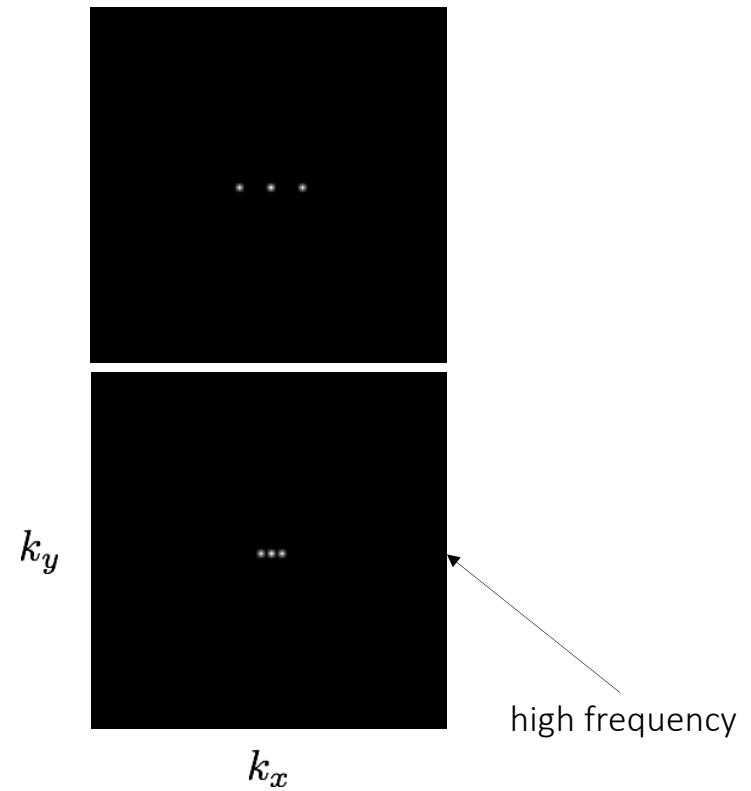


# Examples

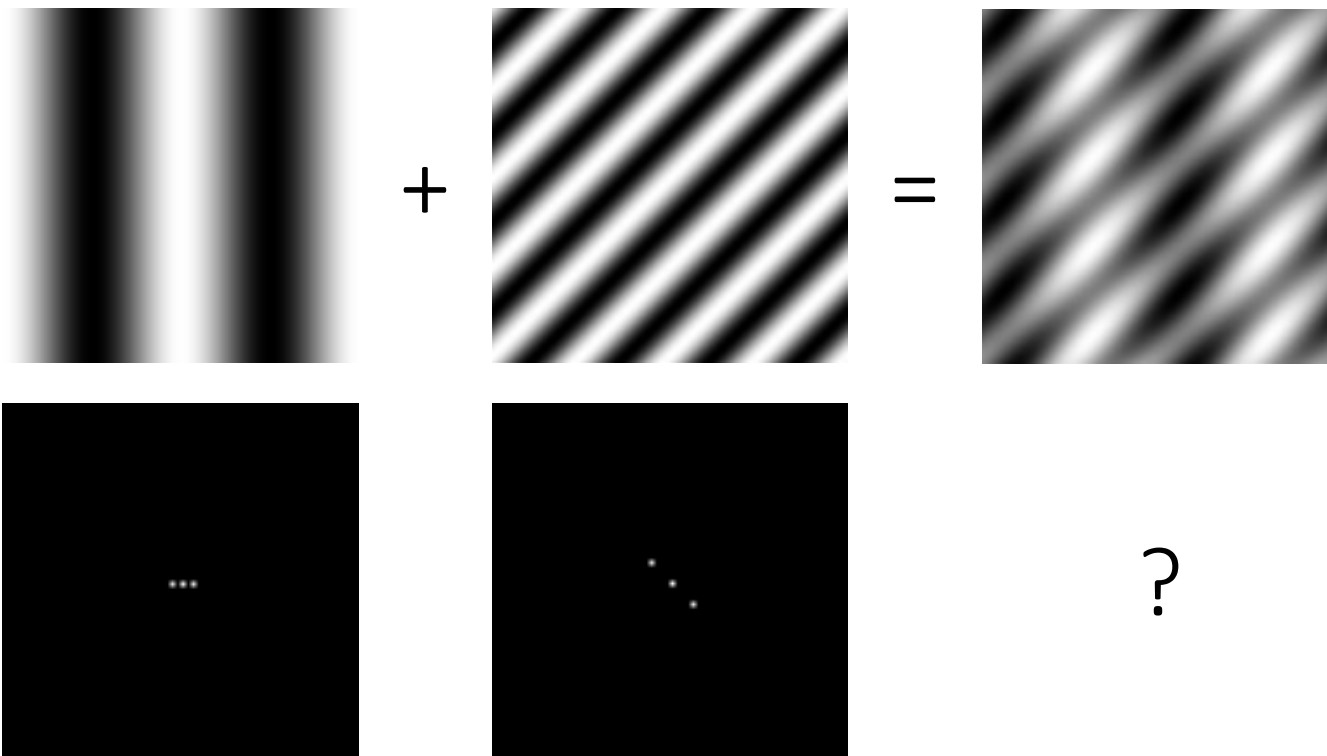
Spatial domain visualization



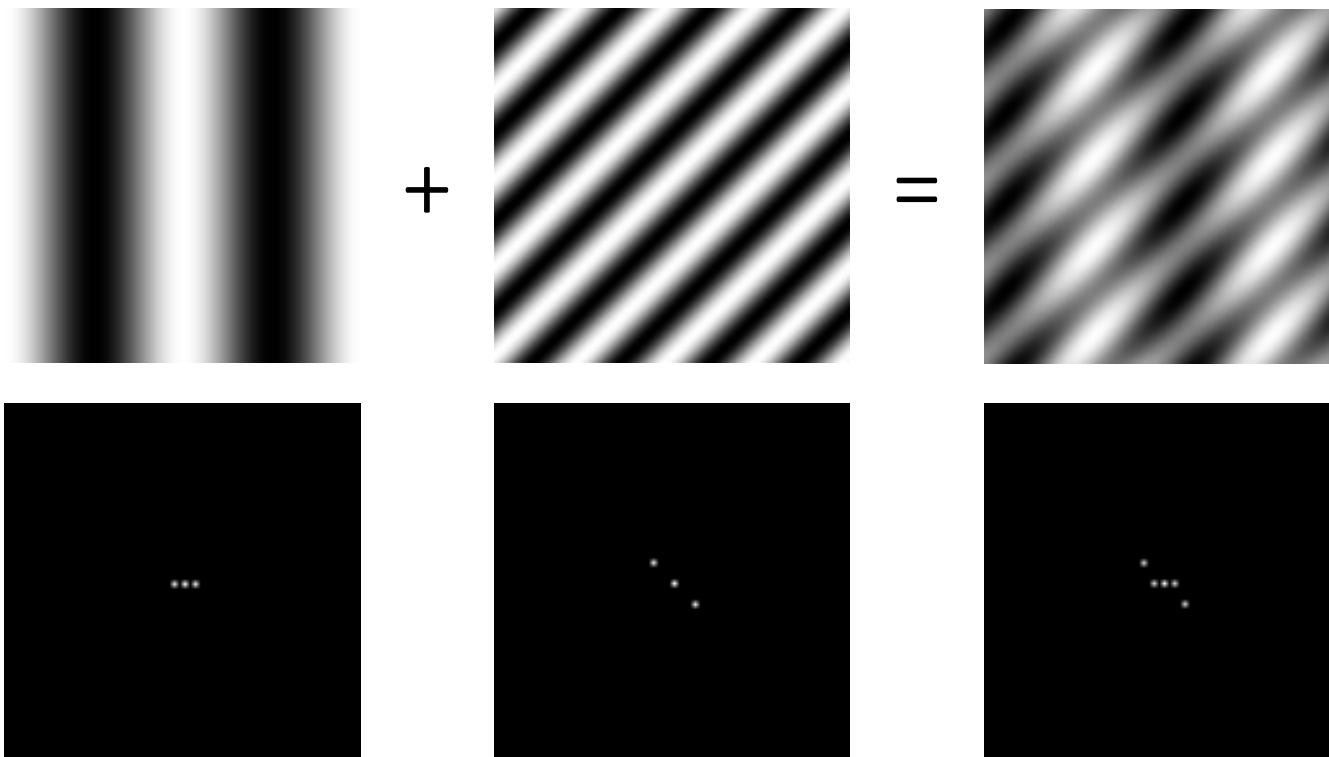
Frequency domain visualization



# Examples



# Examples

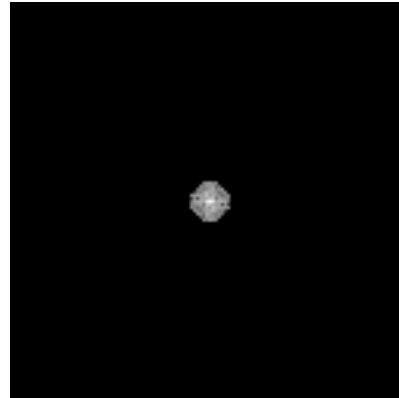


# More filtering examples

original image

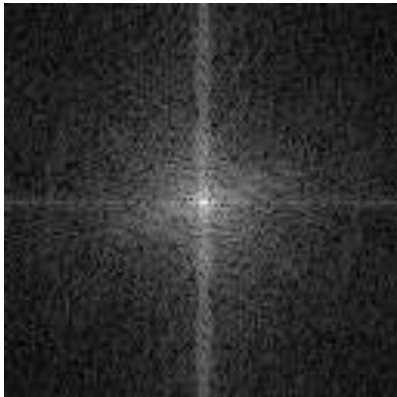


low-pass filter



?

frequency magnitude

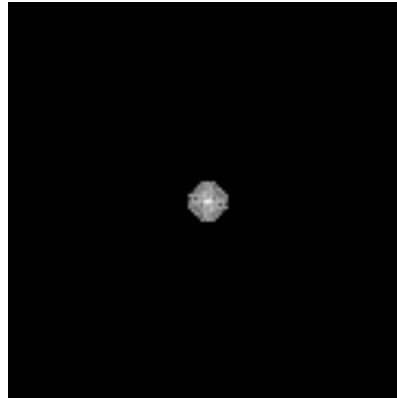


# More filtering examples

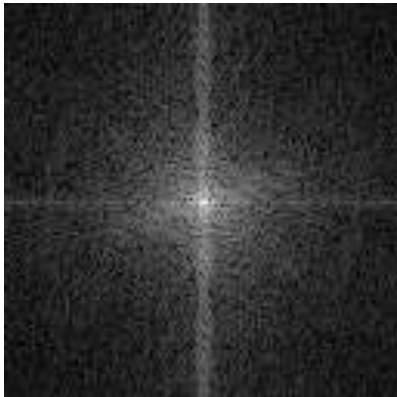
original image



low-pass filter



frequency magnitude

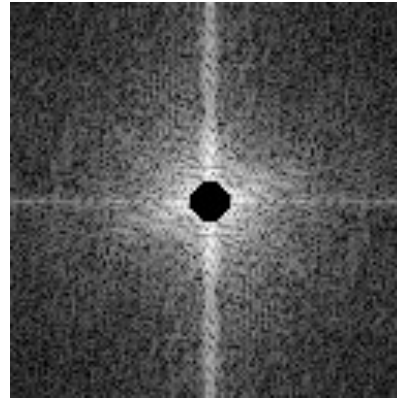


# More filtering examples

original image

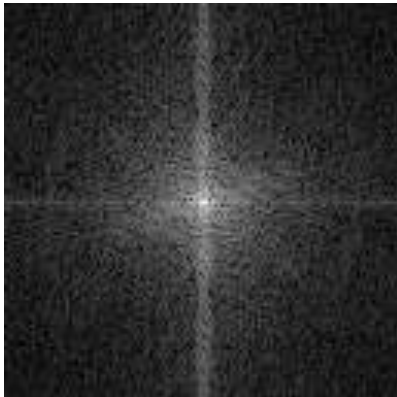


high-pass filter



?

frequency magnitude

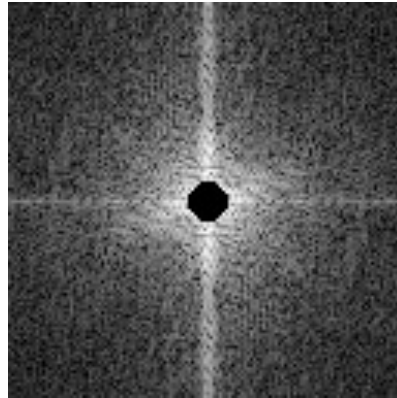


# More filtering examples

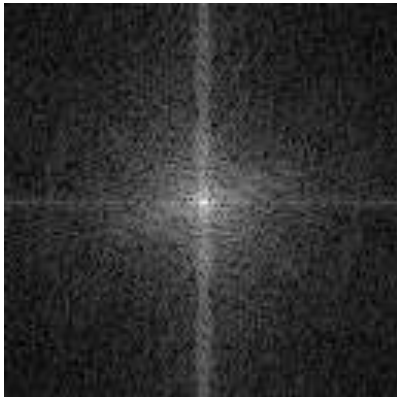
original image



high-pass filter



frequency magnitude

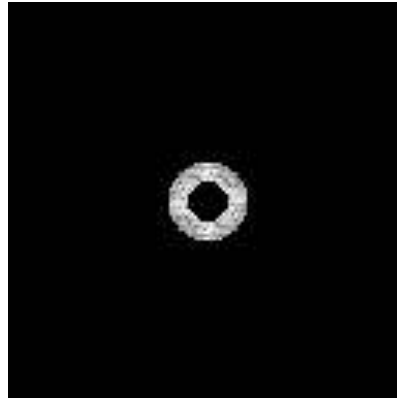


# More filtering examples

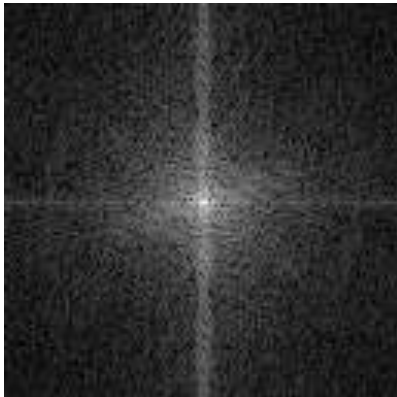
original image



band-pass filter



frequency magnitude

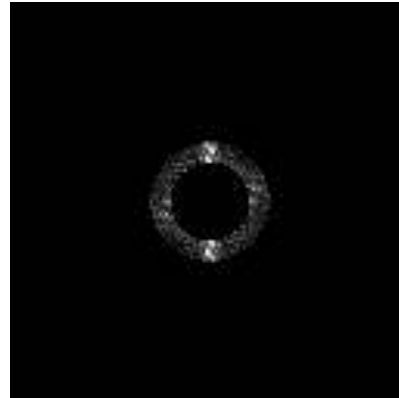


# More filtering examples

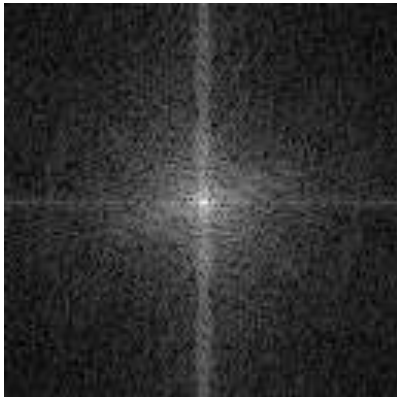
original image



band-pass filter

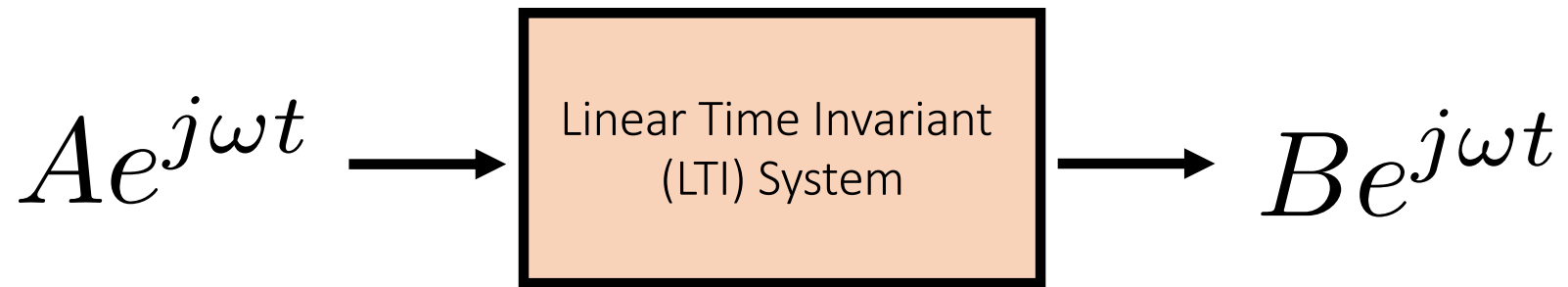


frequency magnitude



Why do we care about all this?

# Eigenfunctions of LTI systems



The complex exponential functions are eigenfunctions of linear time-invariant systems since they pass through the system unchanged except for a multiplicative constant

## Some Fourier Transform Properties

Convolution Theorem

$$\mathcal{F}\{g * h\} = \mathcal{F}\{g\}\mathcal{F}\{h\}$$

Shift Theorem

$$\mathcal{F}\{f(t - a)\} = e^{-j\omega a} \mathcal{F}\{f(t)\}$$

# The convolution theorem

The Fourier transform of the convolution of two functions is the product of their Fourier transforms:

$$\mathcal{F}\{g * h\} = \mathcal{F}\{g\}\mathcal{F}\{h\}$$

The inverse Fourier transform of the product of two Fourier transforms is the convolution of the two inverse Fourier transforms:

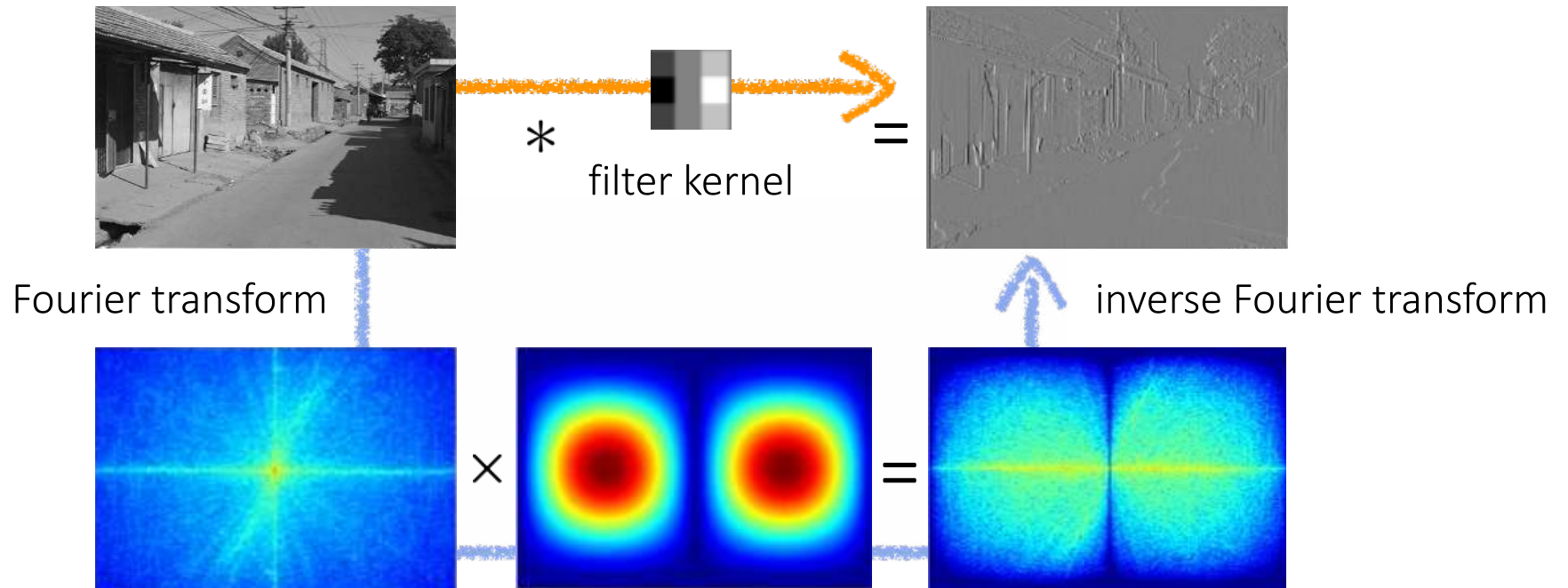
$$\mathcal{F}^{-1}\{gh\} = \mathcal{F}^{-1}\{g\} * \mathcal{F}^{-1}\{h\}$$

Convolution in spatial domain is equivalent to multiplication in frequency domain!

# Convolution

$$\begin{aligned}
 \int_{-\infty}^{\infty} f(k)g(t-k)dk &= \int_{-\infty}^{\infty} \left[ \int_{-\infty}^{\infty} f(k)g(t-k)dk \right] e^{-j\omega t} dt \\
 &= \int_{-\infty}^{\infty} f(k) \left[ \int_{-\infty}^{\infty} g(t-k)e^{-j\omega t} dt \right] dk \quad z = t - k \\
 &= \int_{-\infty}^{\infty} f(k) \left[ \int_{-\infty}^{\infty} g(z)e^{-j\omega z} dz e^{-j\omega k} \right] dk \\
 &= \left[ \int_{-\infty}^{\infty} f(k)e^{-j\omega k} dk \right] \left[ \int_{-\infty}^{\infty} g(z)e^{-j\omega z} dz \right] \\
 &= \mathcal{F}\{f\}\mathcal{F}\{g\} \longleftarrow \text{Convolution is multiplication in frequency domain}
 \end{aligned}$$

## Spatial domain filtering

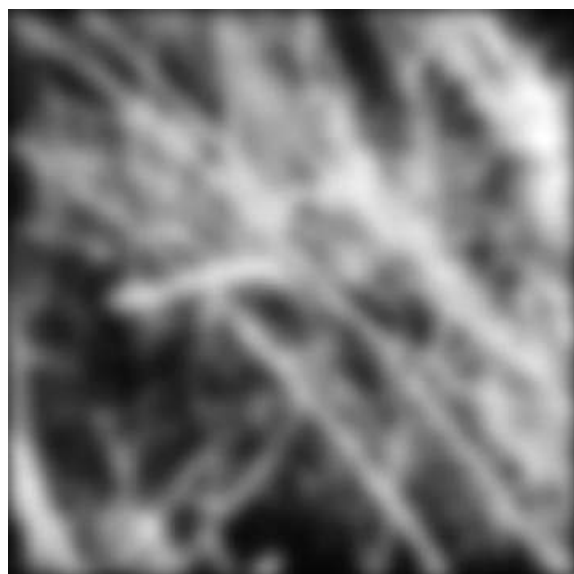


## Frequency domain filtering

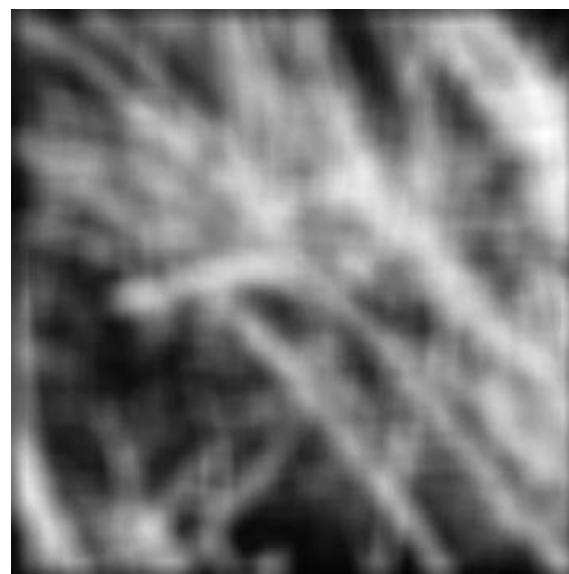
- Note when you use a FFT/DFT to do convolution it uses circular padding

# Revisiting blurring

Why does the Gaussian give a nice smooth image, but the square filter give edgy artifacts?



Gaussian  
filter

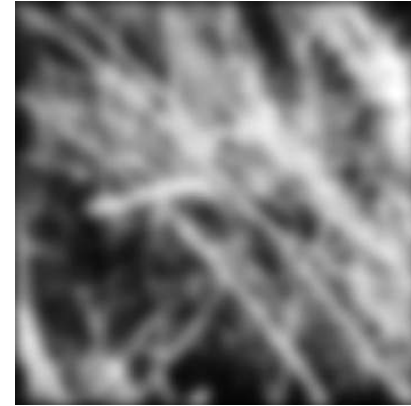
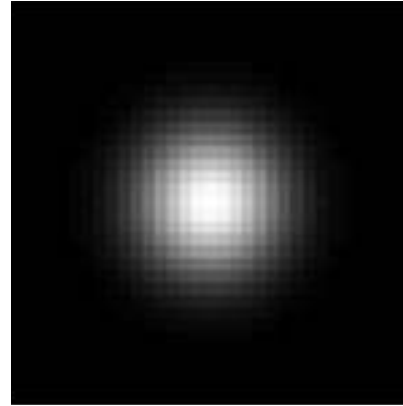


Box  
filter

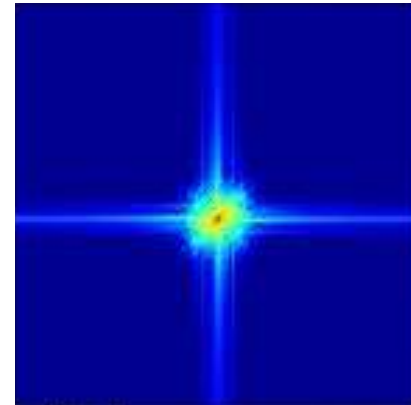
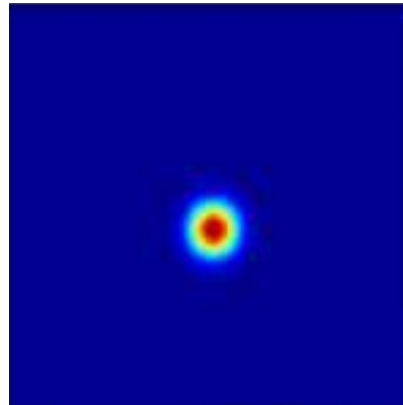
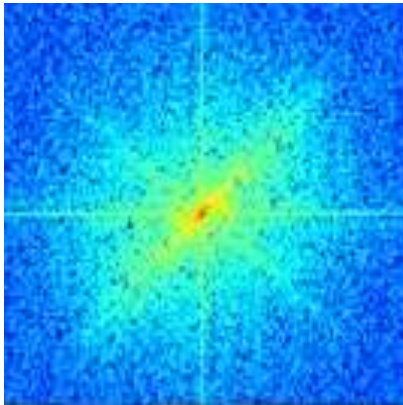


# Gaussian blur

Space

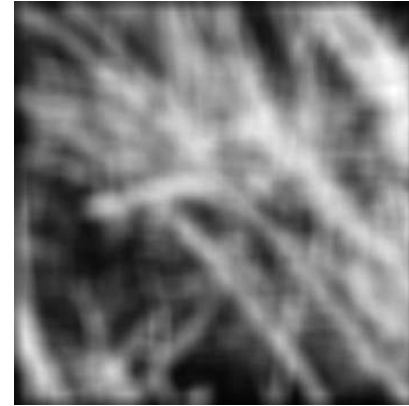
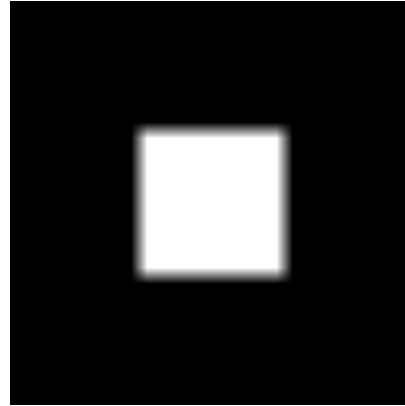


Fourier Transform

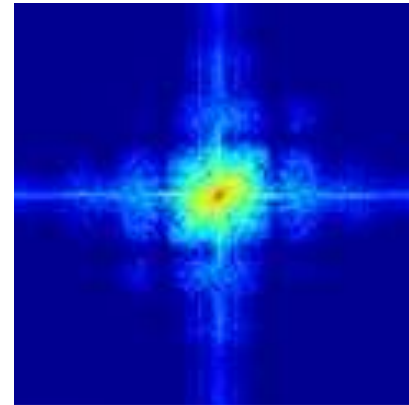
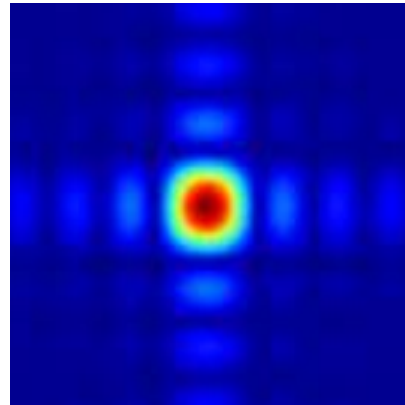
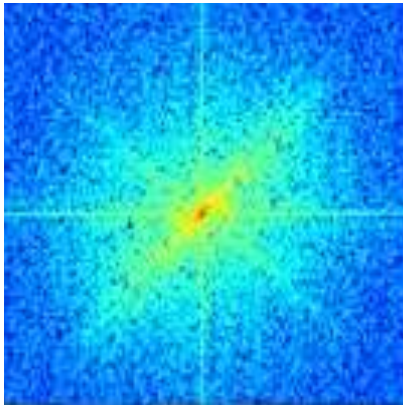


# Box blur

Space



Fourier Transform



## 3+ Main Takeaways

- A linear time-invariant system is fully characterized by its impulse response
- You can represent any signal as a sum of complex exponentials
- Convolution is multiplication in the frequency domain

## Some Fourier Transform Properties

Convolution Theorem

$$\mathcal{F}\{g * h\} = \mathcal{F}\{g\}\mathcal{F}\{h\}$$

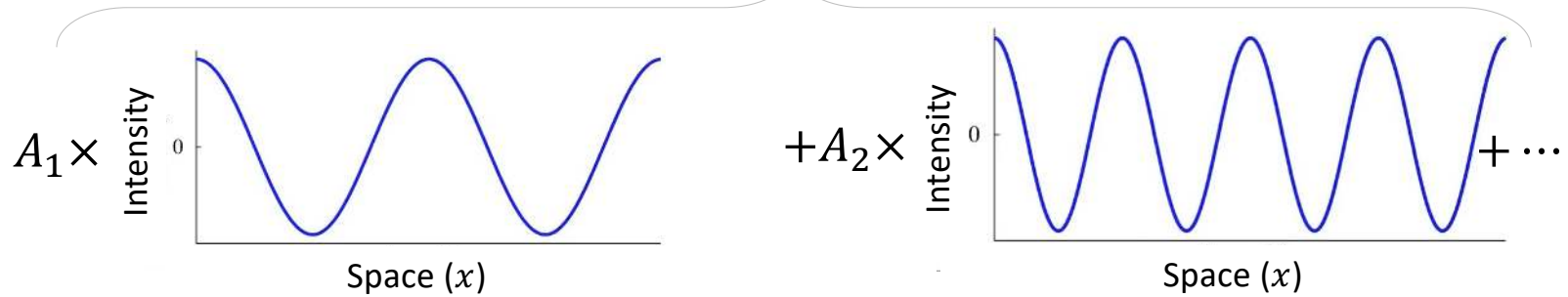
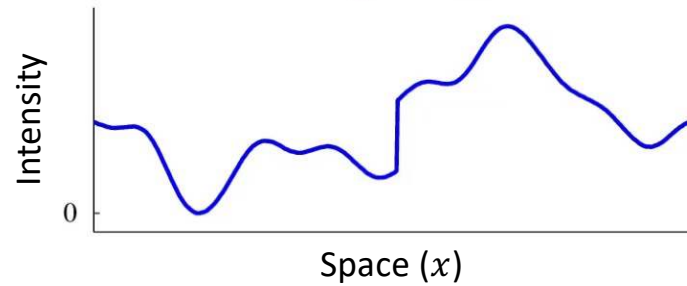
Shift Theorem

$$\mathcal{F}\{f(t - a)\} = e^{-j\omega a} \mathcal{F}\{f(t)\}$$

$$\begin{aligned}
\mathcal{F}\{f(t - a)\} &= \int_{-\infty}^{\infty} f(t) e^{-j\omega t} dt \\
&= \int_{-\infty}^{\infty} f(t - a) e^{-j\omega t} dt \\
&= \int_{-\infty}^{\infty} f(u) e^{-j\omega(u+a)} dt \\
&= e^{-j\omega a} \int_{-\infty}^{\infty} f(u) e^{-j\omega u} dt = e^{-j\omega a} \mathcal{F}\{f(t)\}
\end{aligned}$$

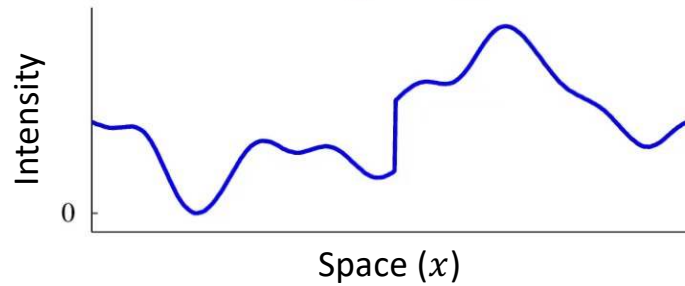
# Fourier Shift Theorem

$$I(x) \xLeftrightarrow{\text{FFT}} \sum_{\omega=-\infty}^{\infty} A_{\omega} e^{i\omega x}$$

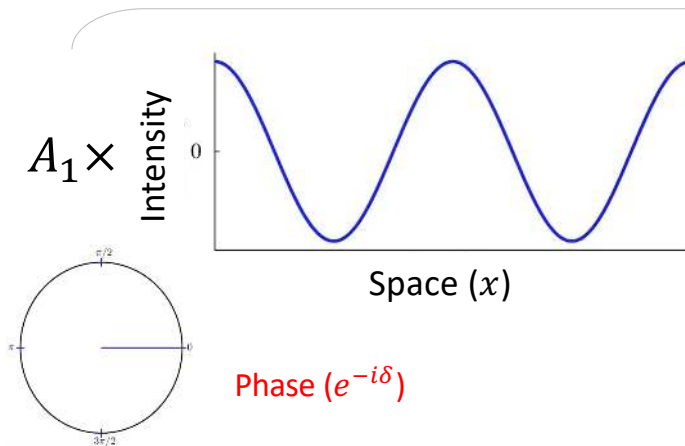


# Fourier Shift Theorem

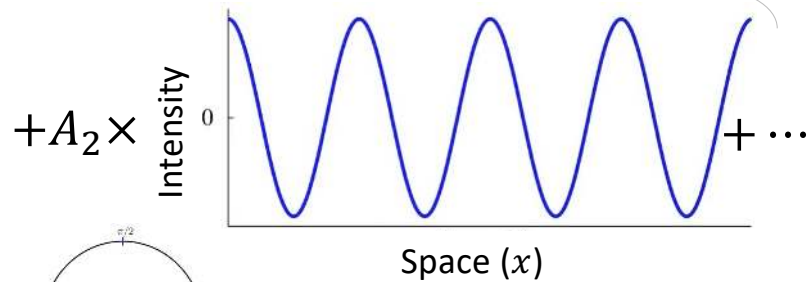
$$I(x - \delta) \xLeftrightarrow{\text{FFT}} \sum_{\omega=-\infty}^{\infty} A_{\omega} e^{i\omega x} e^{-i\omega\delta}$$



**Phase shift  $\Leftrightarrow$  Translation**



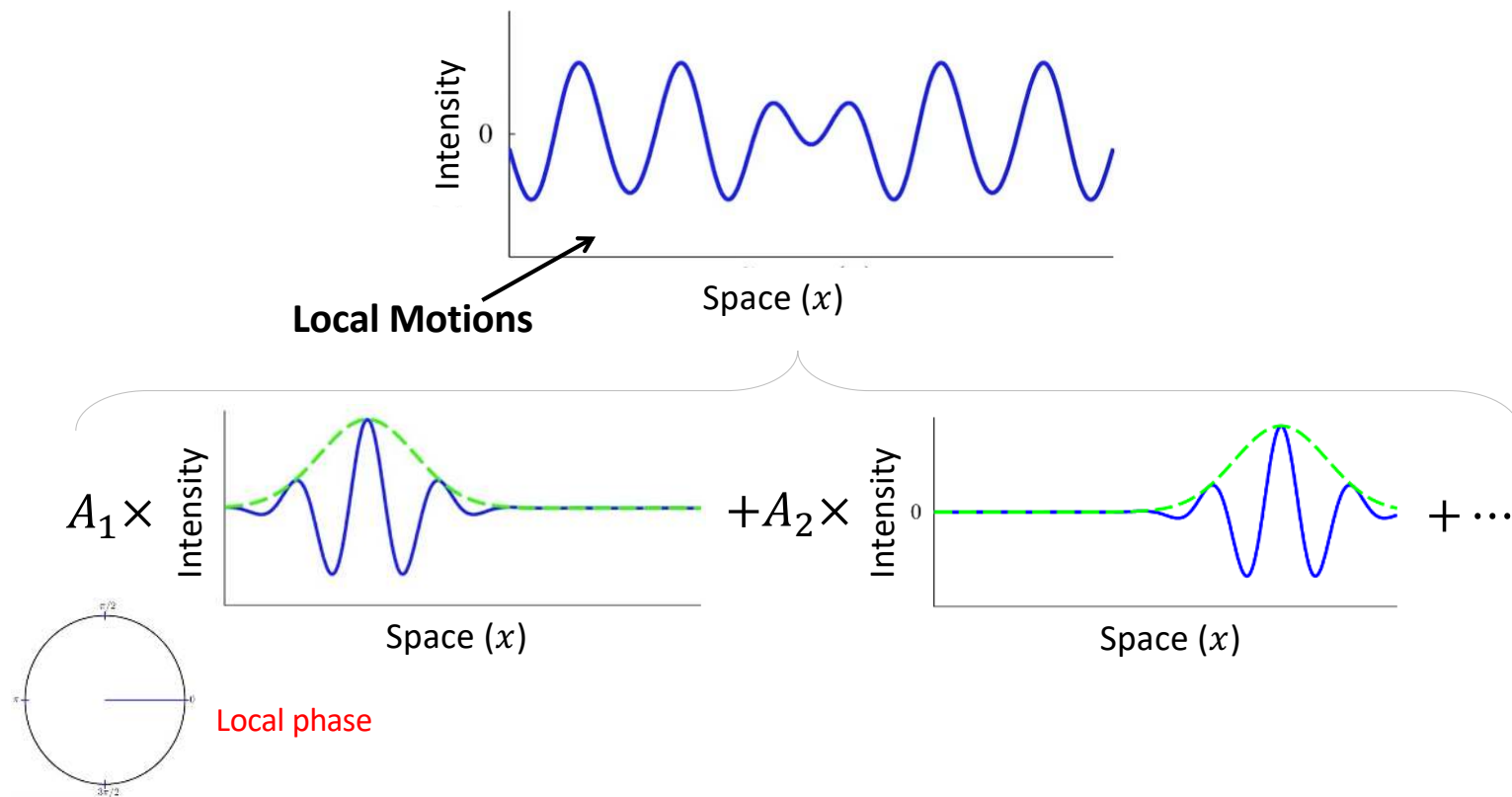
Phase ( $e^{-i\delta}$ )



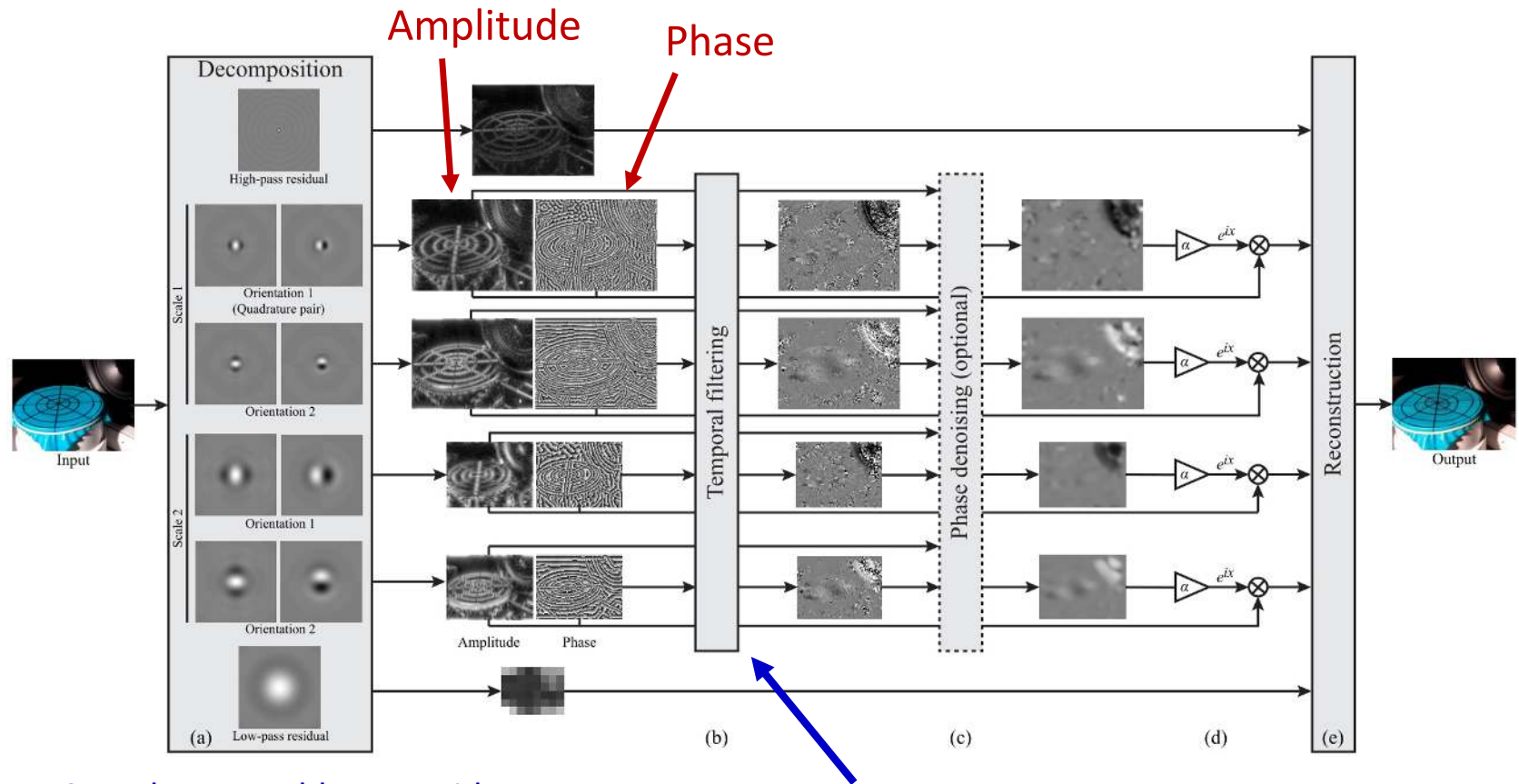
Phase ( $e^{-i2\delta}$ )

# Local Phase Shift

**Local phase shift  $\leftrightarrow$  Local translation**



# Phase-based Motion Processing

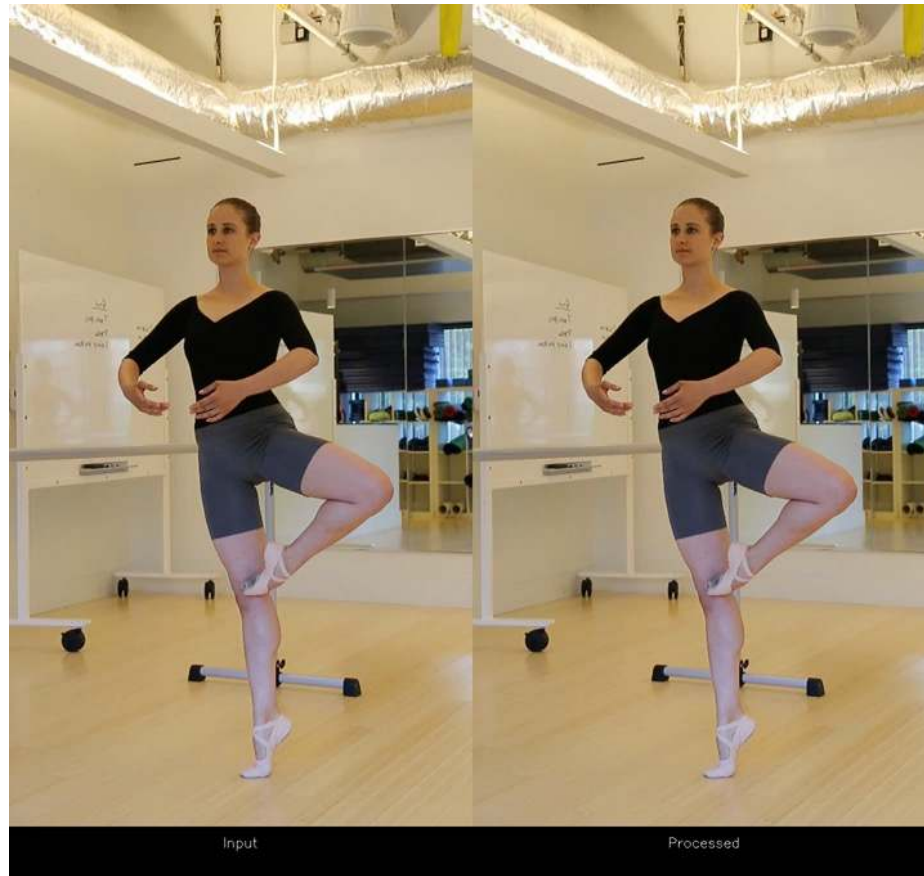


Complex steerable pyramid  
[Simoncelli and Freeman 1995]

Temporal filtering on **phases**

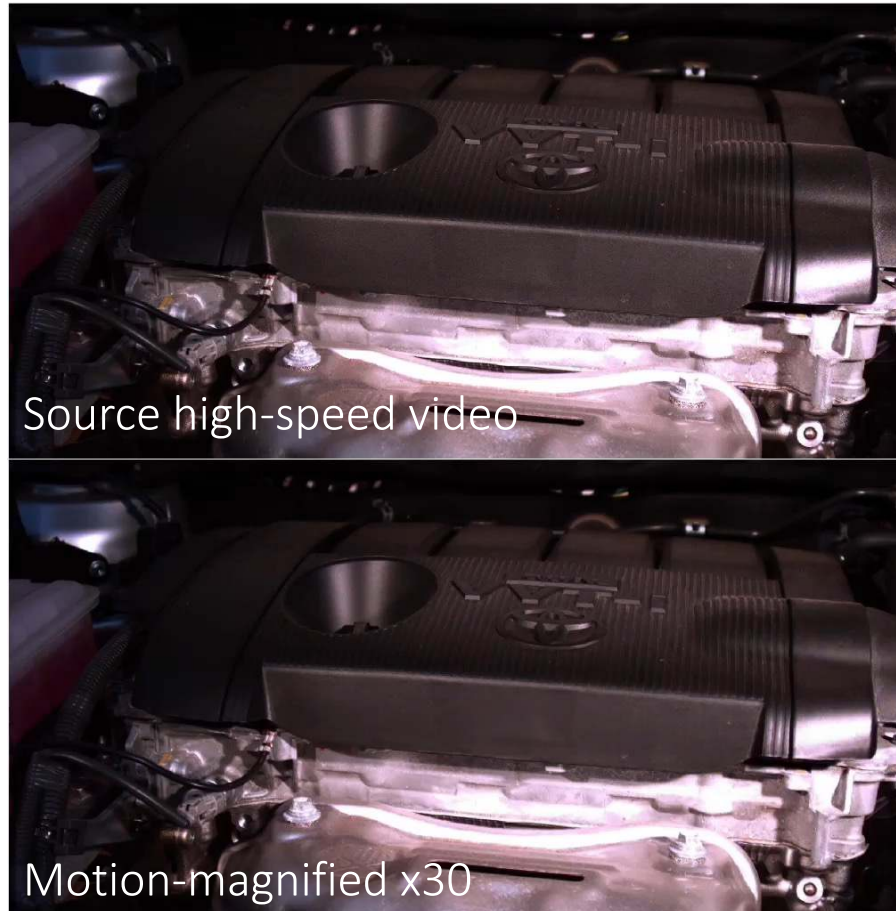
[Wadha et al., SIGGRAPH 2013]

# Phase-based Motion Processing



4-8 Hz amplified by a factor of 10.

# Phase-based Motion Processing



400 fps video of car engine, 30x  
motion magnified, centered at  
engine RPM

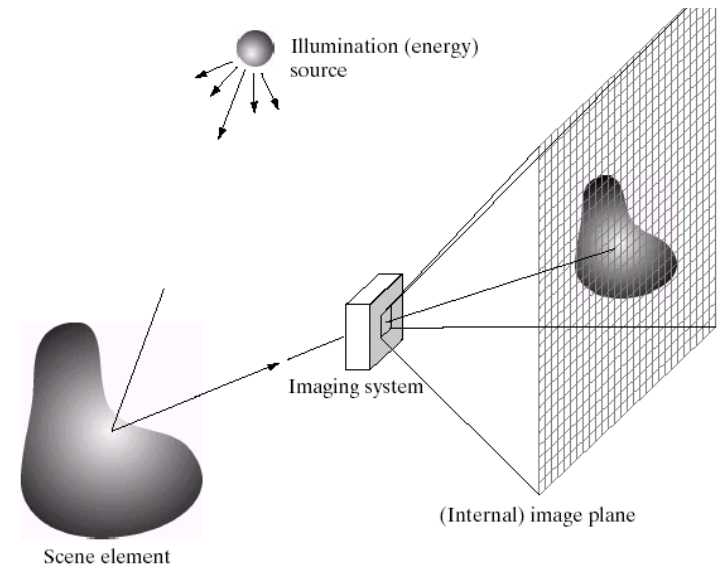
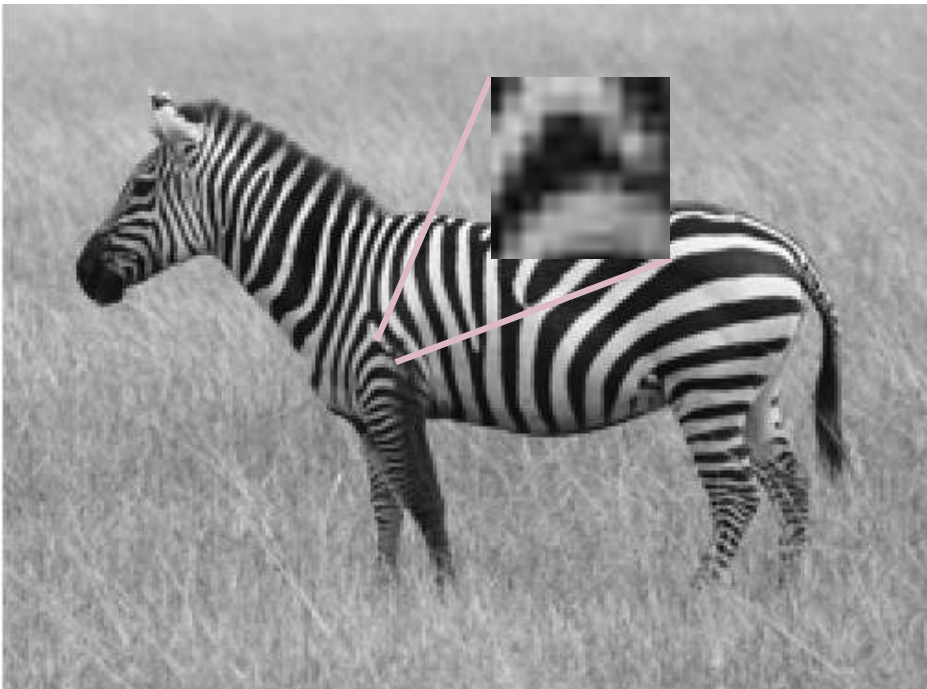
Questions?

# Sampling & Aliasing

## 3+ Main Takeaways

- A linear time-invariant system is fully characterized by its impulse response
- You can represent any signal as a sum of complex exponentials
- Convolution is multiplication in the frequency domain
- If there is time: To prevent aliasing you must sample at twice the maximum frequency in a band-limited signal

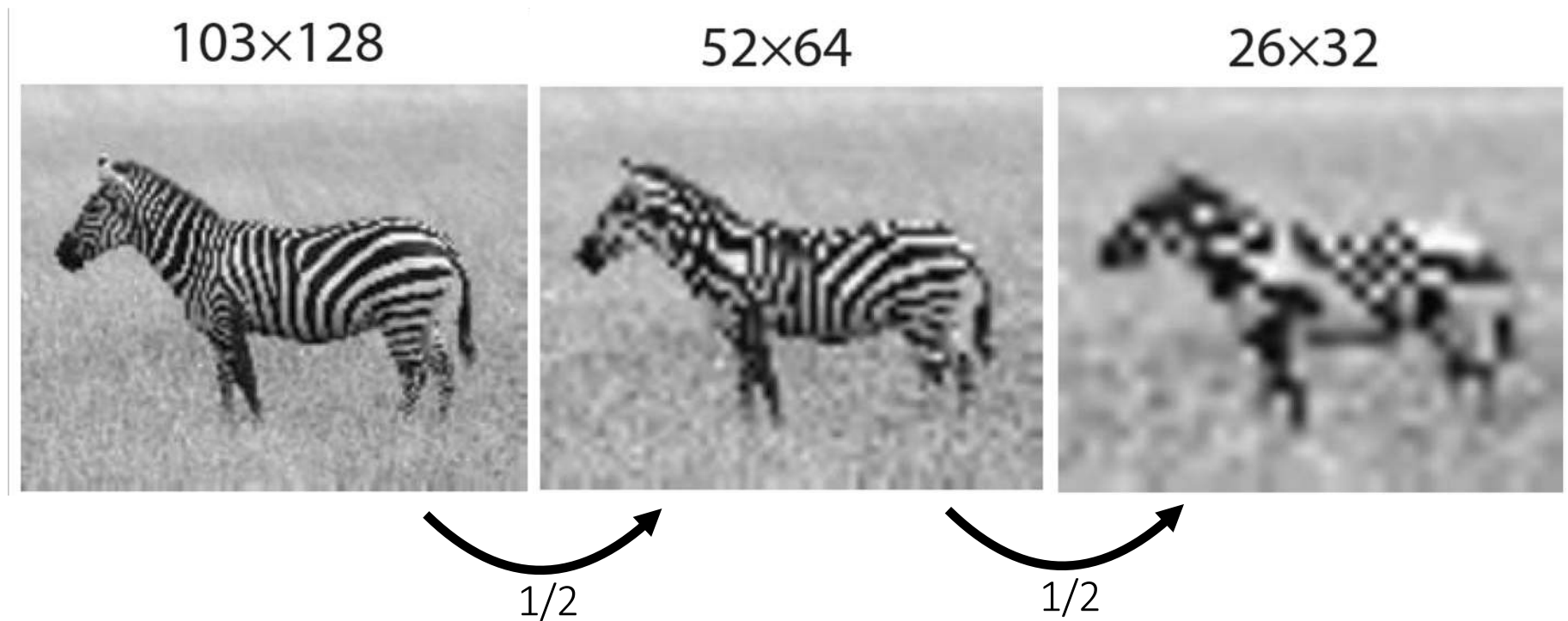
# Reminder



Images are a *discrete*, or *sampled*, representation of a *continuous* world

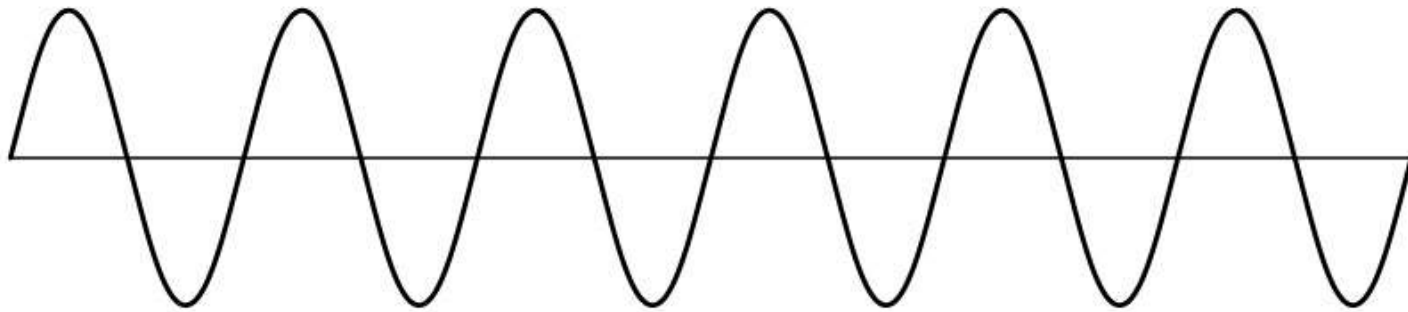
# Subsampling

Can't just subsample an image to  $\frac{1}{2}$  its size due to aliasing



# Sampling

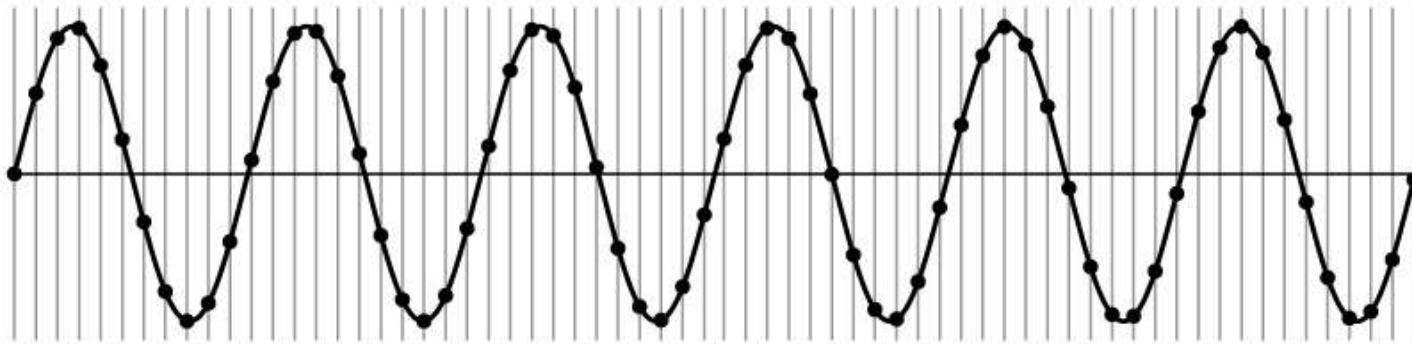
Very simple example: a sine wave



How would you discretize this signal?

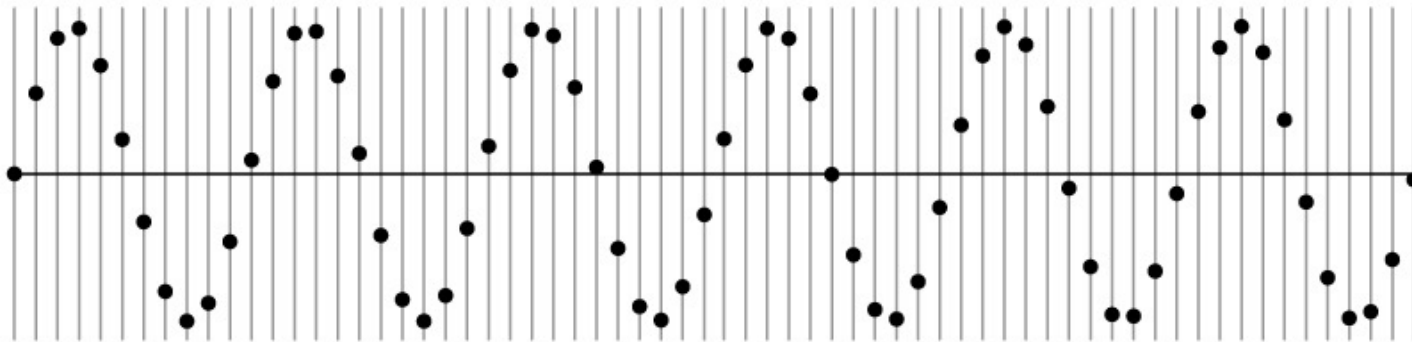
# Sampling

Very simple example: a sine wave



# Sampling

Very simple example: a sine wave

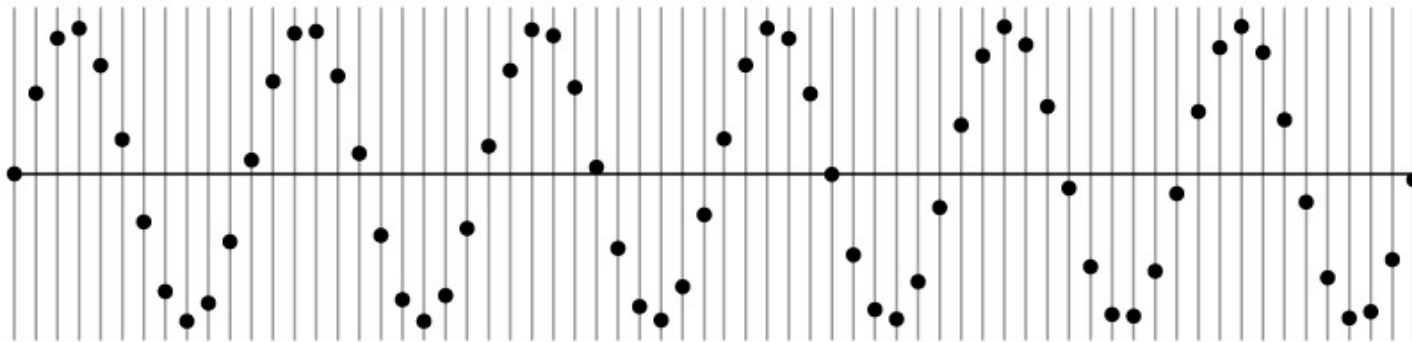


How many samples should I take?

Can I take as *many* samples as I want?

# Sampling

Very simple example: a sine wave

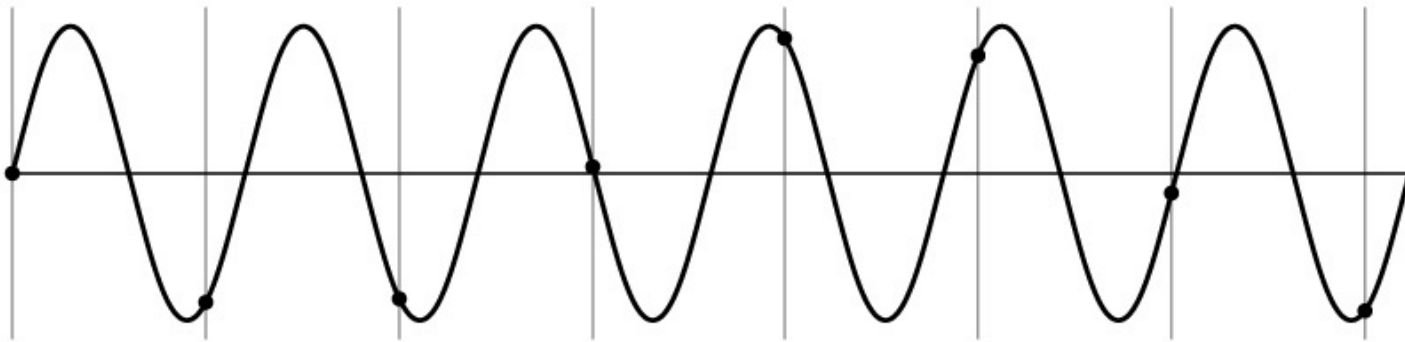


How many samples should I take?

Can I take as *few* samples as I want?

# Undersampling

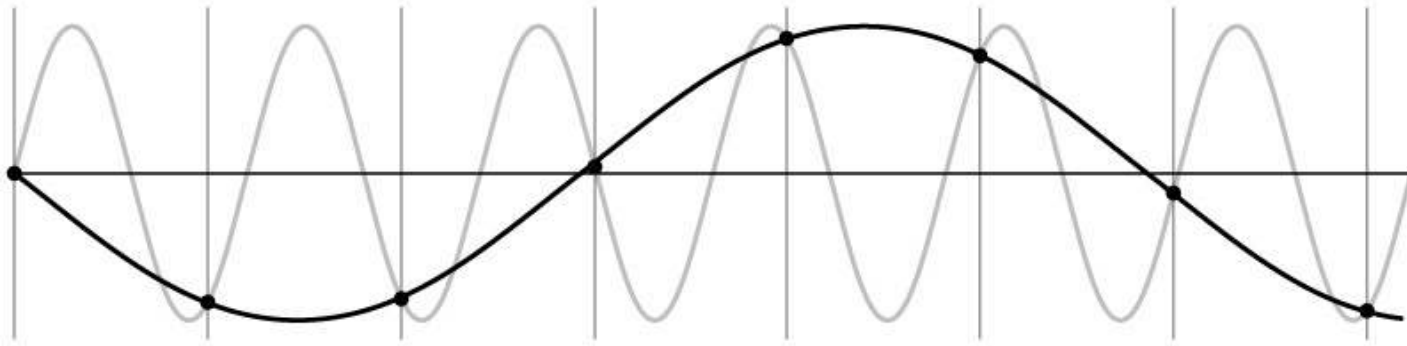
Very simple example: a sine wave



Unsurprising effect: information is lost.

# Undersampling

Very simple example: a sine wave

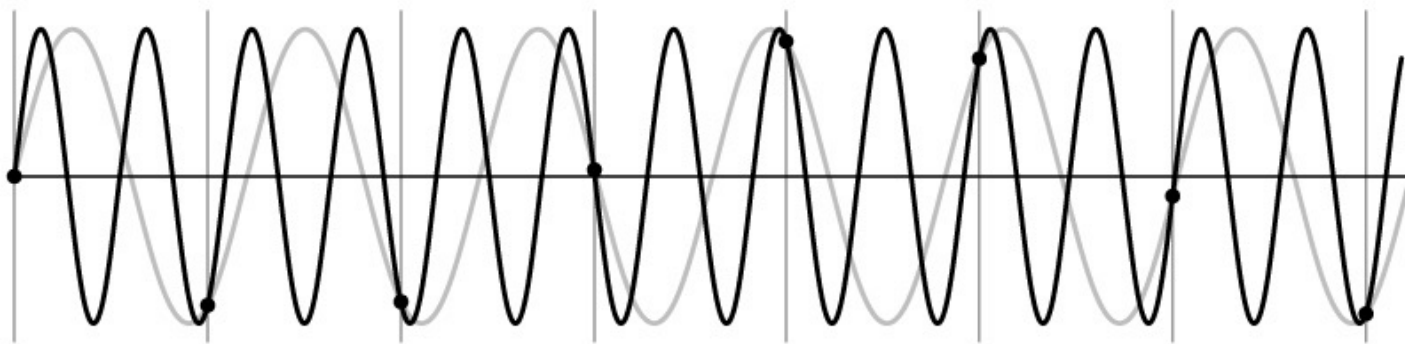


Unsurprising effect: information is lost.

Surprising effect: can confuse the signal with one of *lower* frequency.

# Undersampling

Very simple example: a sine wave



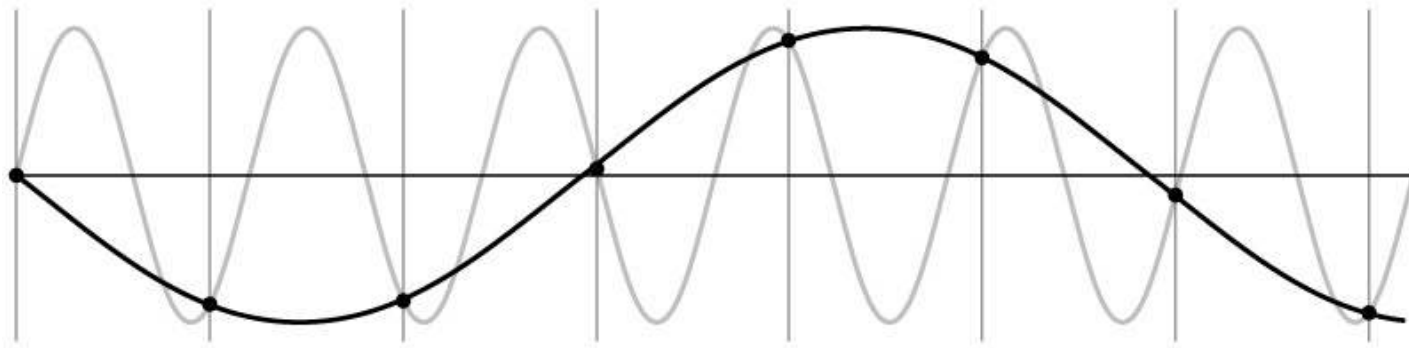
Unsurprising effect: information is lost.

Surprising effect: can confuse the signal with one of *lower* frequency.

Note: we could always confuse the signal with one of *higher* frequency.

# Aliasing

Fancy term for: *Undersampling can disguise a signal as one of a lower frequency*



Unsurprising effect: information is lost.

Surprising effect: can confuse the signal with one of *lower* frequency.

Note: we could always confuse the signal with one of *higher* frequency.

# Aliasing in photographs

This is also known as “moire”

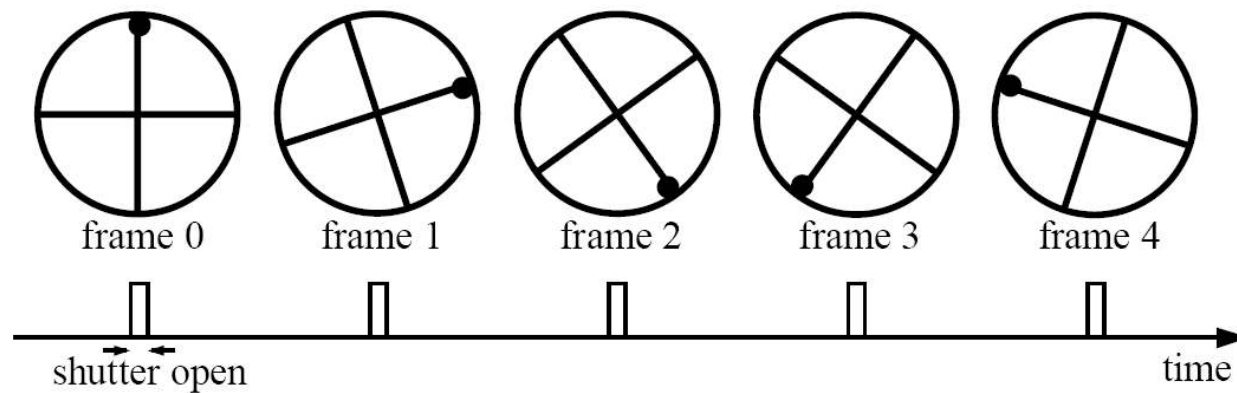


# Temporal aliasing

Imagine a spoked wheel moving to the right (rotating clockwise).

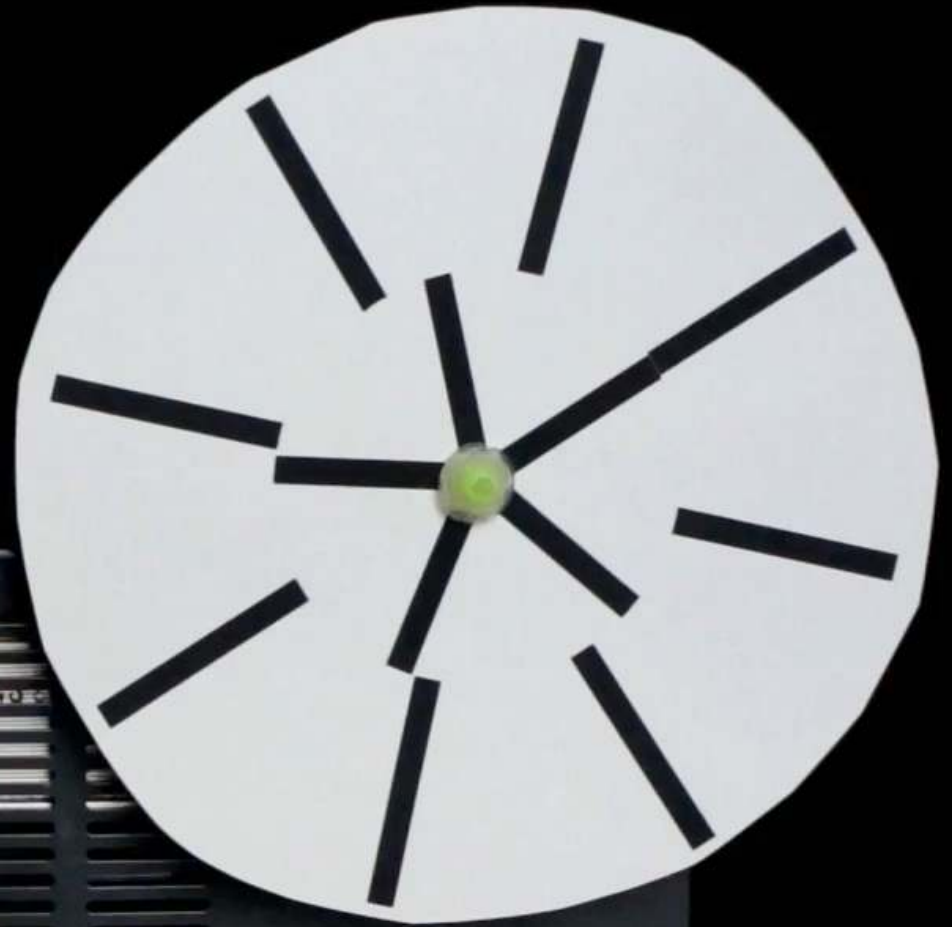
Mark wheel with dot so we can see what's happening.

If camera shutter is only open for a fraction of a frame time (frame time =  $1/30$  sec. for video,  $1/24$  sec. for film):



Without dot, wheel appears to be rotating slowly backwards!  
(counterclockwise)

# Wagon wheel effect





# Anti-aliasing

How would you deal with aliasing?

# Anti-aliasing

How would you deal with aliasing?

Approach 1: Oversample the signal

Approach 2: Smooth the signal

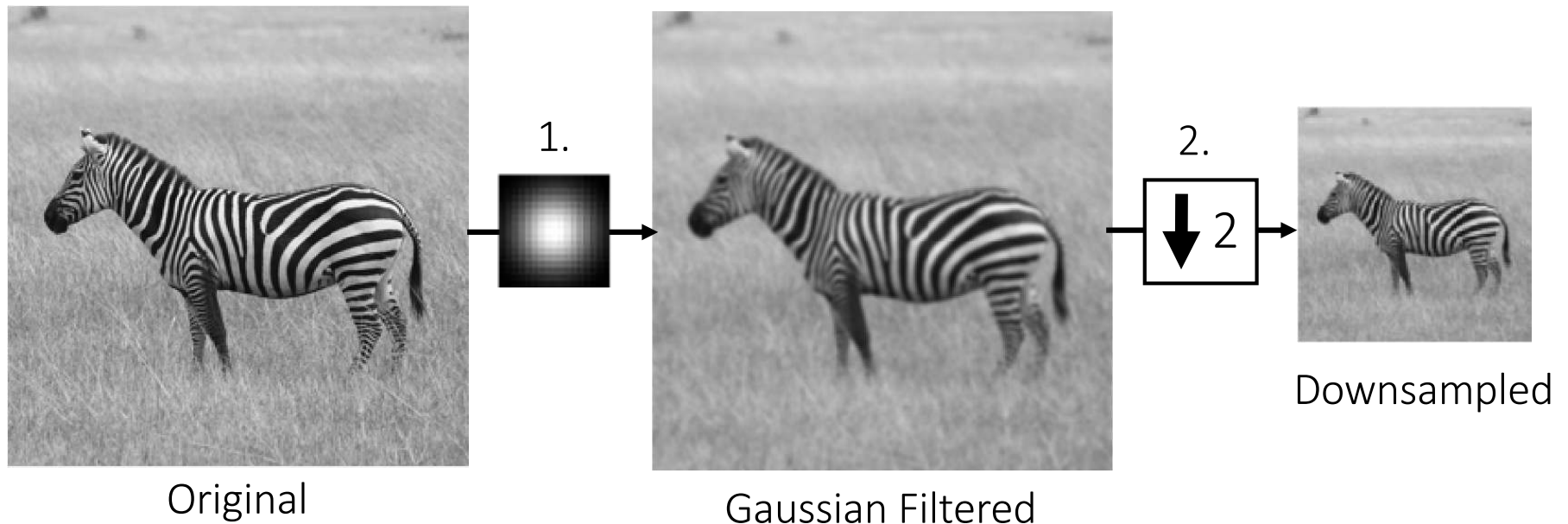
- Remove some of the detail effects that cause aliasing.
- Lose information, but better than aliasing artifacts.

How would you do Approach 2?

# Gaussian Pyramid

For each level:

1. Blur input image with a Gaussian filter
2. Downsample image



# Anti-aliasing

How would you deal with aliasing?

Approach 1: Oversample the signal

Approach 2: Smooth the signal

- Remove some of the detail effects that cause aliasing.
- Lose information, but better than aliasing artifacts.

How do we know how much to sample for Approach 1?

# The Nyquist-Shannon sampling theorem

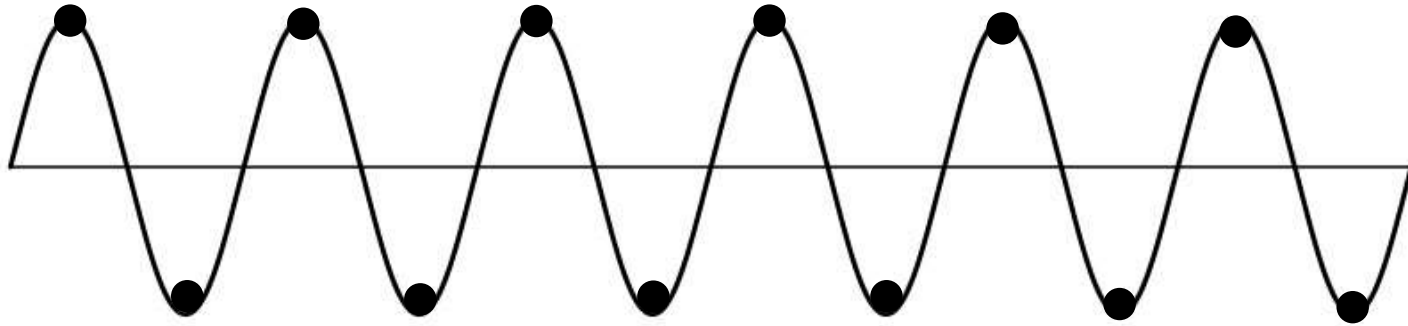
A continuous signal can be perfectly reconstructed from its discrete version using linear interpolation, if sampling occurred with frequency:

$$f_s \geq 2f_{\max} \quad \leftarrow \quad \begin{array}{l} \text{This is called the} \\ \text{Nyquist frequency} \end{array}$$

Equivalent reformulation: When downsampling, aliasing does not occur if samples are taken at the Nyquist frequency or higher.

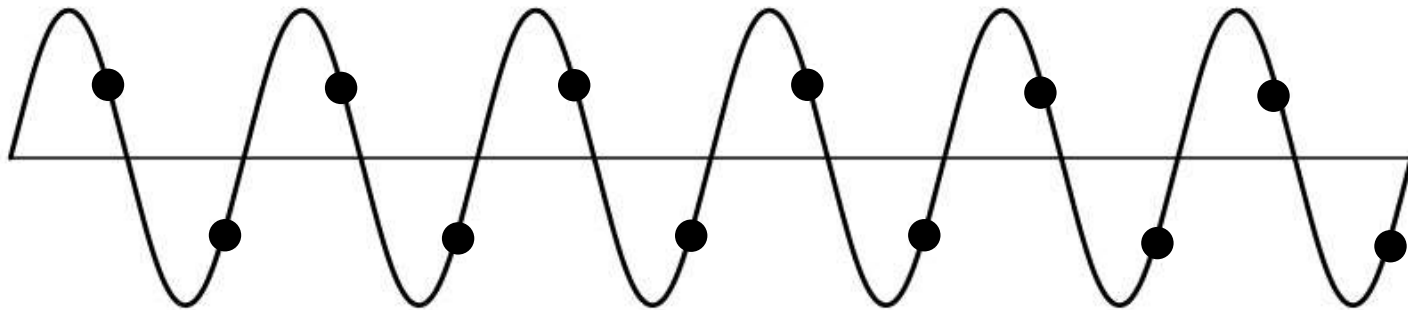
# Nyquist Sampling

Very simple example: a sine wave



# Nyquist Sampling

Very simple example: a sine wave



# The Nyquist-Shannon sampling theorem

A continuous signal can be perfectly reconstructed from its discrete version using linear interpolation, if sampling occurred with frequency:

$$f_s \geq 2f_{\max} \quad \leftarrow \quad \begin{array}{l} \text{This is called the} \\ \text{Nyquist frequency} \end{array}$$

Equivalent reformulation: When downsampling, aliasing does not occur if samples are taken at the Nyquist frequency or higher.

## 3+ Main Takeaways

- A linear time-invariant system is fully characterized by its impulse response
- You can represent any signal as a sum of complex exponentials
- Convolution is multiplication in the frequency domain
- If there is time: To prevent aliasing you must sample at twice the maximum frequency in a band-limited signal

Questions?